RAJALAKSHMI ENGINEERING COLLEGE
An Autonomous Institution

BONAFIDE CERTIFICATE

Name: HARIHARAN S

Academic Year: 2026 - 2027 **Semester:** IV **Branch:** B.E.CSE-H

Register No. 211625070174 3

Certified that this is the bonafide record of work done by the above student in the CS23334 - Fundamentals of Data Science *Laboratory during the academic year 2026 – 2027.*

3/11/26

Signature of Faculty in-charge

Submitted for the Practical Examination held on 06.11.26

Internal Examiner

External Examiner

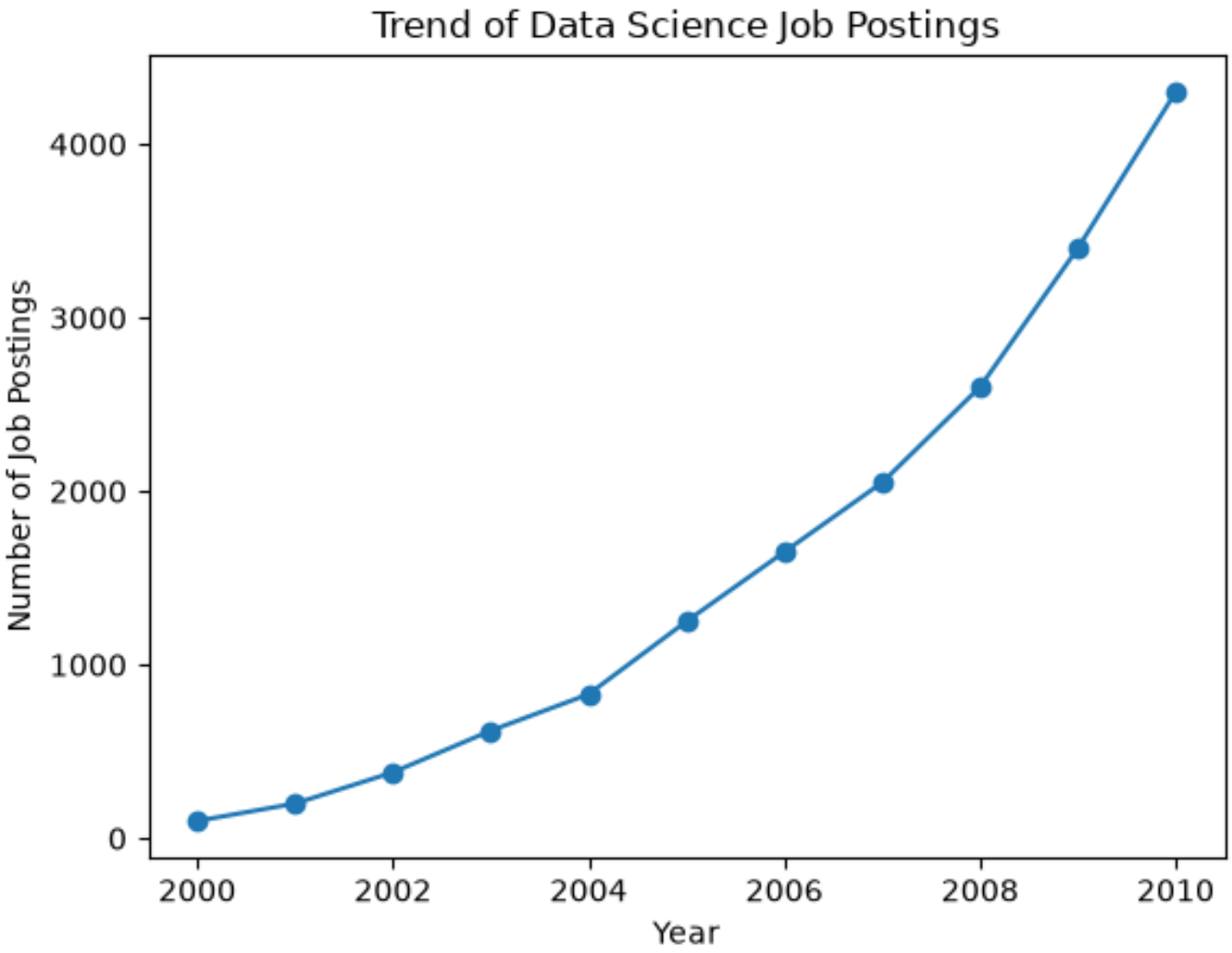
Experiment 1

Code Cell 1

```
import pandas as pd
import matplotlib.pyplot as plt
data = {'Year': list(range(2000, 2011)),
'Job Postings': [100, 200, 380, 620, 830, 1250, 1650, 2050, 2600, 3400, 4300]}

df = pd.DataFrame(data)
plt.plot(df['Year'], df['Job Postings'], marker='o')
plt.title('Trend of Data Science Job Postings')
plt.xlabel('Year')
plt.ylabel('Number of Job Postings')
plt.show()
```

Output



Code Cell 2

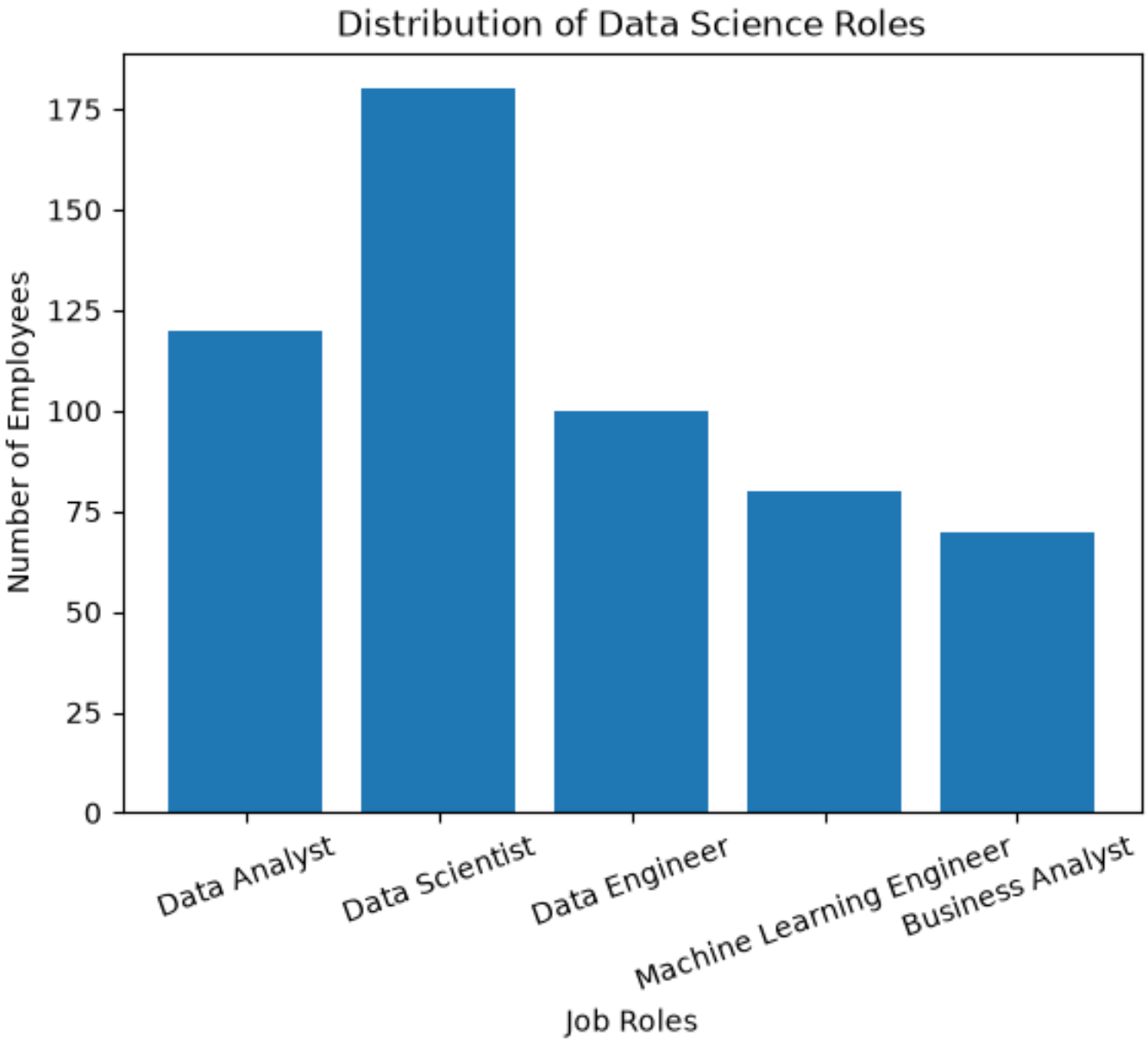
```
import pandas as pd
import matplotlib.pyplot as plt

data = {'Role': ['Data Analyst', 'Data Scientist', 'Data Engineer',
            'Machine Learning Engineer', 'Business Analyst'],
    'Count': [120, 180, 100, 80, 70]}

df = pd.DataFrame(data)

plt.bar(df['Role'], df['Count'])
plt.title('Distribution of Data Science Roles')
plt.xlabel('Job Roles')
plt.ylabel('Number of Employees')
plt.xticks(rotation=20)
plt.show()
```

Output


Distribution of Data Science Roles
Number of Employees
0
25
50
75
100
125
150
175
Data Analyst
Data Scientist
Data Engineer
Machine Learning Engineer
Business Analyst
Job Roles

Code Cell 3

```
import pandas as pd

structured_data = {
    'ID': [1, 2, 3],
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [22, 25, 24]
}

unstructured_data = {
    'Text': [
        'Data Science is interesting.',
        'Python is easy to learn.',
        'Big Data is the future.'
    ]
}

semi_structured_data = {
    'Student': [
        {'ID': 1, 'Name': 'Alice', 'Marks': 90},
        {'ID': 2, 'Name': 'Bob', 'Marks': 85},
        {'ID': 3, 'Name': 'Charlie', 'Marks': 88}
    ]
}

df1 = pd.DataFrame(structured_data)
df2 = pd.DataFrame(unstructured_data)
df3 = pd.DataFrame(semi_structured_data)

print("Structured Data")
print(df1)

print("\nUn-Structured Data")
print(df2)

print("\nSemi-Structured Data")
print(df3)
```

Output

```
Structured Data
   ID     Name  Age
0   1    Alice   22
1   2      Bob   25
2   3  Charlie   24

Un-Structured Data
                          Text
0  Data Science is interesting.
1      Python is easy to learn.
2       Big Data is the future.

Semi-Structured Data
                                      Student
0    {'ID': 1, 'Name': 'Alice', 'Marks': 90}
1      {'ID': 2, 'Name': 'Bob', 'Marks': 85}
2  {'ID': 3, 'Name': 'Charlie', 'Marks': 88}
```

Code Cell 4

```
from cryptography.fernet import Fernet

key = Fernet.generate_key()

cipher = Fernet(key)

data = "My Password is 12345"

encrypted_data = cipher.encrypt(data.encode())

decrypted_data = cipher.decrypt(encrypted_data).decode()

print("Original Data:", data)
print("Encrypted Data:", encrypted_data)
print("Decrypted Data:", decrypted_data)
```

Output

```
Original Data: My Password is 12345
Encrypted Data: b'gAAAAABqchCYVomNZvnSmhIqoNtfiWzjr3wngCcQ0D24HN7Kl1V3r-lpZ4dVYA--ywzU5jjO0z0OEu-
DdlzTqLa5QyY6AbjHlc_FtuDBYYQgqd5fQNtDdpI='
Decrypted Data: My Password is 12345
```

Experiment 2

Code Cell 1

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

Code Cell 2

```
file_path = 'sales_data_practical.csv'
df = pd.read_csv(file_path)
```

Code Cell 3

```
print(df.head())
```

Output

```
         Date     Product    Sales  Quantity Region
0  2024-01-05      Laptop  75000.0         3  North
1  2024-01-12       Mouse   2500.0        10  South
2  2024-02-03    Keyboard   4500.0         8   East
3  2024-02-18      Laptop  68000.0         2   West
4  2024-03-07  Headphones   6000.0         6  South
```

Code Cell 4

```
print(df.isnull().sum())
```

Output

```
Date        0
Product     1
Sales       1
Quantity    0
Region      1
dtype: int64
```

Code Cell 5

```
df['Sales'] = df['Sales'].fillna(df['Sales'].mean())
```

Code Cell 6

```
df.dropna(subset=['Product', 'Quantity', 'Region'], inplace=True)
```

Code Cell 7

```
print(df.isnull().sum())
```

Output

```
Date        0
Product     0
Sales       0
Quantity    0
Region      0
dtype: int64
```

Code Cell 8

```
print(df.describe())
```

Output

```
              Sales   Quantity
count     18.000000  18.000000
mean   21722.222222   7.277778
std    25069.727598   3.250440
min     2500.000000   2.000000
25%     4850.000000   5.000000
50%     8350.000000   7.500000
75%    29125.000000   9.750000
max    75000.000000  13.000000
```

Code Cell 9

```
product_summary = df.groupby('Product').agg({
    'Sales': 'sum',
    'Quantity': 'sum'
}).reset_index()

print(product_summary)
```

Output

```
      Product     Sales  Quantity
0  Headphones   31700.0        27
1    Keyboard   19500.0        36
2      Laptop  215000.0         8
3     Monitor   88500.0        14
4       Mouse   36300.0        46
```

Code Cell 10

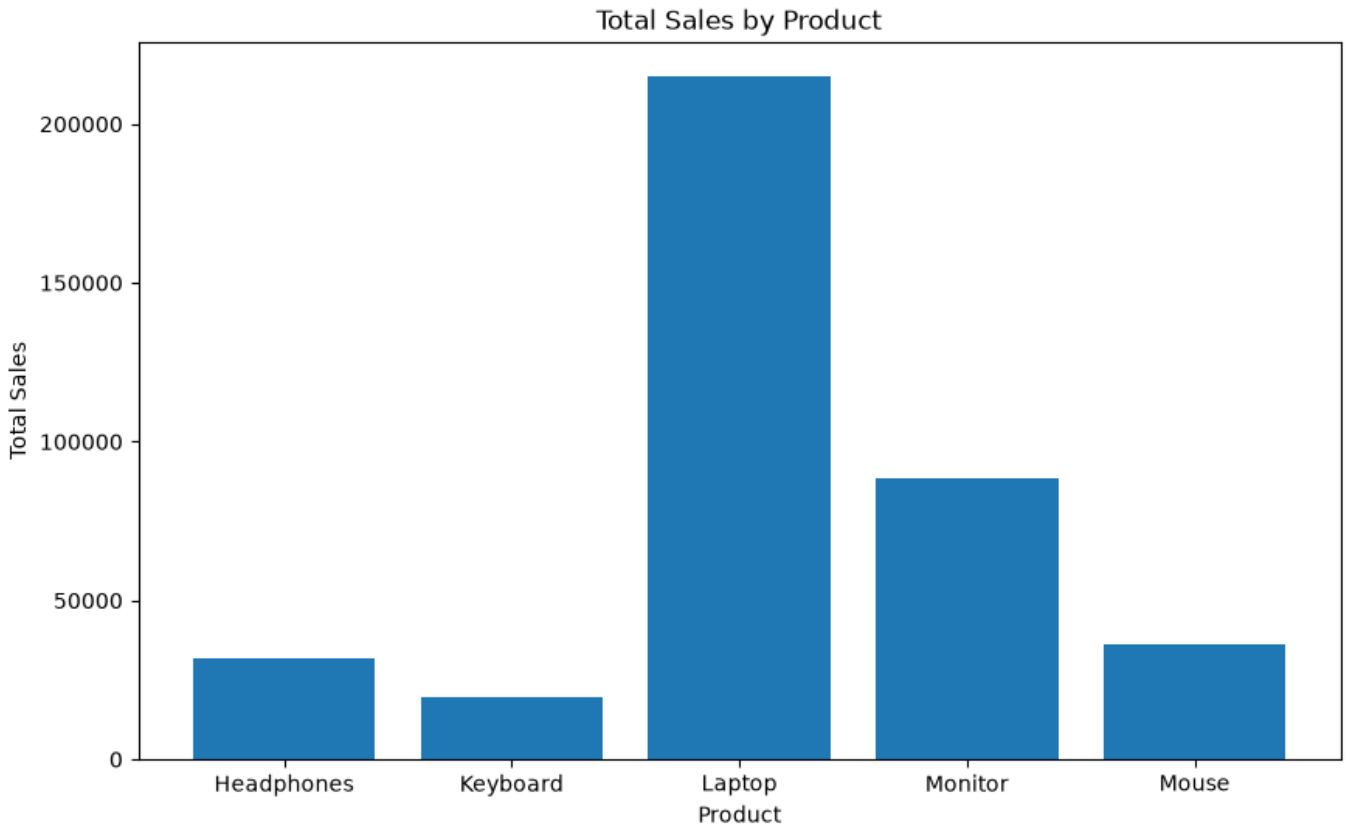
```
plt.figure(figsize=(10, 6))

plt.bar(product_summary['Product'],
        product_summary['Sales'])

plt.xlabel('Product')
plt.ylabel('Total Sales')
plt.title('Total Sales by Product')

plt.show()
```

Output



Code Cell 11

```
df['Date'] = pd.to_datetime(df['Date'])
```

Code Cell 12

```
sales_over_time = df.groupby('Date').agg({
    'Sales': 'sum'
}).reset_index()

print(sales_over_time)
```

Output

```
         Date    Sales
0  2024-01-05  75000.0
1  2024-01-12   2500.0
2  2024-02-03   4500.0
3  2024-02-18  68000.0
4  2024-03-07   6000.0
5  2024-03-21  27400.0
6  2024-04-04   5000.0
7  2024-04-19  72000.0
8  2024-05-06  28000.0
9  2024-05-22   8500.0
10 2024-06-08   2900.0
11 2024-07-10   5200.0
12 2024-07-27  31000.0
13 2024-08-09   9000.0
14 2024-08-24   3500.0
15 2024-09-28   4800.0
16 2024-10-13  29500.0
17 2024-10-30   8200.0
```

Code Cell 13

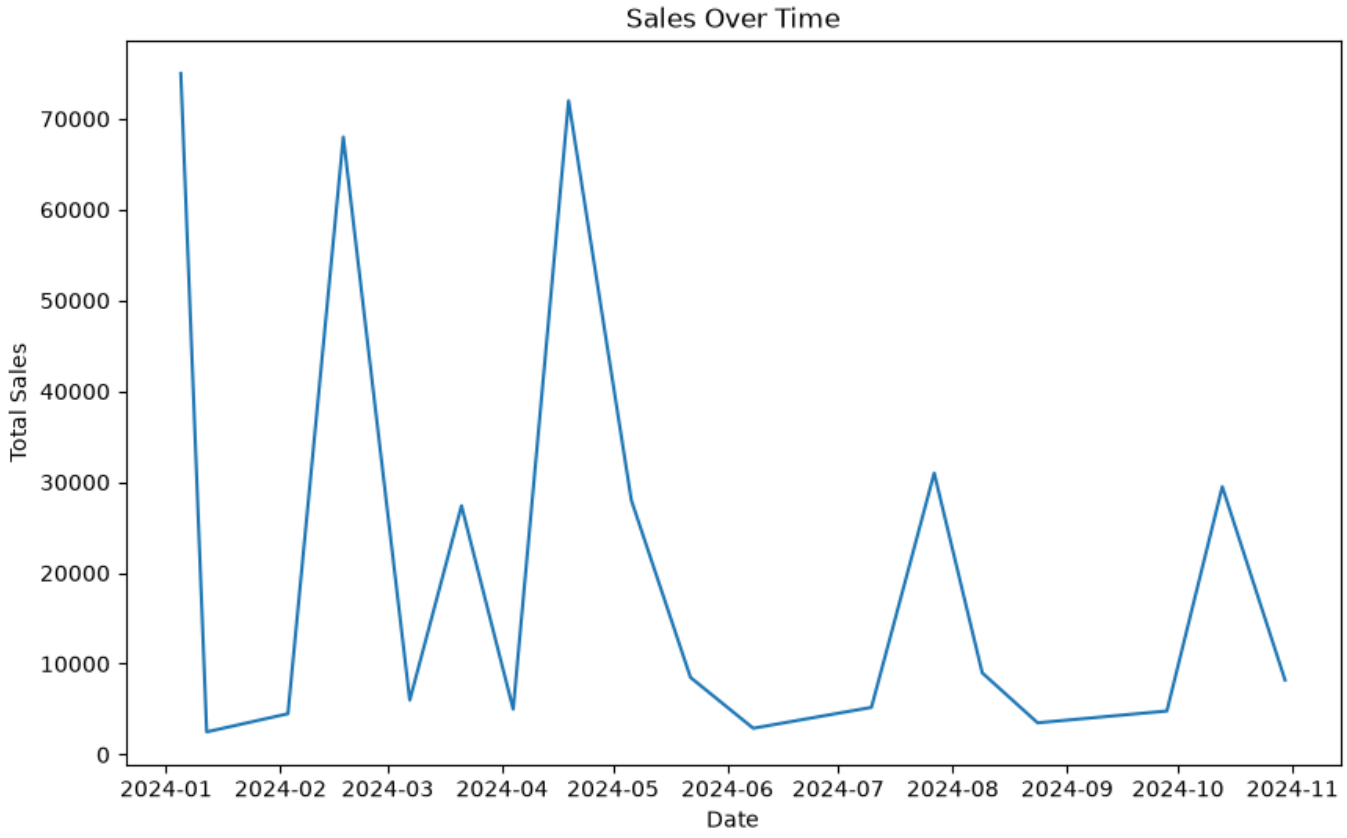
```
plt.figure(figsize=(10, 6))

plt.plot(sales_over_time['Date'],
         sales_over_time['Sales'])

plt.xlabel('Date')
plt.ylabel('Total Sales')
plt.title('Sales Over Time')

plt.show()
```

Output



Code Cell 14

```
pivot_table = df.pivot_table(
    values='Sales',
    index='Region',
    columns='Product',
    aggfunc='sum',
    fill_value=0
)

print(pivot_table)
```

Output

```
Product  Headphones  Keyboard   Laptop  Monitor    Mouse
Region
East         8500.0   14300.0      0.0  31000.0      0.0
North           0.0       0.0  75000.0  29500.0  30900.0
South       23200.0       0.0  72000.0      0.0   5400.0
West            0.0    5200.0  68000.0  28000.0      0.0
```

Code Cell 15

```
correlation_matrix = df[['Sales', 'Quantity']].corr()

print(correlation_matrix)
```

Output

```
             Sales  Quantity
Sales     1.000000 -0.743535
Quantity -0.743535  1.000000
```

Code Cell 16

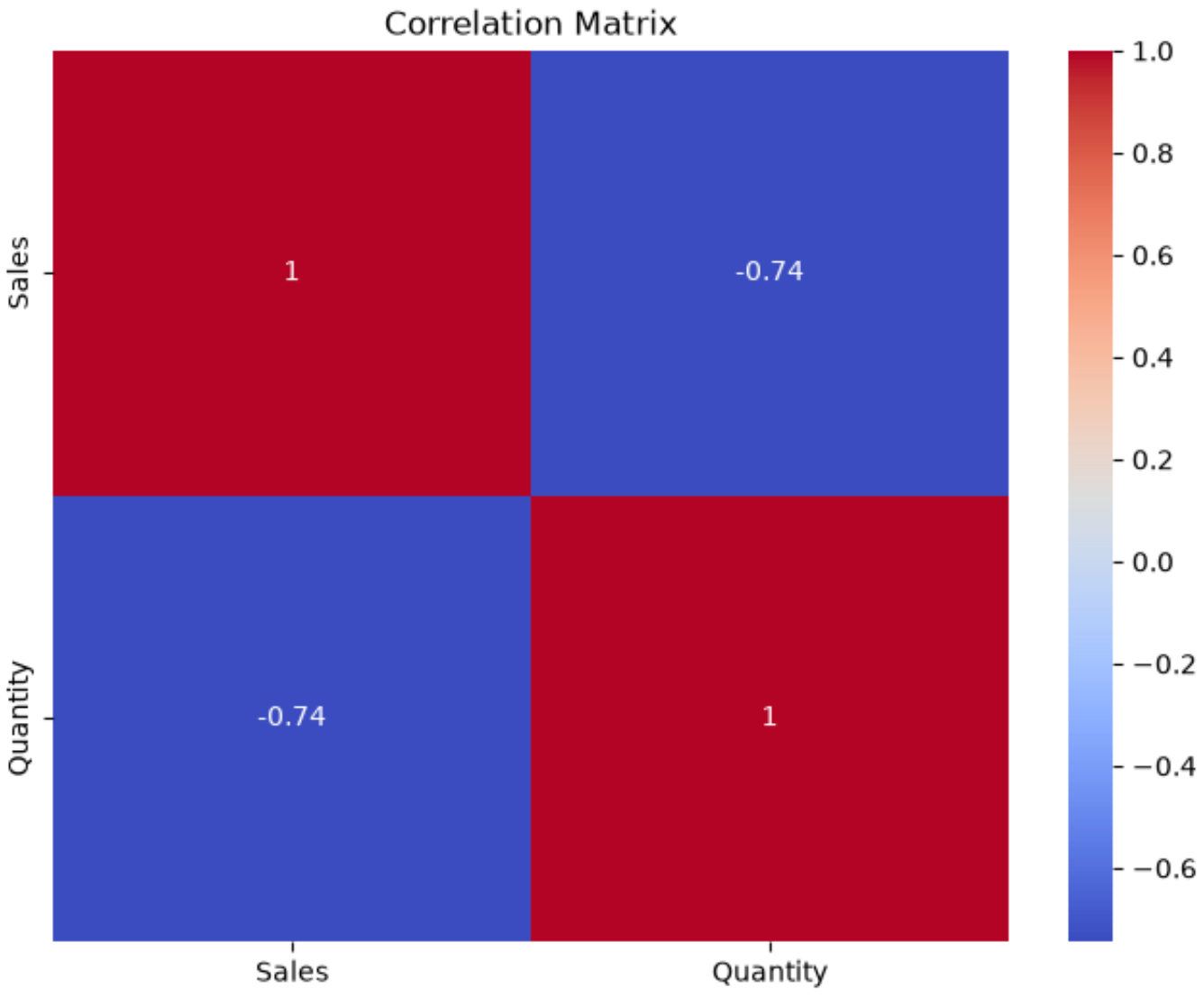
```
import seaborn as sns

plt.figure(figsize=(8, 6))

sns.heatmap(
    correlation_matrix,
    annot=True,
    cmap='coolwarm'
)

plt.title('Correlation Matrix')
plt.show()
```

Output



Experiment 3

Code Cell 1

```
import numpy as np
import pandas as pd
```

Code Cell 2

```
df = pd.read_csv("Hotel_Dataset.csv")
df
```

Output

```
    CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference  Bill  \
0            1     20-25            4       Ibis            veg  1300
1            2     30-35            5  LemonTree        Non-Veg  2000
2            3     25-30            6     RedFox            Veg  1322
3            4     20-25           -1  LemonTree            Veg  1234
4            5       35+            3       Ibis     Vegetarian   989
5            6       35+            3       Ibys        Non-Veg  1909
6            7       35+            4     RedFox     Vegetarian  1000
7            8     20-25            7  LemonTree            Veg  2999
8            9     25-30            2       Ibis        Non-Veg  3456
9            9     25-30            2       Ibis        Non-Veg  3456
10          10     30-35            5     RedFox        non-Veg -6755

    NoOfPax  EstimatedSalary Age_Group.1
0         2           40000       20-25
1         3           59000       30-35
2         2           30000       25-30
3         2          120000       20-25
4         2           45000         35+
5         2          122220         35+
6        -1           21122         35+
7       -10          345673       20-25
8         3          -99999       25-30
9         3          -99999       25-30
10        4           87777       30-35
```

Code Cell 3

```
df.duplicated()
```

Output

```
0     False
1     False
2     False
3     False
4     False
5     False
6     False
7     False
8     False
9      True
10    False
dtype: bool
```

Code Cell 4

```
df = df.drop_duplicates()
df
```

Output

```
    CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference  Bill  \
0            1     20-25            4       Ibis            veg  1300
1            2     30-35            5  LemonTree        Non-Veg  2000
2            3     25-30            6    RedFox            Veg  1322
3            4     20-25           -1  LemonTree            Veg  1234
4            5       35+            3       Ibis     Vegetarian   989
5            6       35+            3       Ibys        Non-Veg  1909
6            7       35+            4    RedFox     Vegetarian  1000
7            8     20-25            7  LemonTree            Veg  2999
8            9     25-30            2       Ibis        Non-Veg  3456
10          10     30-35            5    RedFox        non-Veg -6755

    NoOfPax  EstimatedSalary Age_Group.1
0         2           40000       20-25
1         3           59000       30-35
2         2           30000       25-30
3         2          120000       20-25
4         2           45000         35+
5         2          122220         35+
6        -1           21122         35+
7       -10          345673       20-25
8         3          -99999       25-30
10        4           87777       30-35
```

Code Cell 5

```
len(df)
```

Output

```
10
```

Code Cell 6

```
index = np.array(list(range(0, len(df))))
df = df.set_index(index)
df
```

Output

```
   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference  Bill  NoOfPax  \
0           1     20-25            4       Ibis            veg  1300        2
1           2     30-35            5  LemonTree        Non-Veg  2000        3
2           3     25-30            6    RedFox            Veg  1322        2
3           4     20-25           -1  LemonTree            Veg  1234        2
4           5       35+            3       Ibis     Vegetarian   989        2
5           6       35+            3       Ibys        Non-Veg  1909        2
6           7       35+            4    RedFox     Vegetarian  1000       -1
7           8     20-25            7  LemonTree            Veg  2999      -10
8           9     25-30            2       Ibis        Non-Veg  3456        3
9          10     30-35            5    RedFox        non-Veg -6755        4

   EstimatedSalary Age_Group.1
0            40000       20-25
1            59000       30-35
2            30000       25-30
3           120000       20-25
4            45000         35+
5           122220         35+
6            21122         35+
7           345673       20-25
8           -99999       25-30
9            87777       30-35
```

Code Cell 7

```
df.info()
```

Output

```
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 9 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   CustomerID       10 non-null     int64
 1   Age_Group        10 non-null     str
 2   Rating(1-5)      10 non-null     int64
 3   Hotel            10 non-null     str
 4   FoodPreference   10 non-null     str
 5   Bill             10 non-null     int64
 6   NoOfPax          10 non-null     int64
 7   EstimatedSalary  10 non-null     int64
 8   Age_Group.1      10 non-null     str
dtypes: int64(5), str(4)
memory usage: 1.0 KB
```

Code Cell 8

```
df = df.drop(columns=['Age_Group.1'])
df
```

Output

```
   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference  Bill  NoOfPax  \
0           1     20-25            4       Ibis            veg  1300        2
1           2     30-35            5  LemonTree        Non-Veg  2000        3
2           3     25-30            6     RedFox            Veg  1322        2
3           4     20-25           -1  LemonTree            Veg  1234        2
4           5       35+            3       Ibis     Vegetarian   989        2
5           6       35+            3       Ibys        Non-Veg  1909        2
6           7       35+            4     RedFox     Vegetarian  1000       -1
7           8     20-25            7  LemonTree            Veg  2999      -10
8           9     25-30            2       Ibis        Non-Veg  3456        3
9          10     30-35            5     RedFox        non-Veg -6755        4

   EstimatedSalary
0            40000
1            59000
2            30000
3           120000
4            45000
5           122220
6            21122
7           345673
8           -99999
9            87777
```

Code Cell 9

```
df.loc[df['CustomerID'] < 0, 'CustomerID'] = np.nan

df.loc[df['Bill'] < 0, 'Bill'] = np.nan

df.loc[df['EstimatedSalary'] < 0, 'EstimatedSalary'] = np.nan

df
```

Output

```
   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference    Bill  \
0         1.0     20-25            4       Ibis            veg  1300.0
1         2.0     30-35            5  LemonTree        Non-Veg  2000.0
2         3.0     25-30            6     RedFox            Veg  1322.0
3         4.0     20-25           -1  LemonTree            Veg  1234.0
4         5.0       35+            3       Ibis     Vegetarian   989.0
5         6.0       35+            3       Ibys        Non-Veg  1909.0
6         7.0       35+            4     RedFox     Vegetarian  1000.0
7         8.0     20-25            7  LemonTree            Veg  2999.0
8         9.0     25-30            2       Ibis        Non-Veg  3456.0
9        10.0     30-35            5     RedFox        non-Veg     NaN

   NoOfPax  EstimatedSalary
0        2          40000.0
1        3          59000.0
2        2          30000.0
3        2         120000.0
4        2          45000.0
5        2         122220.0
6       -1          21122.0
7      -10         345673.0
8        3              NaN
9        4          87777.0
```

Code Cell 10

```
df.loc[
    (df['NoOfPax'] < 1) | (df['NoOfPax'] > 20),
    'NoOfPax'
] = np.nan

df
```

Output

```
   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference    Bill  \
0         1.0     20-25            4       Ibis            veg  1300.0
1         2.0     30-35            5  LemonTree        Non-Veg  2000.0
2         3.0     25-30            6    RedFox            Veg  1322.0
3         4.0     20-25           -1  LemonTree            Veg  1234.0
4         5.0       35+            3       Ibis     Vegetarian   989.0
5         6.0       35+            3       Ibys        Non-Veg  1909.0
6         7.0       35+            4    RedFox     Vegetarian  1000.0
7         8.0     20-25            7  LemonTree            Veg  2999.0
8         9.0     25-30            2       Ibis        Non-Veg  3456.0
9        10.0     30-35            5    RedFox        non-Veg     NaN

   NoOfPax  EstimatedSalary
0      2.0          40000.0
1      3.0          59000.0
2      2.0          30000.0
3      2.0         120000.0
4      2.0          45000.0
5      2.0         122220.0
6      NaN          21122.0
7      NaN         345673.0
8      3.0              NaN
9      4.0          87777.0
```

Code Cell 11

```
df['Age_Group'].unique()
```

Output

```
<ArrowStringArray>
['20-25', '30-35', '25-30', '35+']
Length: 4, dtype: str
```

Code Cell 12

```
df['Hotel'].unique()
```

Output

```
<ArrowStringArray>
['Ibis', 'LemonTree', 'RedFox', 'Ibys']
Length: 4, dtype: str
```

Code Cell 13

```
df['Hotel'] = df['Hotel'].replace('Ibys', 'Ibis')
df
```

Output

```
   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference    Bill  \
0         1.0     20-25            4       Ibis            veg  1300.0
1         2.0     30-35            5  LemonTree        Non-Veg  2000.0
2         3.0     25-30            6    RedFox            Veg  1322.0
3         4.0     20-25           -1  LemonTree            Veg  1234.0
4         5.0       35+            3       Ibis     Vegetarian   989.0
5         6.0       35+            3       Ibis        Non-Veg  1909.0
6         7.0       35+            4    RedFox     Vegetarian  1000.0
7         8.0     20-25            7  LemonTree            Veg  2999.0
8         9.0     25-30            2       Ibis        Non-Veg  3456.0
9        10.0     30-35            5    RedFox        non-Veg     NaN

   NoOfPax  EstimatedSalary
0      2.0          40000.0
1      3.0          59000.0
2      2.0          30000.0
3      2.0         120000.0
4      2.0          45000.0
5      2.0         122220.0
6      NaN          21122.0
7      NaN         345673.0
8      3.0              NaN
9      4.0          87777.0
```

Code Cell 14

```
df['FoodPreference'].unique()
```

Output

```
<ArrowStringArray>
['veg', 'Non-Veg', 'Veg', 'Vegetarian', 'non-Veg']
Length: 5, dtype: str
```

Code Cell 15

```
df['FoodPreference'] = df['FoodPreference'].replace({
    'veg': 'Veg',
    'Vegetarian': 'Veg',
    'non-Veg': 'Non-Veg'
})

df
```

Output

```
   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference    Bill  \
0         1.0     20-25            4       Ibis            Veg  1300.0
1         2.0     30-35            5  LemonTree        Non-Veg  2000.0
2         3.0     25-30            6     RedFox            Veg  1322.0
3         4.0     20-25           -1  LemonTree            Veg  1234.0
4         5.0       35+            3       Ibis            Veg   989.0
5         6.0       35+            3       Ibis        Non-Veg  1909.0
6         7.0       35+            4     RedFox            Veg  1000.0
7         8.0     20-25            7  LemonTree            Veg  2999.0
8         9.0     25-30            2       Ibis        Non-Veg  3456.0
9        10.0     30-35            5     RedFox        Non-Veg     NaN

   NoOfPax  EstimatedSalary
0      2.0          40000.0
1      3.0          59000.0
2      2.0          30000.0
3      2.0         120000.0
4      2.0          45000.0
5      2.0         122220.0
6      NaN          21122.0
7      NaN         345673.0
8      3.0              NaN
9      4.0          87777.0
```

Code Cell 16

```
df['FoodPreference'].unique()
```

Output

```
<ArrowStringArray>
['Veg', 'Non-Veg']
Length: 2, dtype: str
```

Code Cell 17

```
df['EstimatedSalary'] = df['EstimatedSalary'].fillna(
    round(df['EstimatedSalary'].mean())
)

df
```

Output

```
   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference    Bill  \
0         1.0     20-25            4       Ibis            Veg  1300.0
1         2.0     30-35            5  LemonTree        Non-Veg  2000.0
2         3.0     25-30            6     RedFox            Veg  1322.0
3         4.0     20-25           -1  LemonTree            Veg  1234.0
4         5.0       35+            3       Ibis            Veg   989.0
5         6.0       35+            3       Ibis        Non-Veg  1909.0
6         7.0       35+            4     RedFox            Veg  1000.0
7         8.0     20-25            7  LemonTree            Veg  2999.0
8         9.0     25-30            2       Ibis        Non-Veg  3456.0
9        10.0     30-35            5     RedFox        Non-Veg     NaN

   NoOfPax  EstimatedSalary
0      2.0          40000.0
1      3.0          59000.0
2      2.0          30000.0
3      2.0         120000.0
4      2.0          45000.0
5      2.0         122220.0
6      NaN          21122.0
7      NaN         345673.0
8      3.0          96755.0
9      4.0          87777.0
```

Code Cell 18

```
df['NoOfPax'] = df['NoOfPax'].fillna(
    round(df['NoOfPax'].median())
)

df
```

Output

```
   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference    Bill  \
0         1.0     20-25            4       Ibis            Veg  1300.0
1         2.0     30-35            5  LemonTree        Non-Veg  2000.0
2         3.0     25-30            6    RedFox            Veg  1322.0
3         4.0     20-25           -1  LemonTree            Veg  1234.0
4         5.0       35+            3       Ibis            Veg   989.0
5         6.0       35+            3       Ibis        Non-Veg  1909.0
6         7.0       35+            4    RedFox            Veg  1000.0
7         8.0     20-25            7  LemonTree            Veg  2999.0
8         9.0     25-30            2       Ibis        Non-Veg  3456.0
9        10.0     30-35            5    RedFox        Non-Veg     NaN

   NoOfPax  EstimatedSalary
0      2.0          40000.0
1      3.0          59000.0
2      2.0          30000.0
3      2.0         120000.0
4      2.0          45000.0
5      2.0         122220.0
6      2.0          21122.0
7      2.0         345673.0
8      3.0          96755.0
9      4.0          87777.0
```

Code Cell 19

```
df['Rating(1-5)'] = df['Rating(1-5)'].fillna(
    round(df['Rating(1-5)'].median())
)

df
```

Output

```
   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference    Bill  \
0         1.0     20-25            4       Ibis            Veg  1300.0
1         2.0     30-35            5  LemonTree        Non-Veg  2000.0
2         3.0     25-30            6    RedFox            Veg  1322.0
3         4.0     20-25           -1  LemonTree            Veg  1234.0
4         5.0       35+            3       Ibis            Veg   989.0
5         6.0       35+            3       Ibis        Non-Veg  1909.0
6         7.0       35+            4    RedFox            Veg  1000.0
7         8.0     20-25            7  LemonTree            Veg  2999.0
8         9.0     25-30            2       Ibis        Non-Veg  3456.0
9        10.0     30-35            5    RedFox        Non-Veg     NaN

   NoOfPax  EstimatedSalary
0      2.0          40000.0
1      3.0          59000.0
2      2.0          30000.0
3      2.0         120000.0
4      2.0          45000.0
5      2.0         122220.0
6      2.0          21122.0
7      2.0         345673.0
8      3.0          96755.0
9      4.0          87777.0
```

Code Cell 20

```
df['Bill'] = df['Bill'].fillna(
    round(df['Bill'].mean())
)

df
```

Output

```
   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference    Bill  \
0         1.0     20-25            4       Ibis            Veg  1300.0
1         2.0     30-35            5  LemonTree        Non-Veg  2000.0
2         3.0     25-30            6    RedFox            Veg  1322.0
3         4.0     20-25           -1  LemonTree            Veg  1234.0
4         5.0       35+            3       Ibis            Veg   989.0
5         6.0       35+            3       Ibis        Non-Veg  1909.0
6         7.0       35+            4    RedFox            Veg  1000.0
7         8.0     20-25            7  LemonTree            Veg  2999.0
8         9.0     25-30            2       Ibis        Non-Veg  3456.0
9        10.0     30-35            5    RedFox        Non-Veg  1801.0

   NoOfPax  EstimatedSalary
0      2.0          40000.0
1      3.0          59000.0
2      2.0          30000.0
3      2.0         120000.0
4      2.0          45000.0
5      2.0         122220.0
6      2.0          21122.0
7      2.0         345673.0
8      3.0          96755.0
9      4.0          87777.0
```

Code Cell 21

```
df
```

Output

```
   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference    Bill  \
0         1.0     20-25            4       Ibis            Veg  1300.0
1         2.0     30-35            5  LemonTree        Non-Veg  2000.0
2         3.0     25-30            6    RedFox            Veg  1322.0
3         4.0     20-25           -1  LemonTree            Veg  1234.0
4         5.0       35+            3       Ibis            Veg   989.0
5         6.0       35+            3       Ibis        Non-Veg  1909.0
6         7.0       35+            4    RedFox            Veg  1000.0
7         8.0     20-25            7  LemonTree            Veg  2999.0
8         9.0     25-30            2       Ibis        Non-Veg  3456.0
9        10.0     30-35            5    RedFox        Non-Veg  1801.0

   NoOfPax  EstimatedSalary
0      2.0          40000.0
1      3.0          59000.0
2      2.0          30000.0
3      2.0         120000.0
4      2.0          45000.0
5      2.0         122220.0
6      2.0          21122.0
7      2.0         345673.0
8      3.0          96755.0
9      4.0          87777.0
```

Experiment 4

Code Cell 1

```
import numpy as np
import pandas as pd
```

Code Cell 2

```
df = pd.read_csv("Preprocessing Dataset.csv.csv")
df
```

Output

```
   Country   Age   Salary Purchased
0   France  44.0  72000.0        No
1    Spain  27.0  48000.0       Yes
2  Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4  Germany  40.0      NaN       Yes
5   France  35.0  58000.0       Yes
6    Spain   NaN  52000.0        No
7   France  48.0  79000.0       Yes
8  Germany  50.0  83000.0        No
9   France  37.0  67000.0       Yes
```

Code Cell 3

```
df.info()
```

Output

```
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Country    10 non-null     str
 1   Age        9 non-null      float64
 2   Salary     9 non-null      float64
 3   Purchased  10 non-null     str
dtypes: float64(2), str(2)
memory usage: 537.0 bytes
```

Code Cell 4

```
df["Country"].mode()
```

Output

```
0    France
Name: Country, dtype: str
```

Code Cell 5

```
country_mode = df["Country"].mode()[0]

df["Country"] = df["Country"].fillna(country_mode)

df
```

Output

```
   Country   Age   Salary Purchased
0   France  44.0  72000.0        No
1    Spain  27.0  48000.0       Yes
2  Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4  Germany  40.0      NaN       Yes
5   France  35.0  58000.0       Yes
6    Spain   NaN  52000.0        No
7   France  48.0  79000.0       Yes
8  Germany  50.0  83000.0        No
9   France  37.0  67000.0       Yes
```

Code Cell 6

```
age_median = df["Age"].median()

df["Age"] = df["Age"].fillna(age_median)

df
```

Output

```
   Country   Age   Salary Purchased
0   France  44.0  72000.0        No
1    Spain  27.0  48000.0       Yes
2  Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4  Germany  40.0      NaN       Yes
5   France  35.0  58000.0       Yes
6    Spain  38.0  52000.0        No
7   France  48.0  79000.0       Yes
8  Germany  50.0  83000.0        No
9   France  37.0  67000.0       Yes
```

Code Cell 7

```
salary_mean = round(df["Salary"].mean())

df["Salary"] = df["Salary"].fillna(salary_mean)

df
```

Output

```
   Country   Age   Salary Purchased
0   France  44.0  72000.0        No
1    Spain  27.0  48000.0       Yes
2  Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4  Germany  40.0  63778.0       Yes
5   France  35.0  58000.0       Yes
6    Spain  38.0  52000.0        No
7   France  48.0  79000.0       Yes
8  Germany  50.0  83000.0        No
9   France  37.0  67000.0       Yes
```

Code Cell 8

```
df.isnull().sum()
```

Output

```
Country      0
Age          0
Salary       0
Purchased    0
dtype: int64
```

Code Cell 9

```
country_dummies = pd.get_dummies(df["Country"])

country_dummies
```

Output

```
   France  Germany  Spain
0    True    False  False
1   False    False   True
2   False     True  False
3   False    False   True
4   False     True  False
5    True    False  False
6   False    False   True
7    True    False  False
8   False     True  False
9    True    False  False
```

Code Cell 10

```
updated_dataset = pd.concat(
    [country_dummies, df[["Age", "Salary", "Purchased"]]],
    axis=1
)

updated_dataset
```

Output

```
   France  Germany  Spain   Age   Salary Purchased
0    True    False  False  44.0  72000.0        No
1   False    False   True  27.0  48000.0       Yes
2   False     True  False  30.0  54000.0        No
3   False    False   True  38.0  61000.0        No
4   False     True  False  40.0  63778.0       Yes
5    True    False  False  35.0  58000.0       Yes
6   False    False   True  38.0  52000.0        No
7    True    False  False  48.0  79000.0       Yes
8   False     True  False  50.0  83000.0        No
9    True    False  False  37.0  67000.0       Yes
```

Code Cell 11

```
updated_dataset["Purchased"] = updated_dataset["Purchased"].replace({
    "No": 0,
    "Yes": 1
})

updated_dataset
```

Output

```
   France  Germany  Spain   Age   Salary Purchased
0    True    False  False  44.0  72000.0         0
1   False    False   True  27.0  48000.0         1
2   False     True  False  30.0  54000.0         0
3   False    False   True  38.0  61000.0         0
4   False     True  False  40.0  63778.0         1
5    True    False  False  35.0  58000.0         1
6   False    False   True  38.0  52000.0         0
7    True    False  False  48.0  79000.0         1
8   False     True  False  50.0  83000.0         0
9    True    False  False  37.0  67000.0         1
```

Code Cell 12

```
updated_dataset.info()
```

Output

```
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 6 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   France     10 non-null     bool
 1   Germany    10 non-null     bool
 2   Spain      10 non-null     bool
 3   Age        10 non-null     float64
 4   Salary     10 non-null     float64
 5   Purchased  10 non-null     object
dtypes: bool(3), float64(2), object(1)
memory usage: 402.0+ bytes
```

Experiment 5

Code Cell 1

```
import numpy as np
```

Code Cell 2

```
array = np.random.randint(1,100,16)
array
```

Output

```
array([85, 15, 12, 88, 37, 34, 50, 21, 39, 72, 65, 60, 18, 22, 49, 14],
      dtype=int32)
```

Code Cell 3

```
array.mean()
```

Output

```
np.float64(42.5625)
```

Code Cell 4

```
np.percentile(array,25)
```

Output

```
np.float64(20.25)
```

Code Cell 5

```
np.percentile(array,50)
```

Output

```
np.float64(38.0)
```

Code Cell 6

```
np.percentile(array,75)
```

Output

```
np.float64(61.25)
```

Code Cell 7

```
np.percentile(array,100)
```

Output

```
np.float64(88.0)
```

Code Cell 8

```
def outDetection(array):
    sorted(array)
    Q1,Q3=np.percentile(array,[25,75])
    IQR=Q3-Q1
    lr=Q1-(1.5*IQR)
    ur=Q3+(1.5*IQR)
    return lr,ur
```

Code Cell 9

```
lr,ur=outDetection(array)
lr,ur
```

Output

```
(np.float64(-41.25), np.float64(122.75))
```

Code Cell 10

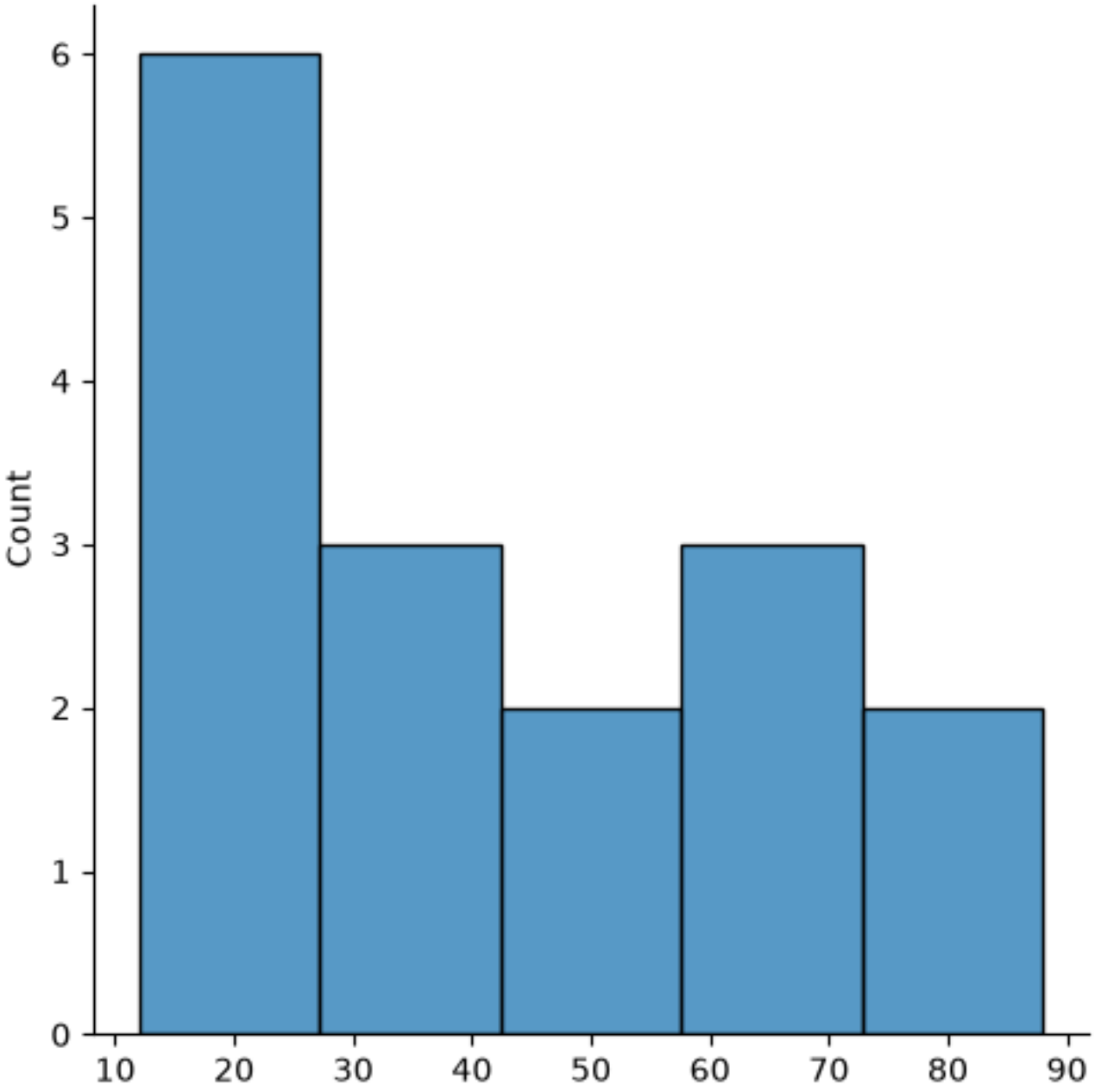
```
import seaborn as sns
%matplotlib inline

sns.displot(array)
```

Output

```
<seaborn.axisgrid.FacetGrid at 0x25345d3bed0>
```

Output



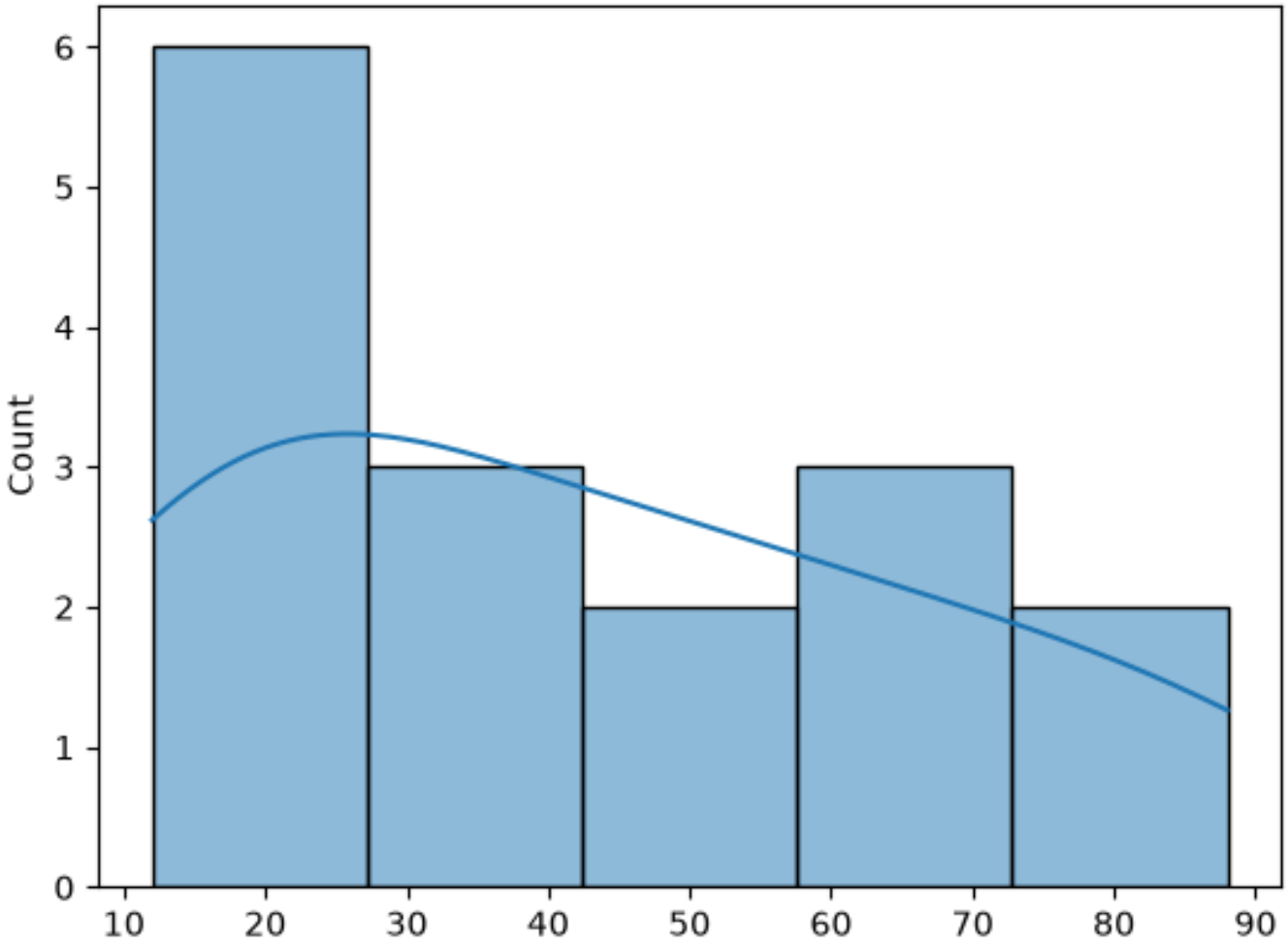
Code Cell 11

```
sns.histplot(array, kde=True)
```

Output

```
<Axes: ylabel='Count'>
```

Output



Code Cell 12

```
new_array=array[(array>lr) & (array<ur)]
new_array
```

Output

```
array([85, 15, 12, 88, 37, 34, 50, 21, 39, 72, 65, 60, 18, 22, 49, 14],
      dtype=int32)
```

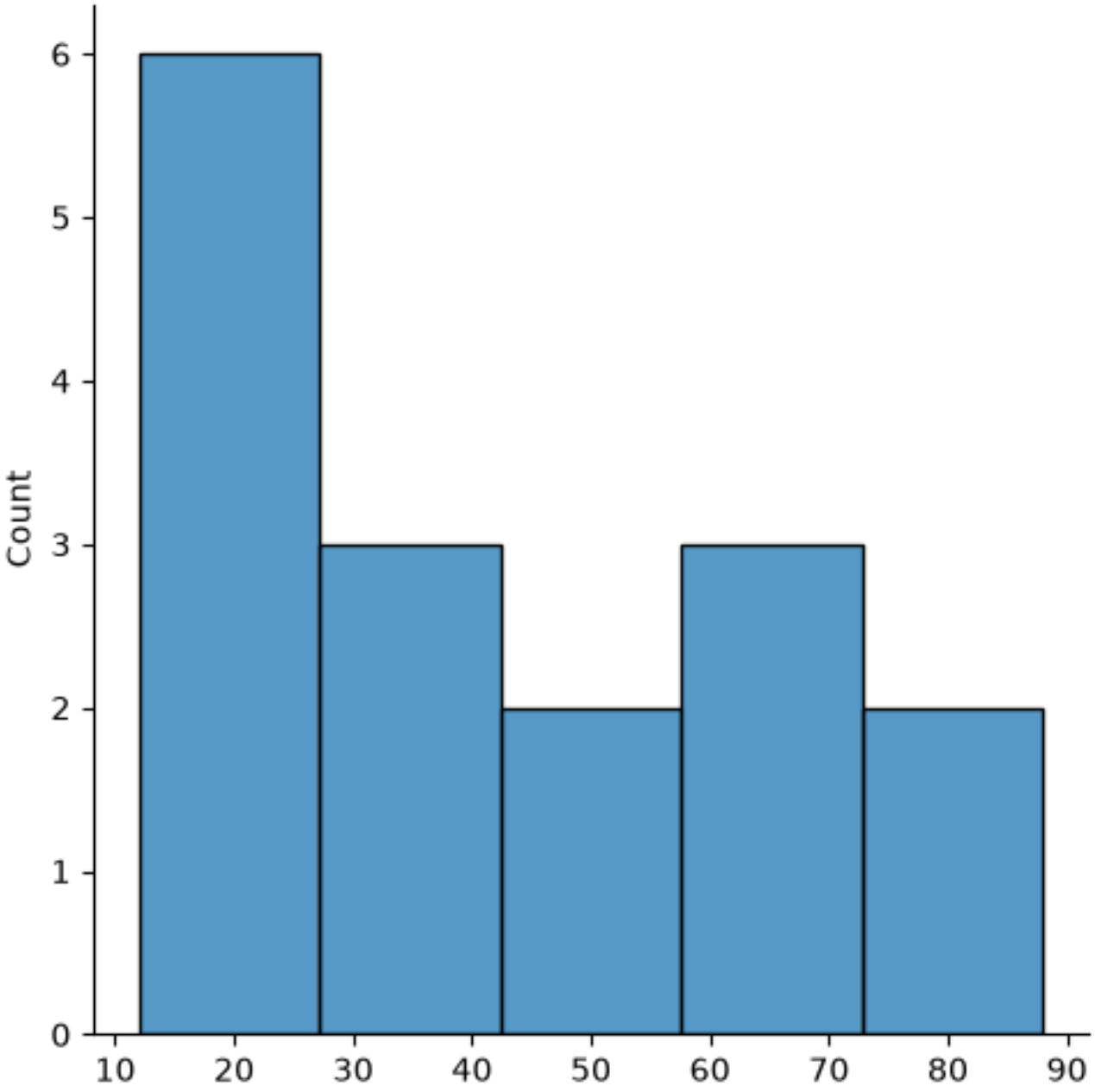
Code Cell 13

```
sns.displot(new_array)
```

Output

```
<seaborn.axisgrid.FacetGrid at 0x25345f58cd0>
```

Output



Code Cell 14

```
lr1,ur1=outDetection(new_array)
lr1,ur1
```

Output

```
(np.float64(-41.25), np.float64(122.75))
```

Code Cell 15

```
final_array=new_array[(new_array>lr1) & (new_array<ur1)]
final_array
```

Output

```
array([85, 15, 12, 88, 37, 34, 50, 21, 39, 72, 65, 60, 18, 22, 49, 14],
      dtype=int32)
```

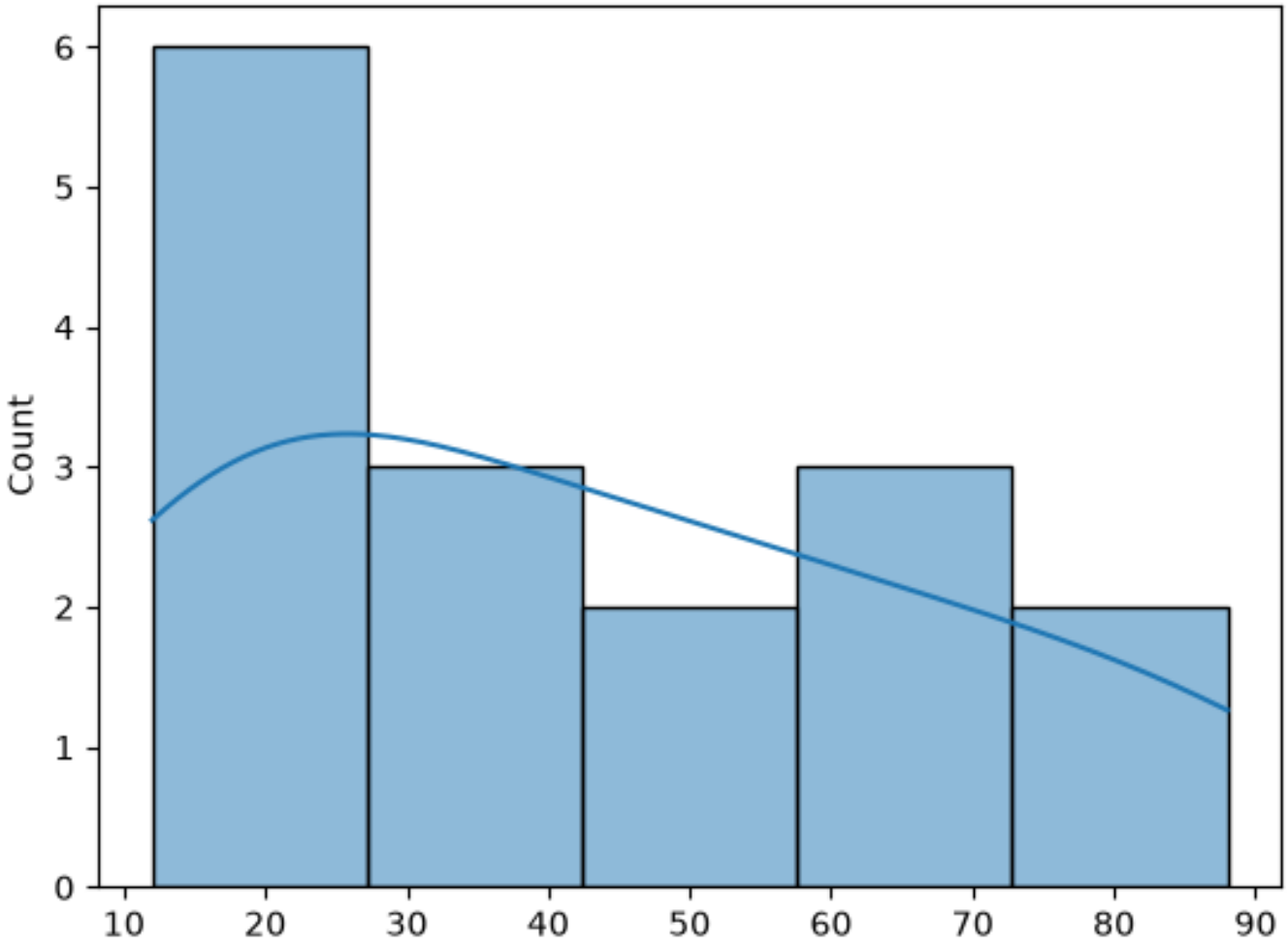
Code Cell 16

```
sns.histplot(final_array,kde=True)
```

Output

```
<Axes: ylabel='Count'>
```

Output



Experiment 6

Code Cell 1

```
import numpy as np
import pandas as pd
```

Code Cell 2

```
df = pd.read_csv("feature_scaling_dataset.csv")
df
```

Output

```
   Country   Age   Salary Purchased
0   France  44.0  72000.0        No
1    Spain  27.0  48000.0       Yes
2  Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4  Germany  40.0      NaN       Yes
5   France  35.0  58000.0       Yes
6    Spain   NaN  52000.0        No
7   France  48.0  79000.0       Yes
8      NaN  50.0  83000.0        No
9   France  37.0  67000.0       Yes
```

Code Cell 3

```
df.head()
```

Output

```
   Country   Age   Salary Purchased
0   France  44.0  72000.0        No
1    Spain  27.0  48000.0       Yes
2  Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4  Germany  40.0      NaN       Yes
```

Code Cell 4

```
df["Country"] = df["Country"].fillna(df["Country"].mode()[0])
```

Code Cell 5

```
df
```

Output

```
   Country   Age   Salary Purchased
0   France  44.0  72000.0        No
1    Spain  27.0  48000.0       Yes
2  Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4  Germany  40.0      NaN       Yes
5   France  35.0  58000.0       Yes
6    Spain   NaN  52000.0        No
7   France  48.0  79000.0       Yes
8   France  50.0  83000.0        No
9   France  37.0  67000.0       Yes
```

Code Cell 6

```
features = df.iloc[:, :-1].values
label = df.iloc[:, -1].values
```

Code Cell 7

```
features
```

Output

```
array([['France', 44.0, 72000.0],
       ['Spain', 27.0, 48000.0],
       ['Germany', 30.0, 54000.0],
       ['Spain', 38.0, 61000.0],
       ['Germany', 40.0, nan],
       ['France', 35.0, 58000.0],
       ['Spain', nan, 52000.0],
       ['France', 48.0, 79000.0],
       ['France', 50.0, 83000.0],
       ['France', 37.0, 67000.0]], dtype=object)
```

Code Cell 8

```
from sklearn.impute import SimpleImputer

age = SimpleImputer(strategy="mean", missing_values=np.nan)
salary = SimpleImputer(strategy="mean", missing_values=np.nan)

age.fit(features[:, [1]])
salary.fit(features[:, [2]])
```

Output

```
SimpleImputer()
```

Code Cell 9

```
features[:, [1]] = age.transform(features[:, [1]])
features[:, [2]] = salary.transform(features[:, [2]])

features
```

Output

```
array([['France', 44.0, 72000.0],
       ['Spain', 27.0, 48000.0],
       ['Germany', 30.0, 54000.0],
       ['Spain', 38.0, 61000.0],
       ['Germany', 40.0, 63777.77777777778],
       ['France', 35.0, 58000.0],
       ['Spain', 38.77777777777778, 52000.0],
       ['France', 48.0, 79000.0],
       ['France', 50.0, 83000.0],
       ['France', 37.0, 67000.0]], dtype=object)
```

Code Cell 10

```
from sklearn.preprocessing import OneHotEncoder

oh = OneHotEncoder(sparse_output=False)
country = oh.fit_transform(features[:, [0]])

country
```

Output

```
array([[1., 0., 0.],
       [0., 0., 1.],
       [0., 1., 0.],
       [0., 0., 1.],
       [0., 1., 0.],
       [1., 0., 0.],
       [0., 0., 1.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.]])
```

Code Cell 11

```
final_set = np.concatenate((country, features[:, [1, 2]]), axis=1)
final_set
```

Output

```
array([[1.0, 0.0, 0.0, 44.0, 72000.0],
       [0.0, 0.0, 1.0, 27.0, 48000.0],
       [0.0, 1.0, 0.0, 30.0, 54000.0],
       [0.0, 0.0, 1.0, 38.0, 61000.0],
       [0.0, 1.0, 0.0, 40.0, 63777.77777777778],
       [1.0, 0.0, 0.0, 35.0, 58000.0],
       [0.0, 0.0, 1.0, 38.77777777777778, 52000.0],
       [1.0, 0.0, 0.0, 48.0, 79000.0],
       [1.0, 0.0, 0.0, 50.0, 83000.0],
       [1.0, 0.0, 0.0, 37.0, 67000.0]], dtype=object)
```

Code Cell 12

```
from sklearn.preprocessing import StandardScaler

sc = StandardScaler()
sc.fit(final_set)

feat_standard_scaler = sc.transform(final_set)

feat_standard_scaler
```

Output

```
array([[ 1.00000000e+00, -5.00000000e-01, -6.54653671e-01,
         7.58874362e-01,  7.49473254e-01],
       [-1.00000000e+00, -5.00000000e-01,  1.52752523e+00,
        -1.71150388e+00, -1.43817841e+00],
       [-1.00000000e+00,  2.00000000e+00, -6.54653671e-01,
        -1.27555478e+00, -8.91265492e-01],
       [-1.00000000e+00, -5.00000000e-01,  1.52752523e+00,
        -1.13023841e-01, -2.53200424e-01],
       [-1.00000000e+00,  2.00000000e+00, -6.54653671e-01,
         1.77608893e-01,  6.63219199e-16],
       [ 1.00000000e+00, -5.00000000e-01, -6.54653671e-01,
        -5.48972942e-01, -5.26656882e-01],
       [-1.00000000e+00, -5.00000000e-01,  1.52752523e+00,
         0.00000000e+00, -1.07356980e+00],
       [ 1.00000000e+00, -5.00000000e-01, -6.54653671e-01,
         1.34013983e+00,  1.38753832e+00],
       [ 1.00000000e+00, -5.00000000e-01, -6.54653671e-01,
         1.63077256e+00,  1.75214693e+00],
       [ 1.00000000e+00, -5.00000000e-01, -6.54653671e-01,
        -2.58340208e-01,  2.93712492e-01]])
```

Code Cell 13

```
from sklearn.preprocessing import MinMaxScaler

mms = MinMaxScaler(feature_range=(0,1))
mms.fit(final_set)

feat_minmax_scaler = mms.transform(final_set)

feat_minmax_scaler
```

Output

```
array([[1.        , 0.        , 0.        , 0.73913043, 0.68571429],
       [0.        , 0.        , 1.        , 0.        , 0.        ],
       [0.        , 1.        , 0.        , 0.13043478, 0.17142857],
       [0.        , 0.        , 1.        , 0.47826087, 0.37142857],
       [0.        , 1.        , 0.        , 0.56521739, 0.45079365],
       [1.        , 0.        , 0.        , 0.34782609, 0.28571429],
       [0.        , 0.        , 1.        , 0.51207729, 0.11428571],
       [1.        , 0.        , 0.        , 0.91304348, 0.88571429],
       [1.        , 0.        , 0.        , 1.        , 1.        ],
       [1.        , 0.        , 0.        , 0.43478261, 0.54285714]])
```

Experiment 7

Code Cell 1

```
import seaborn as sns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

%matplotlib inline
```

Code Cell 2

```
tips=sns.load_dataset('tips')
```

Code Cell 3

```
tips.head()
```

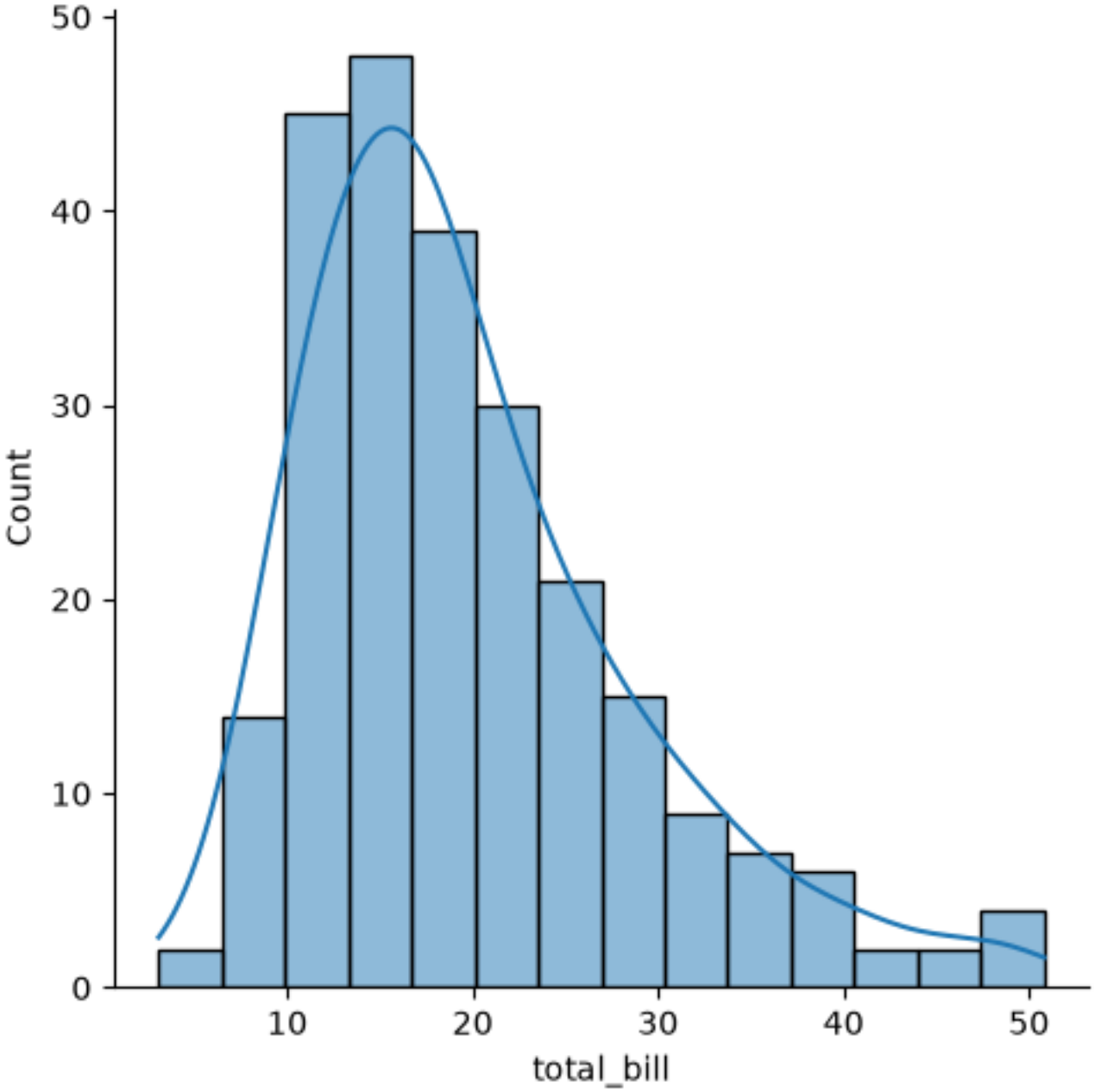
Output

```
   total_bill   tip     sex smoker  day    time  size
0       16.99  1.01  Female     No  Sun  Dinner     2
1       10.34  1.66    Male     No  Sun  Dinner     3
2       21.01  3.50    Male     No  Sun  Dinner     3
3       23.68  3.31    Male     No  Sun  Dinner     2
4       24.59  3.61  Female     No  Sun  Dinner     4
```

Code Cell 4

```
sns.displot(tips.total_bill, kde=True)
plt.show()
```

Output



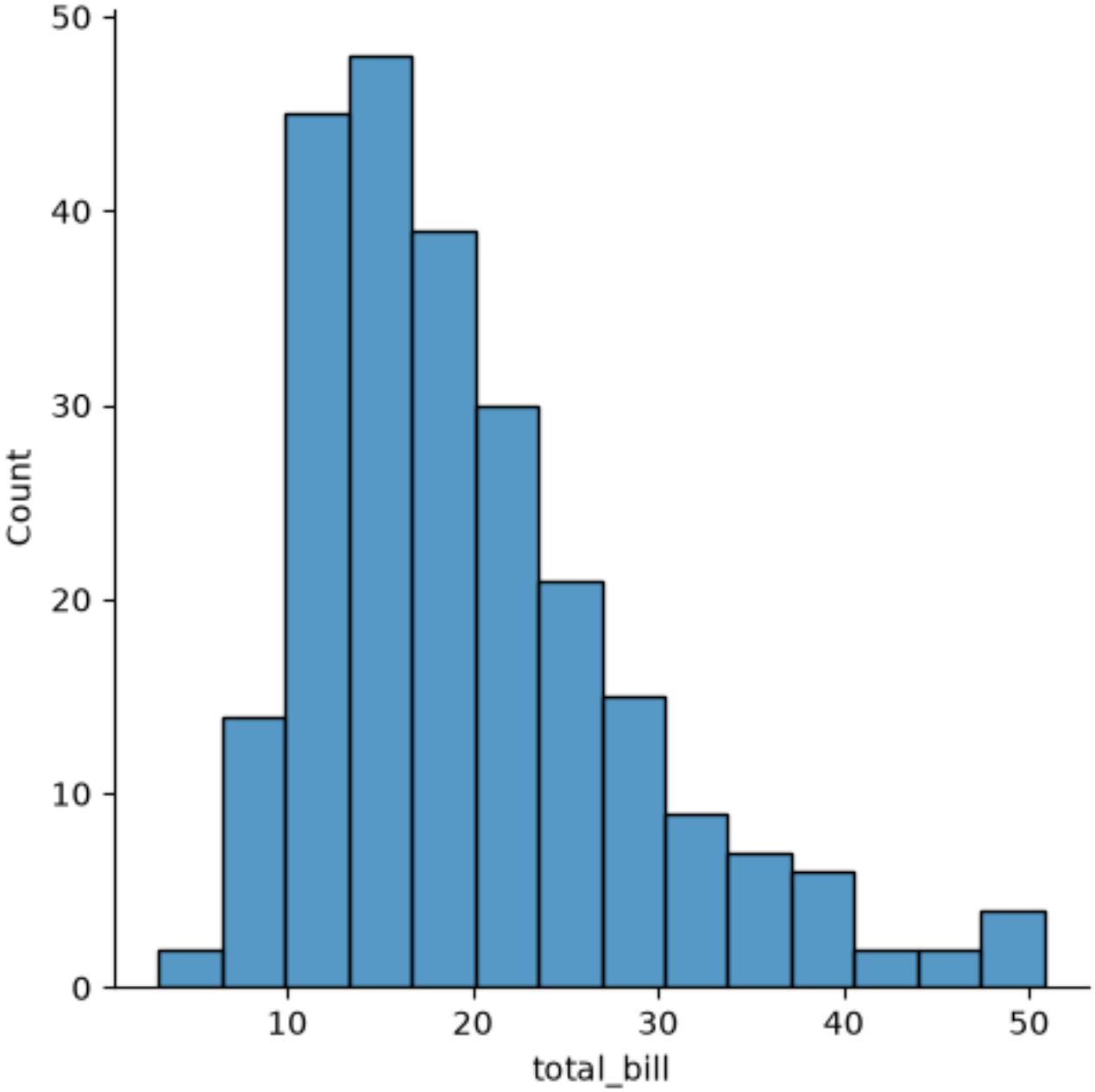
Code Cell 5

```
sns.displot(tips.total_bill, kde=False)
```

Output

```
<seaborn.axisgrid.FacetGrid at 0x1c4394a8690>
```

Output



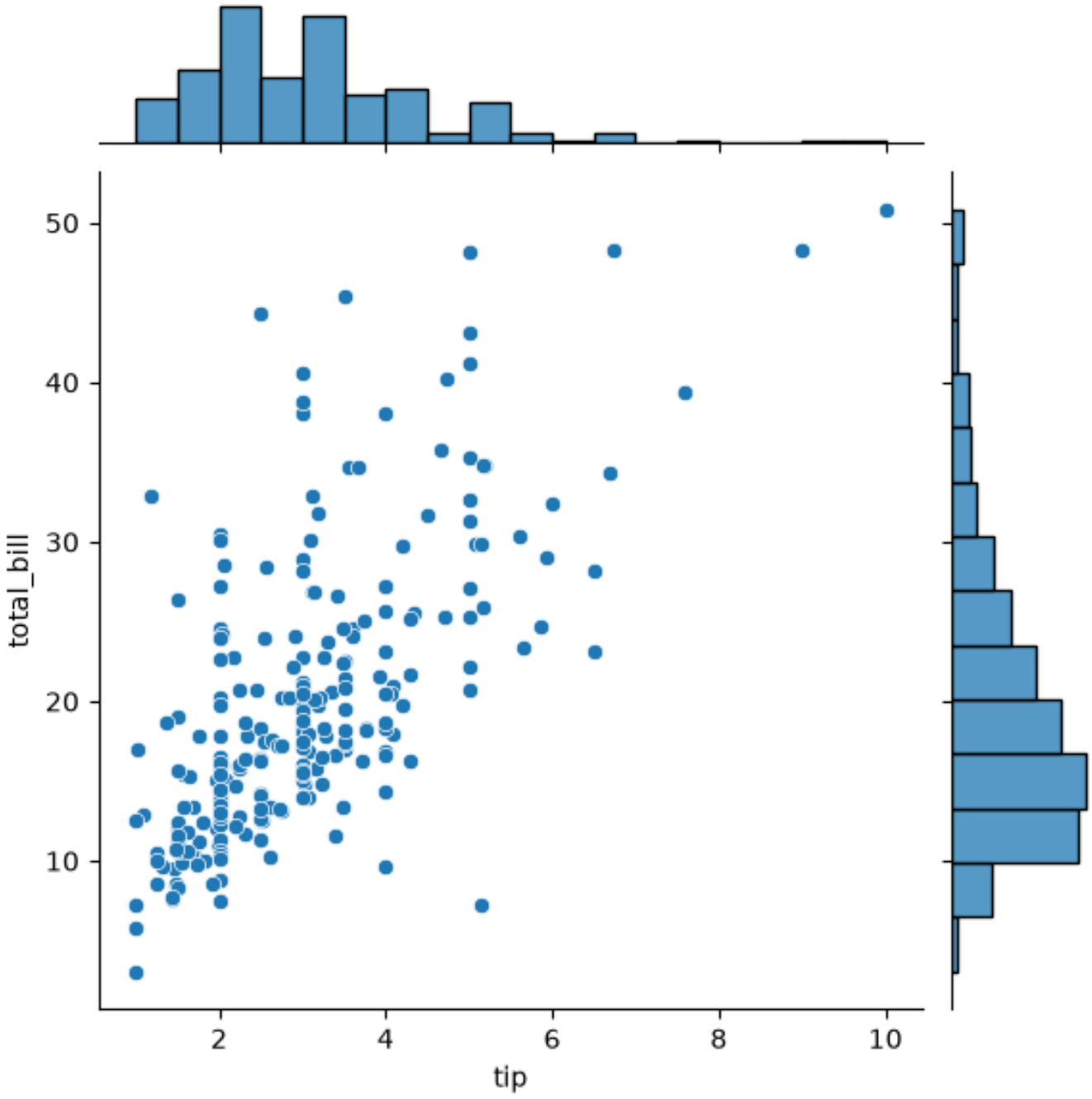
Code Cell 6

```
sns.jointplot(x=tips.tip, y=tips.total_bill)
```

Output

```
<seaborn.axisgrid.JointGrid at 0x1c439b7e510>
```

Output



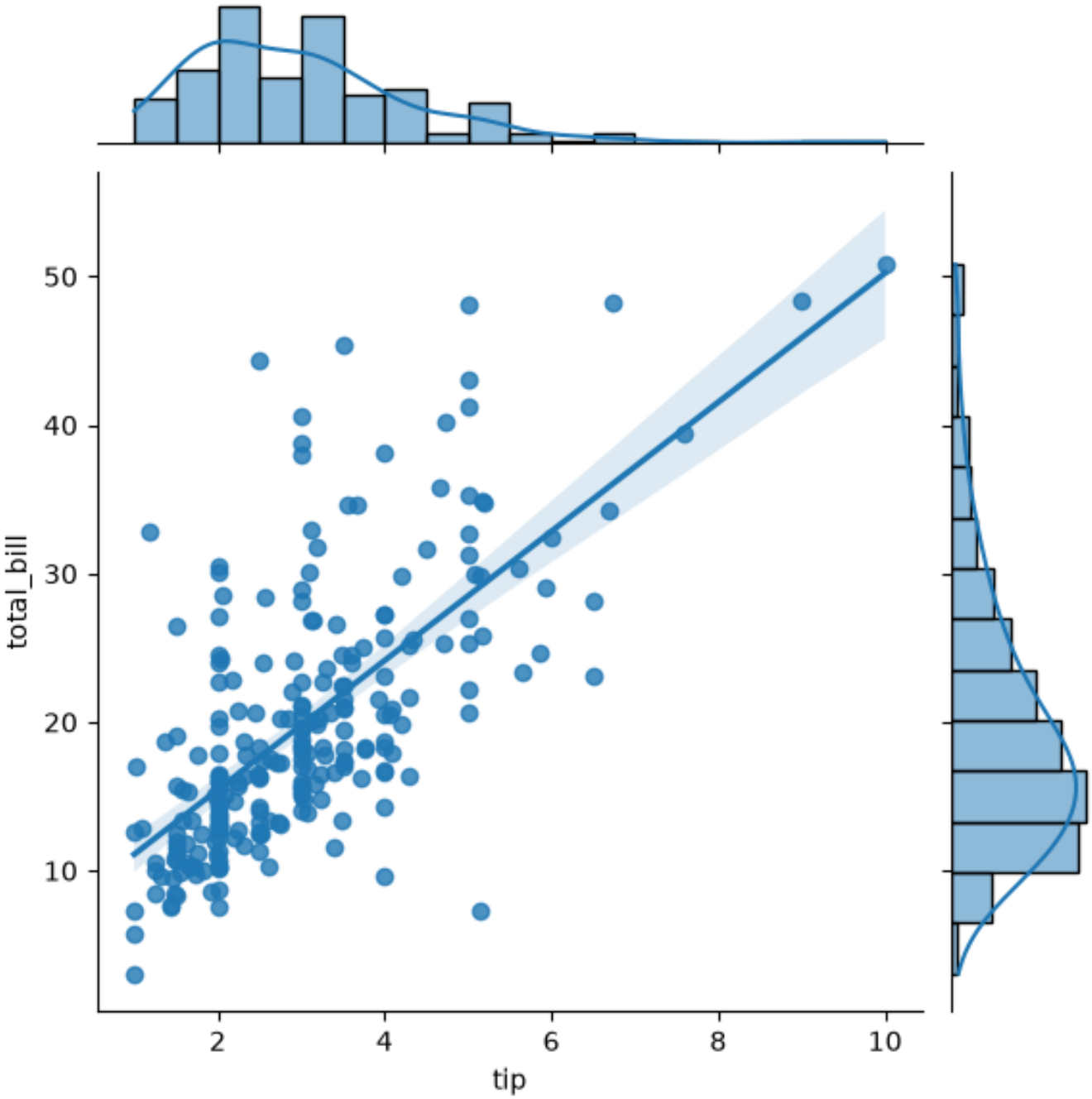
Code Cell 7

```
sns.jointplot(x=tips.tip, y=tips.total_bill, kind="reg")
```

Output

```
<seaborn.axisgrid.JointGrid at 0x1c439cf8a50>
```

Output



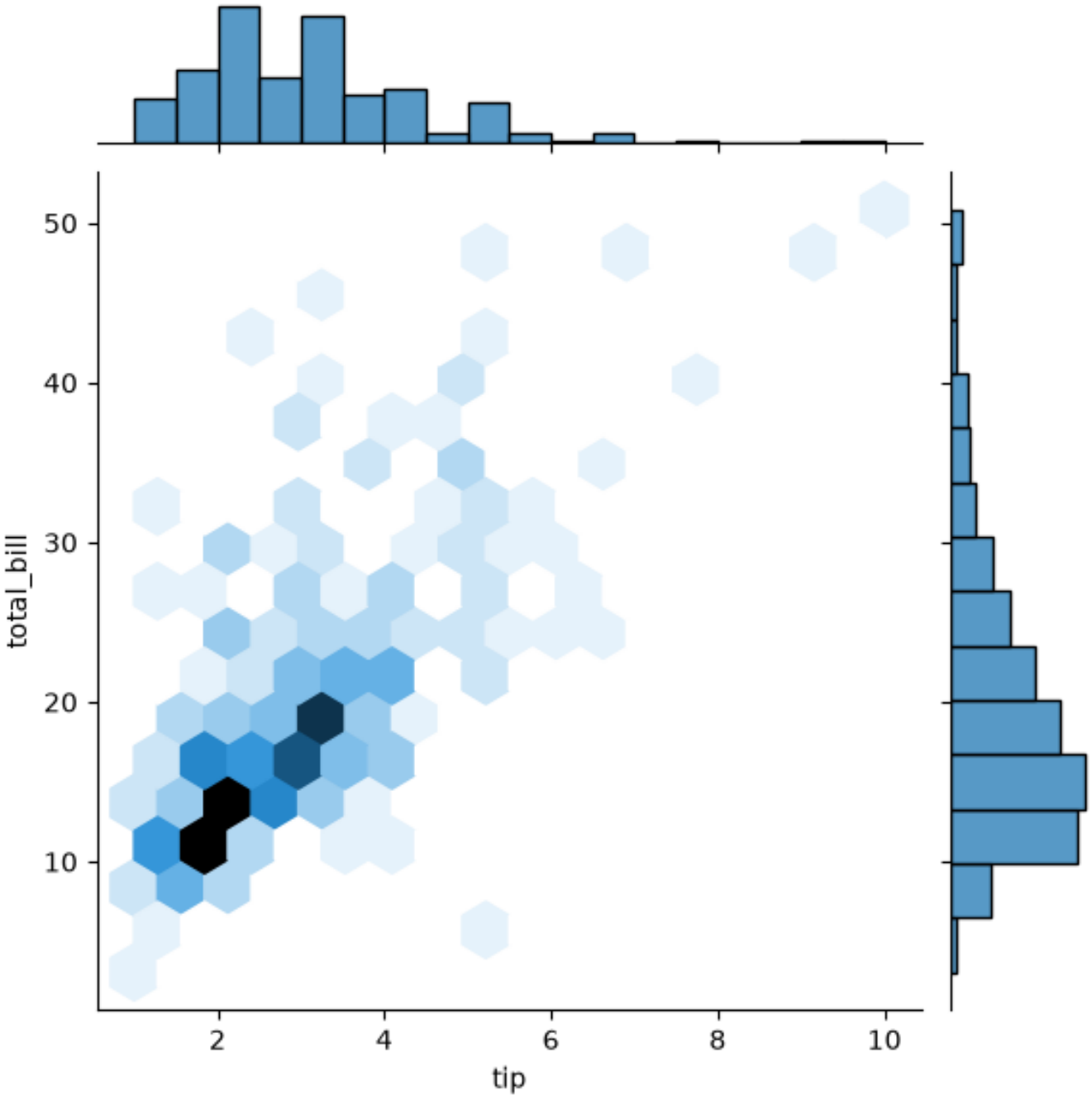
Code Cell 8

```
sns.jointplot(x=tips.tip, y=tips.total_bill, kind="hex")
```

Output

```
<seaborn.axisgrid.JointGrid at 0x1c43a26c410>
```

Output



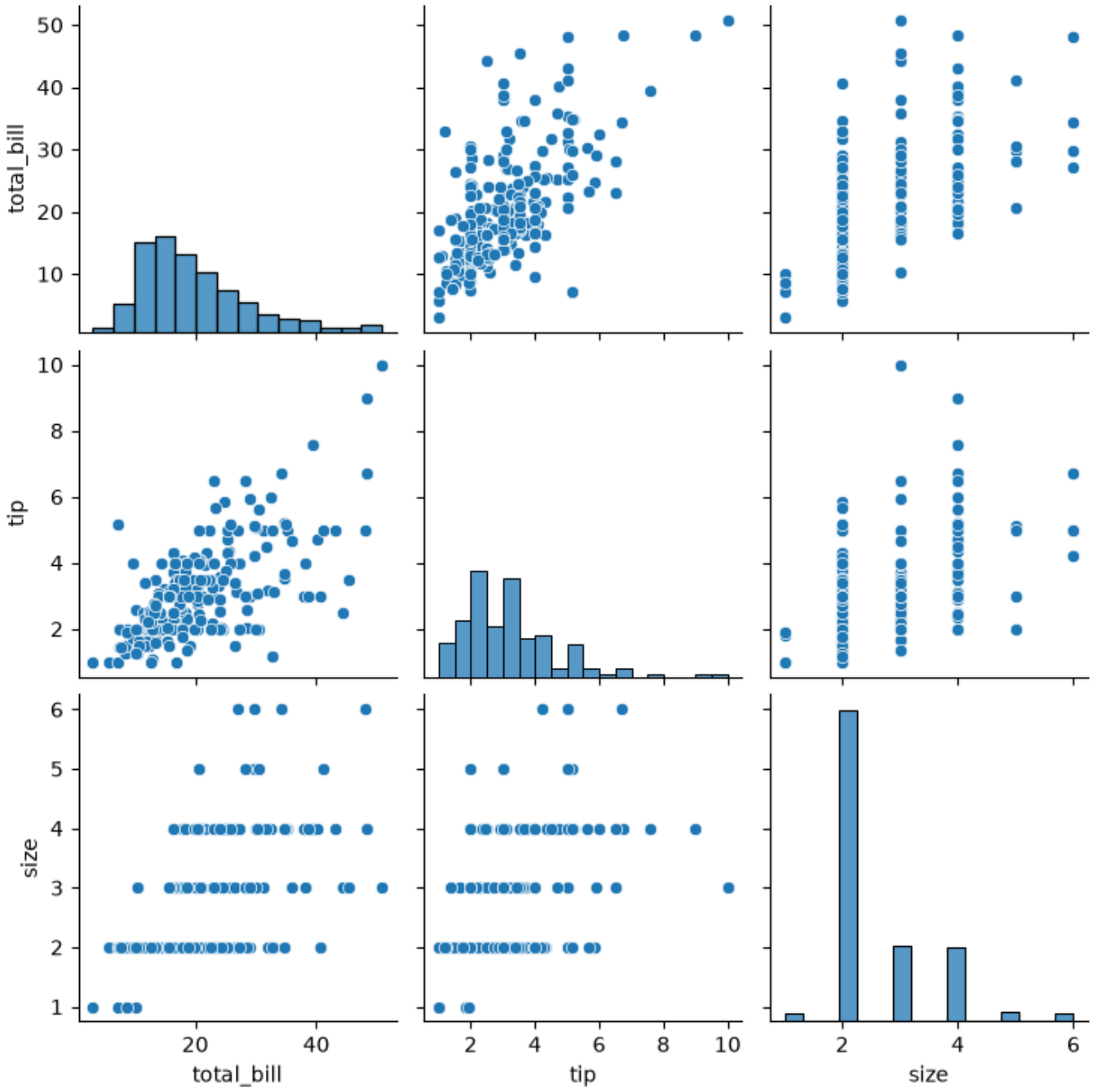
Code Cell 9

```
sns.pairplot(tips)
```

Output

```
<seaborn.axisgrid.PairGrid at 0x1c43b609550>
```

Output



Code Cell 10

```
tips.time.value_counts()
```

Output

```
time
Dinner    176
Lunch      68
Name: count, dtype: int64
```

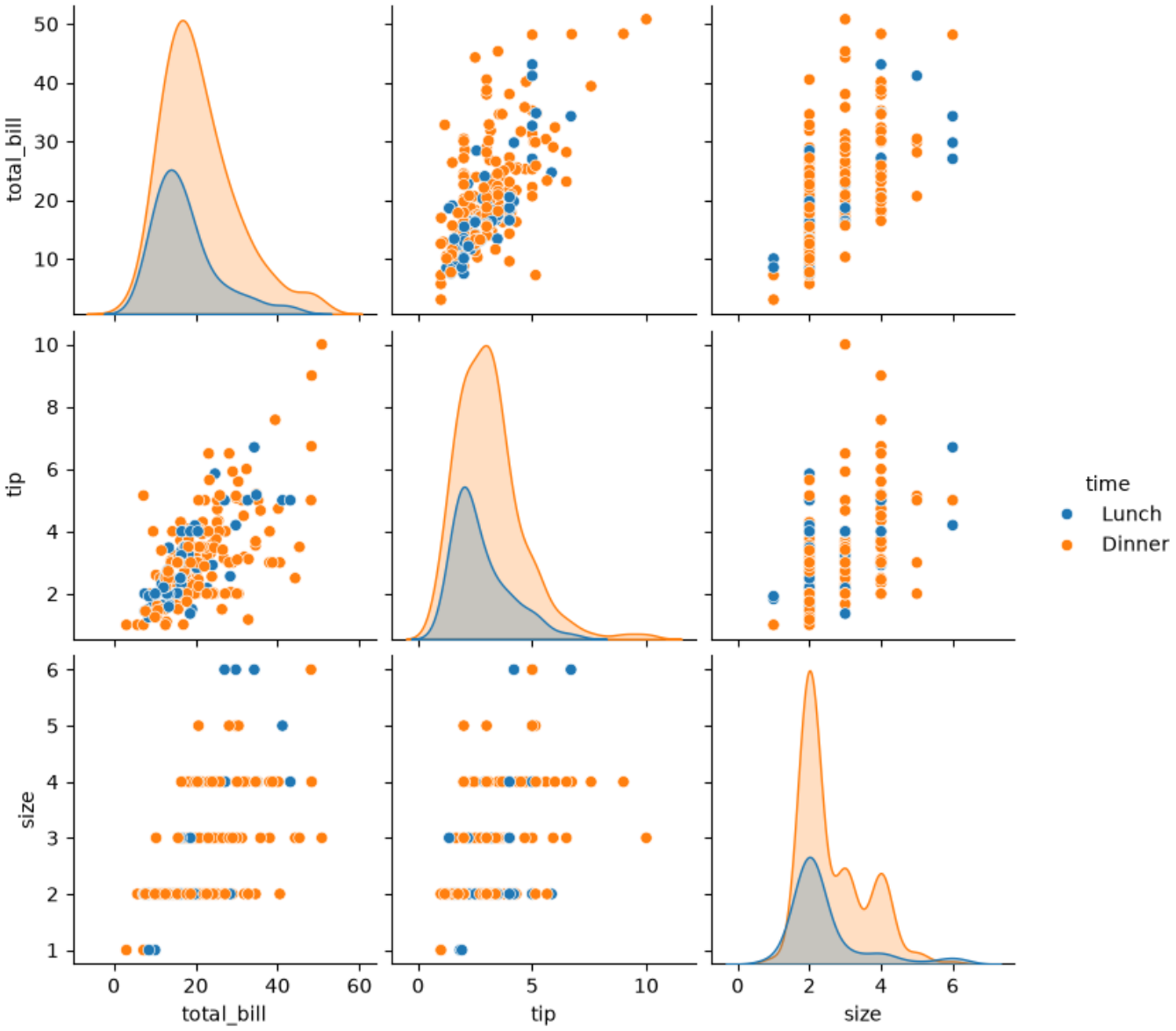
Code Cell 11

```
sns.pairplot(tips, hue='time')
```

Output

```
<seaborn.axisgrid.PairGrid at 0x1c43be687d0>
```

Output



Code Cell 12

```
hue='time'
```

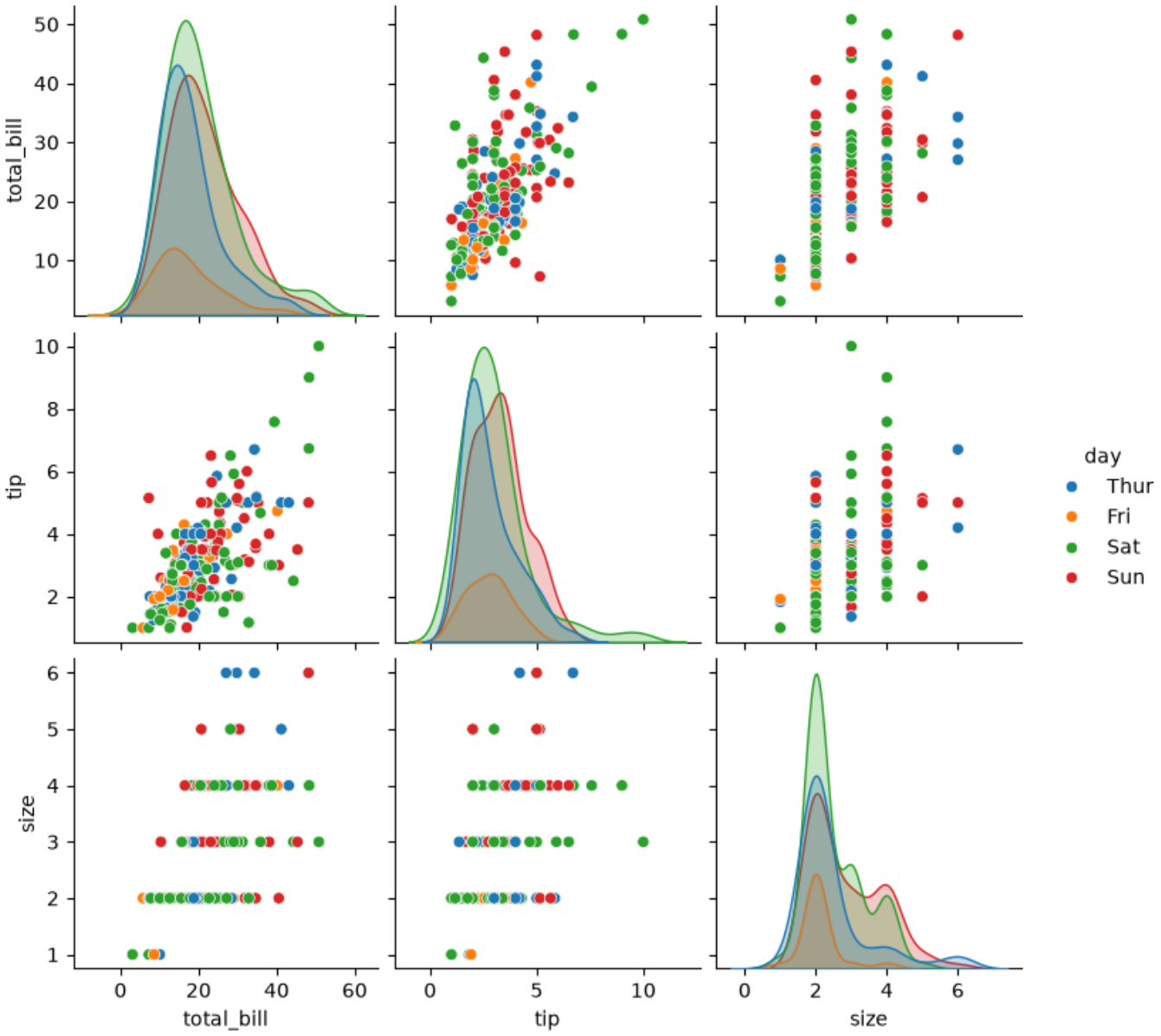
Code Cell 13

```
sns.pairplot(tips, hue='day')
```

Output

```
<seaborn.axisgrid.PairGrid at 0x1c43c68e5d0>
```

Output



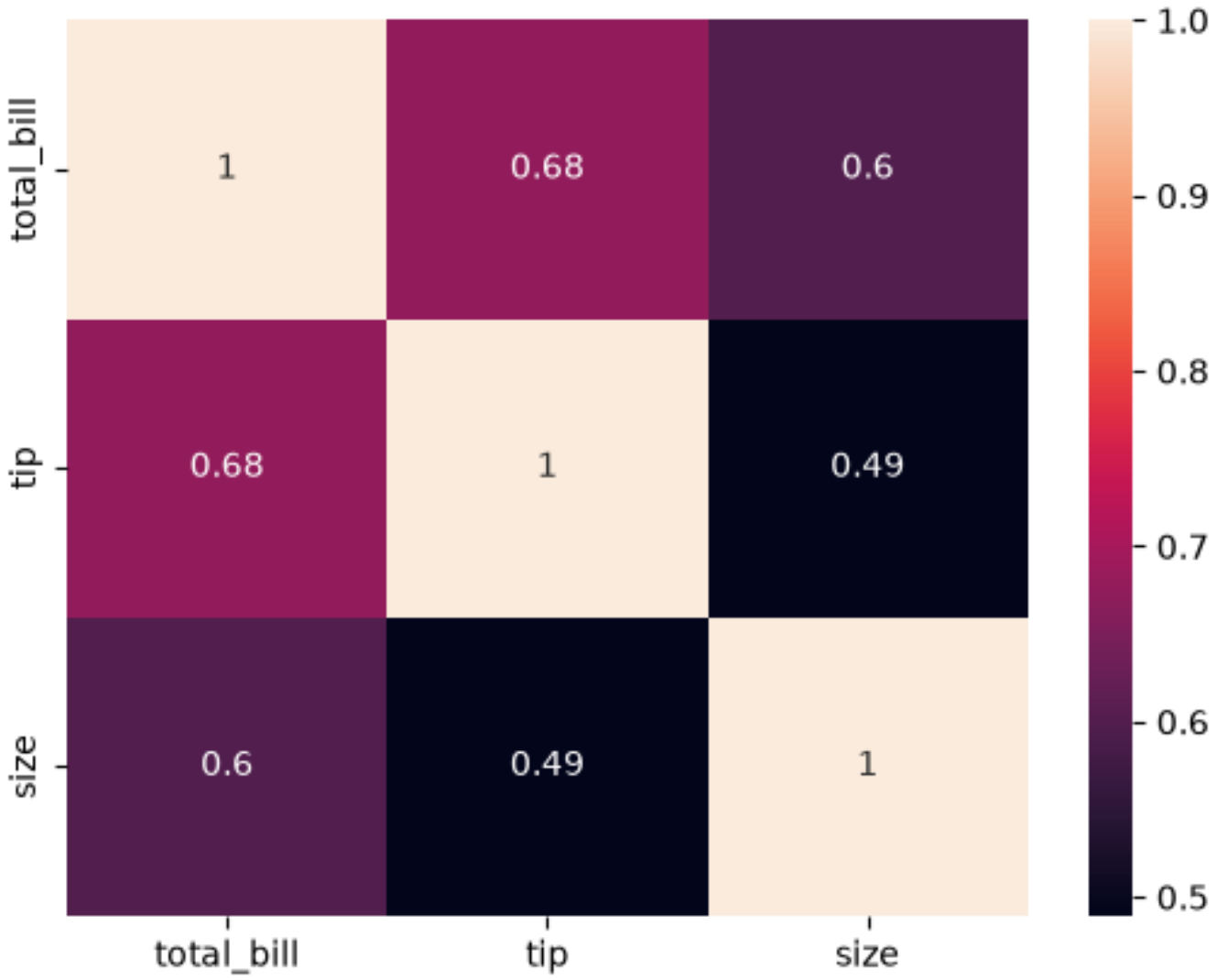
Code Cell 14

```
sns.heatmap(tips.corr(numeric_only=True), annot=True)
```

Output

```
<Axes: >
```

Output



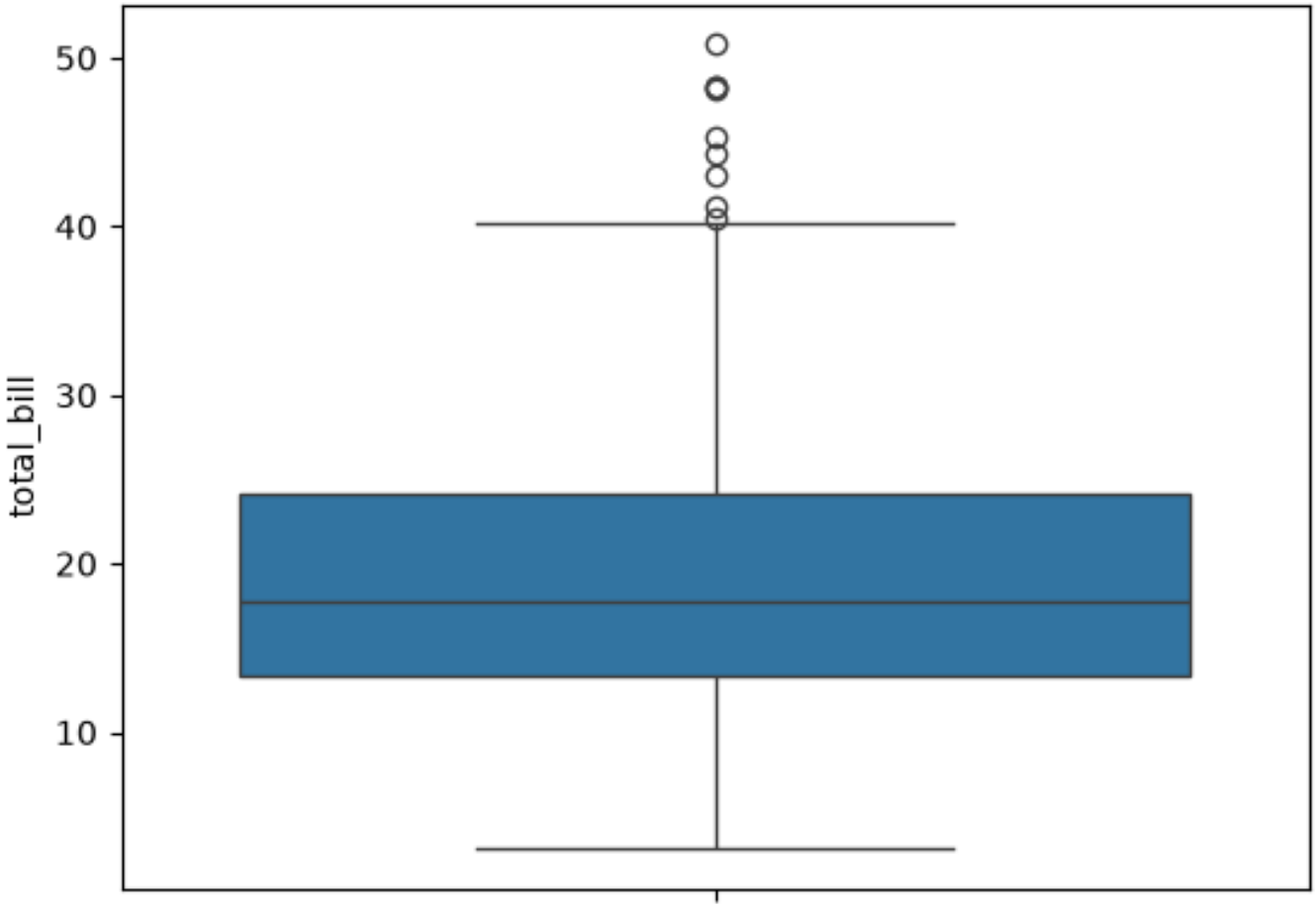
Code Cell 15

```
sns.boxplot(tips.total_bill)
```

Output

```
<Axes: ylabel='total_bill'>
```

Output



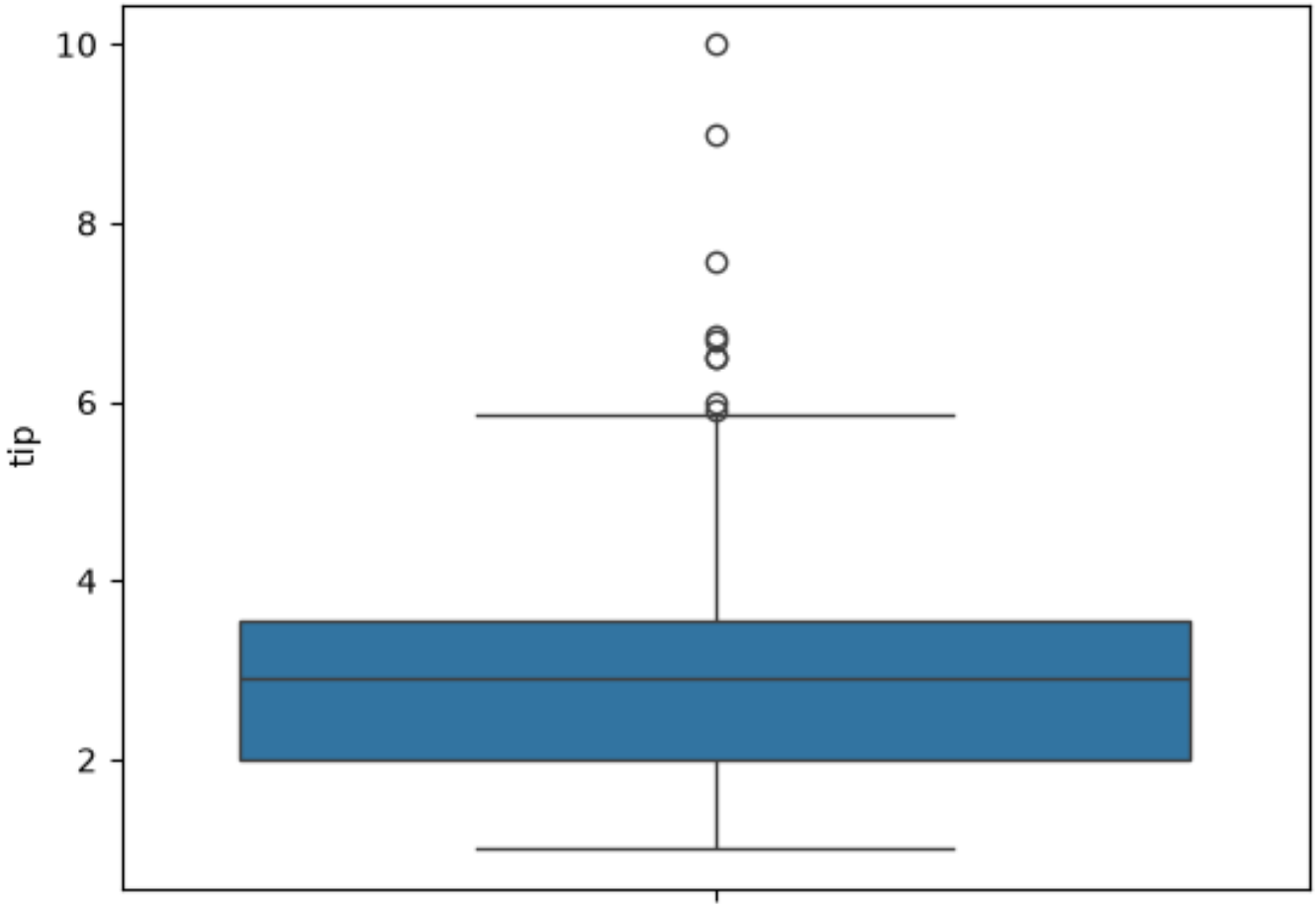
Code Cell 16

```
sns.boxplot(tips.tip)
```

Output

```
<Axes: ylabel='tip'>
```

Output



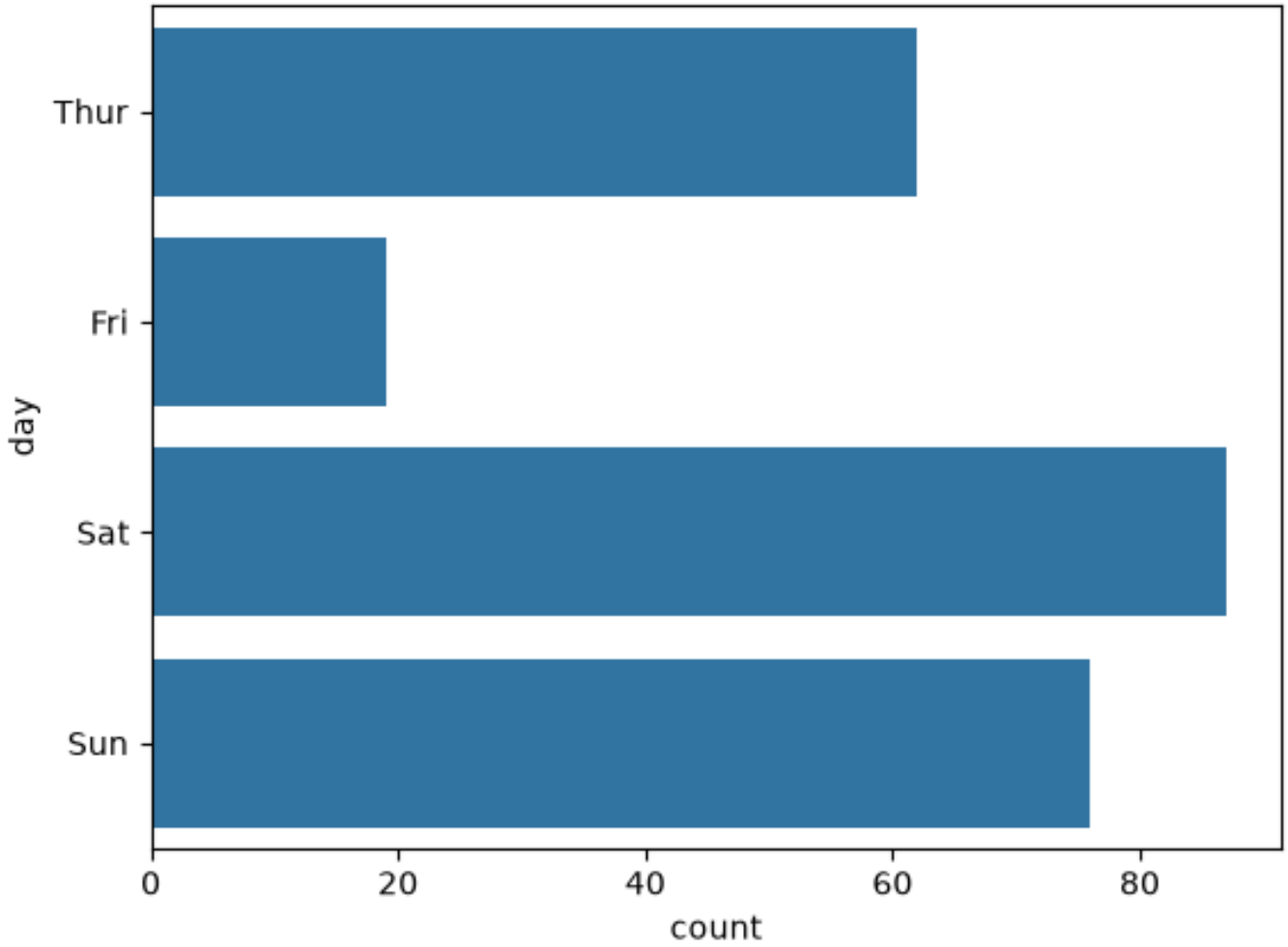
Code Cell 17

```
sns.countplot(tips.day)
```

Output

```
<Axes: xlabel='count', ylabel='day'>
```

Output



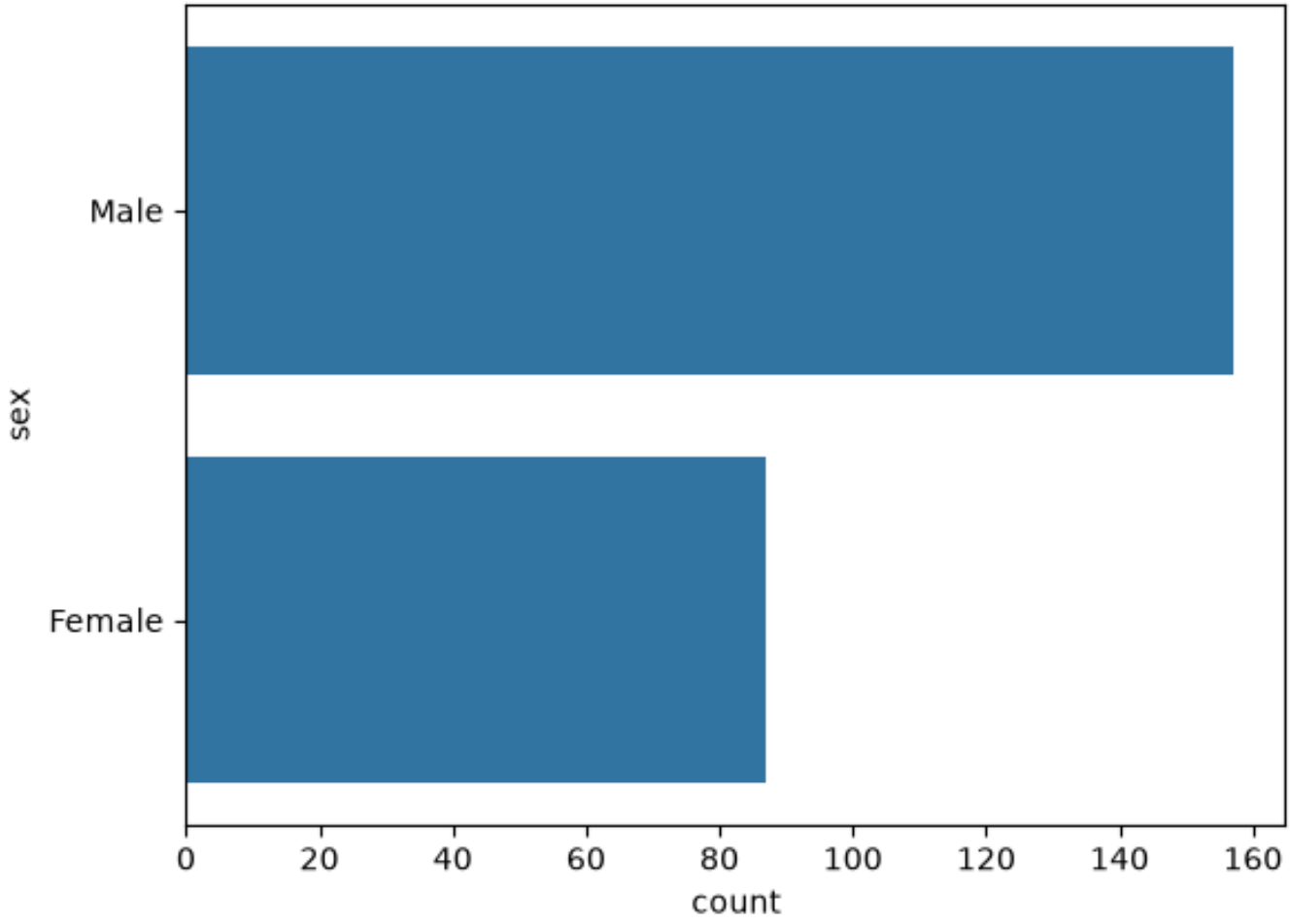
Code Cell 18

```
sns.countplot(tips.sex)
```

Output

```
<Axes: xlabel='count', ylabel='sex'>
```

Output



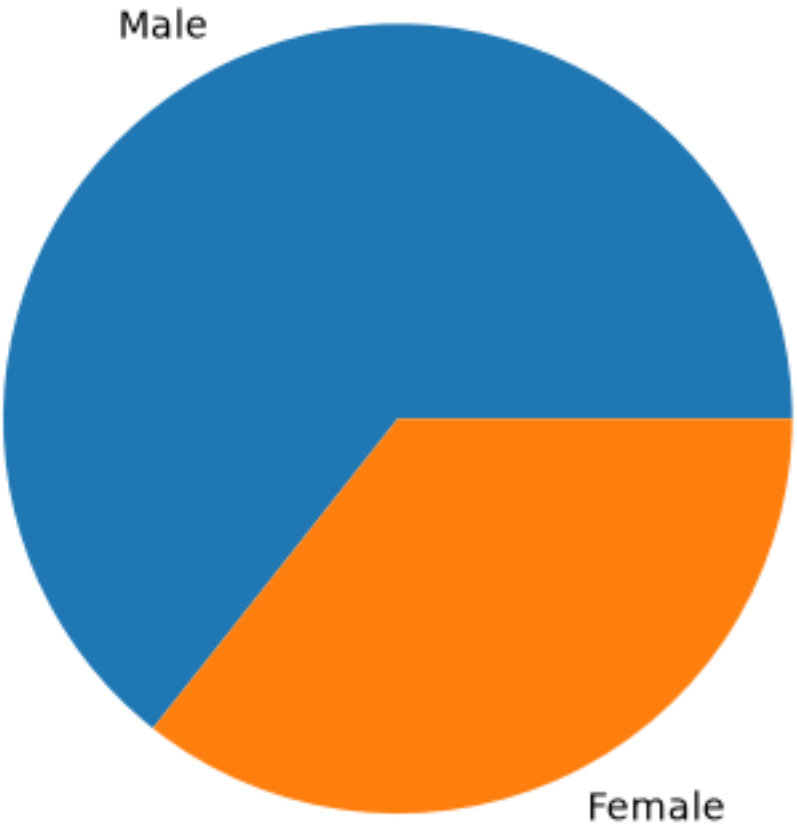
Code Cell 19

```
tips.sex.value_counts().plot(kind='pie')
```

Output

```
<Axes: >
```

Output



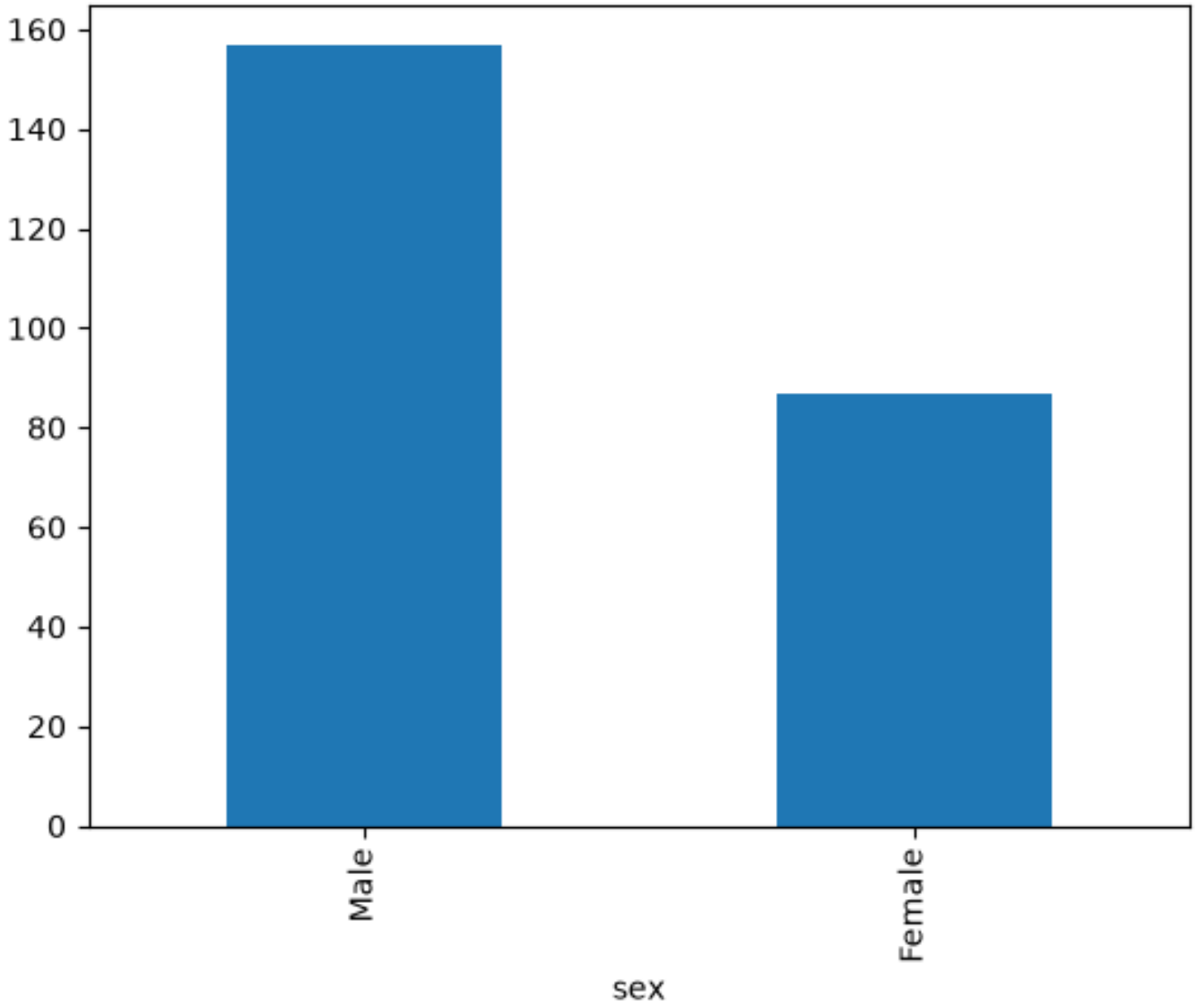
Code Cell 20

```
tips.sex.value_counts().plot(kind='bar')
```

Output

```
<Axes: xlabel='sex'>
```

Output



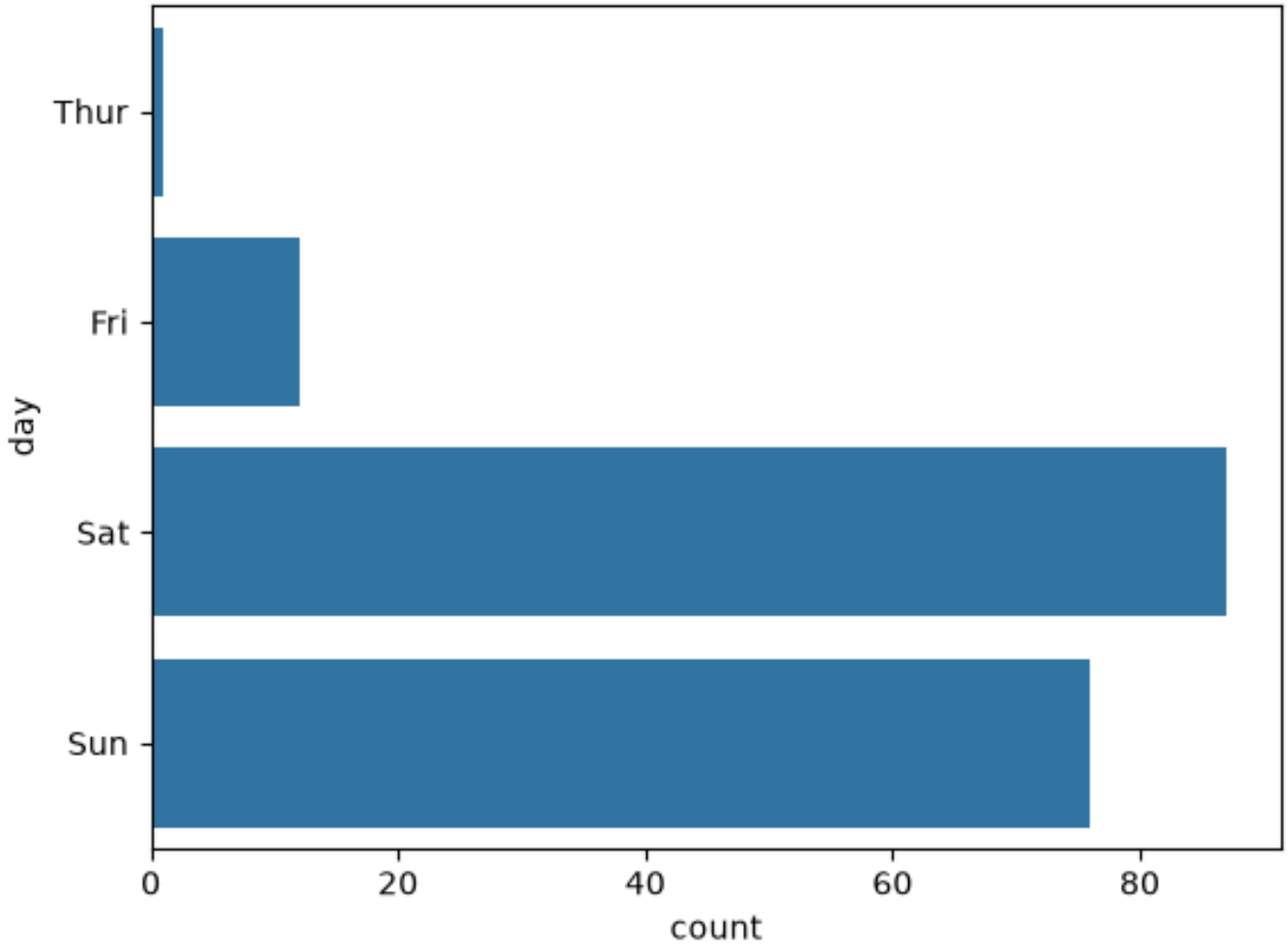
Code Cell 21

```
sns.countplot(tips[tips.time == 'Dinner']['day'])
```

Output

```
<Axes: xlabel='count', ylabel='day'>
```

Output



Code Cell 22

```
tips.time == 'Dinner'
```

Output

```
0      True
1      True
2      True
3      True
4      True
       ...
239    True
240    True
241    True
242    True
243    True
Name: time, Length: 244, dtype: bool
```

Experiment 8

Code Cell 1

```
import numpy as np
import pandas as pd
```

Code Cell 2

```
df=pd.read_csv('Salary_data.csv')
df
```

Output

```
    YearsExperience  Salary
0               1.1   39343
1               1.3   46205
2               1.5   37731
3               2.0   43525
4               2.2   39891
5               2.9   56642
6               3.0   60150
7               3.2   54445
8               3.2   64445
9               3.7   57189
10              3.9   63218
11              4.0   55794
12              4.0   56957
13              4.1   57081
14              4.5   61111
15              4.9   67938
16              5.1   66029
17              5.3   83088
18              5.9   81363
19              6.0   93940
20              6.8   91738
21              7.1   98273
22              7.9  101302
23              8.2  113812
24              8.7  109431
25              9.0  105582
26              9.5  116969
27              9.6  112635
28             10.3  122391
29             10.5  121872
```

Code Cell 3

```
df.info()
```

Output

```
<class 'pandas.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     int64  
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
```

Code Cell 4

```
df.dropna(inplace=True)
```

Code Cell 5

```
df.info()
```

Output

```
<class 'pandas.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     int64  
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
```

Code Cell 6

```
df.describe()
```

Output

```
       YearsExperience         Salary
count        30.000000      30.000000
mean          5.313333   76003.000000
std           2.837888   27414.429785
min           1.100000   37731.000000
25%           3.200000   56720.750000
50%           4.700000   65237.000000
75%           7.700000  100544.750000
max          10.500000  122391.000000
```

Code Cell 7

```
features=df.iloc[:,[0]].values
label=df.iloc[:,[1]].values
```

Code Cell 8

```
from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(
    features, label, test_size=0.2, random_state=42
)
```

Code Cell 9

```
from sklearn.linear_model import LinearRegression
model=LinearRegression()
model.fit(x_train,y_train)
```

Output

```
LinearRegression()
```

Code Cell 10

```
model.score(x_train,y_train)
```

Output

```
0.9645401573418146
```

Code Cell 11

```
model.score(x_test,y_test)
```

Output

```
0.9024461774180497
```

Code Cell 12

```
model.coef_
```

Output

```
array([[9423.81532303]])
```

Code Cell 13

```
model.intercept_
```

Output

```
array([25321.58301178])
```

Code Cell 14

```
import pickle
pickle.dump(model,open('SalaryPred.model','wb'))
```

Code Cell 15

```
model=pickle.load(open('SalaryPred.model','rb'))
```

Code Cell 16

```
yr_of_exp=float(input("Enter Years of Experience: "))
yr_of_exp_NP=np.array([[yr_of_exp]])
Salary=model.predict(yr_of_exp_NP)
```

Output

```
Enter Years of Experience:  44
```

Code Cell 17

```
print("Estimated Salary for {} years of experience is {}".format(yr_of_exp, Salary))
```

Output

```
Estimated Salary for 44.0 years of experience is [[439969.45722514]]
```

Experiment 9

Code Cell 1

```
import numpy as np
import pandas as pd
```

Code Cell 2

```
df=pd.read_csv('Social_Network_Ads.csv')
df
```

Output

```
      User ID  Gender  Age  EstimatedSalary  Purchased
0    15624510    Male   19            19000          0
1    15810944    Male   35            20000          0
2    15668575  Female   26            43000          0
3    15603246  Female   27            57000          0
4    15804002    Male   19            76000          0
..        ...     ...  ...              ...        ...
395  15691863  Female   46            41000          1
396  15706071    Male   51            23000          1
397  15654296  Female   50            20000          1
398  15755018    Male   36            33000          0
399  15594041  Female   49            36000          1

[400 rows x 5 columns]
```

Code Cell 3

```
df.head()
```

Output

```
    User ID  Gender  Age  EstimatedSalary  Purchased
0  15624510    Male   19            19000          0
1  15810944    Male   35            20000          0
2  15668575  Female   26            43000          0
3  15603246  Female   27            57000          0
4  15804002    Male   19            76000          0
```

Code Cell 4

```
features=df.iloc[:,[2,3]].values
label=df.iloc[:,4].values
features
```

Output

```
array([[    19,  19000],
       [    35,  20000],
       [    26,  43000],
       [    27,  57000],
       [    19,  76000],
       [    27,  58000],
       [    27,  84000],
       [    32, 150000],
       [    25,  33000],
       [    35,  65000],
       [    26,  80000],
       [    26,  52000],
       [    20,  86000],
       [    32,  18000],
       [    18,  82000],
       [    29,  80000],
       [    47,  25000],
       [    45,  26000],
       [    46,  28000],
       [    48,  29000],
       [    45,  22000],
       [    47,  49000],
       [    48,  41000],
       [    45,  22000],
       [    46,  23000],
       [    47,  20000],
       [    49,  28000],
       [    47,  30000],
       [    29,  43000],
       [    31,  18000],
       [    31,  74000],
       [    27, 137000],
       [    21,  16000],
       [    28,  44000],
       [    27,  90000],
       [    35,  27000],
       [    33,  28000],
       [    30,  49000],
       [    26,  72000],
       [    27,  31000],
       [    27,  17000],
       [    33,  51000],
       [    35, 108000],
       [    30,  15000],
       [    28,  84000],
       [    23,  20000],
       [    25,  79000],
       [    27,  54000],
       [    30, 135000],
       [    31,  89000],
       [    24,  32000],
       [    18,  44000],
       [    29,  83000],
       [    35,  23000],
       [    27,  58000],
       [    24,  55000],
       [    23,  48000],
       [    28,  79000],
       [    22,  18000],
       [    32, 117000],
       [    27,  20000],
       [    25,  87000],
       [    23,  66000],
       [    32, 120000],
       [    59,  83000],
       [    24,  58000],
       [    24,  19000],
       [    23,  82000],
       [    22,  63000],
       [    31,  68000],
       [    25,  80000],
       [    24,  27000],
       [    20,  23000],
       [    33, 113000],
       [    32,  18000],
       [    34, 112000],
       [    18,  52000],
       [    22,  27000],
       [    28,  87000],
       [    26,  17000],
       [    30,  80000],
       [    39,  42000],
       [    20,  49000],
       [    35,  88000],
```

```
[    30,  62000],
[    31, 118000],
[    24,  55000],
[    28,  85000],
[    26,  81000],
[    35,  50000],
[    22,  81000],
[    30, 116000],
[    26,  15000],
[    29,  28000],
[    29,  83000],
[    35,  44000],
[    35,  25000],
[    28, 123000],
[    35,  73000],
[    28,  37000],
[    27,  88000],
[    28,  59000],
[    32,  86000],
[    33, 149000],
[    19,  21000],
[    21,  72000],
[    26,  35000],
[    27,  89000],
[    26,  86000],
[    38,  80000],
[    39,  71000],
[    37,  71000],
[    38,  61000],
[    37,  55000],
[    42,  80000],
[    40,  57000],
[    35,  75000],
[    36,  52000],
[    40,  59000],
[    41,  59000],
[    36,  75000],
[    37,  72000],
[    40,  75000],
[    35,  53000],
[    41,  51000],
[    39,  61000],
[    42,  65000],
[    26,  32000],
[    30,  17000],
[    26,  84000],
[    31,  58000],
[    33,  31000],
[    30,  87000],
[    21,  68000],
[    28,  55000],
[    23,  63000],
[    20,  82000],
[    30, 107000],
[    28,  59000],
[    19,  25000],
[    19,  85000],
[    18,  68000],
[    35,  59000],
[    30,  89000],
[    34,  25000],
[    24,  89000],
[    27,  96000],
[    41,  30000],
[    29,  61000],
[    20,  74000],
[    26,  15000],
[    41,  45000],
[    31,  76000],
[    36,  50000],
[    40,  47000],
[    31,  15000],
[    46,  59000],
[    29,  75000],
[    26,  30000],
[    32, 135000],
[    32, 100000],
[    25,  90000],
[    37,  33000],
[    35,  38000],
[    33,  69000],
[    18,  86000],
[    22,  55000],
[    35,  71000],
[    29, 148000],
[    29,  47000],
```

```
[    21,  88000],
[    34, 115000],
[    26, 118000],
[    34,  43000],
[    34,  72000],
[    23,  28000],
[    35,  47000],
[    25,  22000],
[    24,  23000],
[    31,  34000],
[    26,  16000],
[    31,  71000],
[    32, 117000],
[    33,  43000],
[    33,  60000],
[    31,  66000],
[    20,  82000],
[    33,  41000],
[    35,  72000],
[    28,  32000],
[    24,  84000],
[    19,  26000],
[    29,  43000],
[    19,  70000],
[    28,  89000],
[    34,  43000],
[    30,  79000],
[    20,  36000],
[    26,  80000],
[    35,  22000],
[    35,  39000],
[    49,  74000],
[    39, 134000],
[    41,  71000],
[    58, 101000],
[    47,  47000],
[    55, 130000],
[    52, 114000],
[    40, 142000],
[    46,  22000],
[    48,  96000],
[    52, 150000],
[    59,  42000],
[    35,  58000],
[    47,  43000],
[    60, 108000],
[    49,  65000],
[    40,  78000],
[    46,  96000],
[    59, 143000],
[    41,  80000],
[    35,  91000],
[    37, 144000],
[    60, 102000],
[    35,  60000],
[    37,  53000],
[    36, 126000],
[    56, 133000],
[    40,  72000],
[    42,  80000],
[    35, 147000],
[    39,  42000],
[    40, 107000],
[    49,  86000],
[    38, 112000],
[    46,  79000],
[    40,  57000],
[    37,  80000],
[    46,  82000],
[    53, 143000],
[    42, 149000],
[    38,  59000],
[    50,  88000],
[    56, 104000],
[    41,  72000],
[    51, 146000],
[    35,  50000],
[    57, 122000],
[    41,  52000],
[    35,  97000],
[    44,  39000],
[    37,  52000],
[    48, 134000],
[    37, 146000],
[    50,  44000],
[    52,  90000],
```

```
[    41,  72000],
[    40,  57000],
[    58,  95000],
[    45, 131000],
[    35,  77000],
[    36, 144000],
[    55, 125000],
[    35,  72000],
[    48,  90000],
[    42, 108000],
[    40,  75000],
[    37,  74000],
[    47, 144000],
[    40,  61000],
[    43, 133000],
[    59,  76000],
[    60,  42000],
[    39, 106000],
[    57,  26000],
[    57,  74000],
[    38,  71000],
[    49,  88000],
[    52,  38000],
[    50,  36000],
[    59,  88000],
[    35,  61000],
[    37,  70000],
[    52,  21000],
[    48, 141000],
[    37,  93000],
[    37,  62000],
[    48, 138000],
[    41,  79000],
[    37,  78000],
[    39, 134000],
[    49,  89000],
[    55,  39000],
[    37,  77000],
[    35,  57000],
[    36,  63000],
[    42,  73000],
[    43, 112000],
[    45,  79000],
[    46, 117000],
[    58,  38000],
[    48,  74000],
[    37, 137000],
[    37,  79000],
[    40,  60000],
[    42,  54000],
[    51, 134000],
[    47, 113000],
[    36, 125000],
[    38,  50000],
[    42,  70000],
[    39,  96000],
[    38,  50000],
[    49, 141000],
[    39,  79000],
[    39,  75000],
[    54, 104000],
[    35,  55000],
[    45,  32000],
[    36,  60000],
[    52, 138000],
[    53,  82000],
[    41,  52000],
[    48,  30000],
[    48, 131000],
[    41,  60000],
[    41,  72000],
[    42,  75000],
[    36, 118000],
[    47, 107000],
[    38,  51000],
[    48, 119000],
[    42,  65000],
[    40,  65000],
[    57,  60000],
[    36,  54000],
[    58, 144000],
[    35,  79000],
[    38,  55000],
[    39, 122000],
[    53, 104000],
[    35,  75000],
```

```
       [    38,  65000],
       [    47,  51000],
       [    47, 105000],
       [    41,  63000],
       [    53,  72000],
       [    54, 108000],
       [    39,  77000],
       [    38,  61000],
       [    38, 113000],
       [    37,  75000],
       [    42,  90000],
       [    37,  57000],
       [    36,  99000],
       [    60,  34000],
       [    54,  70000],
       [    41,  72000],
       [    40,  71000],
       [    42,  54000],
       [    43, 129000],
       [    53,  34000],
       [    47,  50000],
       [    42,  79000],
       [    42, 104000],
       [    59,  29000],
       [    58,  47000],
       [    46,  88000],
       [    38,  71000],
       [    54,  26000],
       [    60,  46000],
       [    60,  83000],
       [    39,  73000],
       [    59, 130000],
       [    37,  80000],
       [    46,  32000],
       [    46,  74000],
       [    42,  53000],
       [    41,  87000],
       [    58,  23000],
       [    42,  64000],
       [    48,  33000],
       [    44, 139000],
       [    49,  28000],
       [    57,  33000],
       [    56,  60000],
       [    49,  39000],
       [    39,  71000],
       [    47,  34000],
       [    48,  35000],
       [    48,  33000],
       [    47,  23000],
       [    45,  45000],
       [    60,  42000],
       [    39,  59000],
       [    46,  41000],
       [    51,  23000],
       [    50,  20000],
       [    36,  33000],
       [    49,  36000]])
```

Code Cell 5

```
label
```

Output

```
array([0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1,
       0, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 0, 1, 1, 0,
       1, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 1, 1, 0,
       1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1,
       0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 0, 1,
       1, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 0, 0, 1, 1,
       0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0,
       1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1,
       0, 1, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1,
       1, 1, 0, 1])
```

Code Cell 6

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
```

Code Cell 7

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

for i in range(1, 401):
    x_train, x_test, y_train, y_test = train_test_split(
        features, label, test_size=0.2, random_state=i
    )

    model = LogisticRegression()
    model.fit(x_train, y_train)

    train_score = model.score(x_train, y_train)
    test_score = model.score(x_test, y_test)

    if test_score > train_score:
        print("Test {} Train {} Random State {}".format(test_score, train_score, i))
```

Output

```
Test 0.9 Train 0.840625 Random State 4
Test 0.8625 Train 0.85 Random State 5
Test 0.8625 Train 0.859375 Random State 6
Test 0.8875 Train 0.8375 Random State 7
Test 0.8625 Train 0.8375 Random State 9
Test 0.9 Train 0.840625 Random State 10
Test 0.8625 Train 0.85625 Random State 14
Test 0.85 Train 0.84375 Random State 15
Test 0.8625 Train 0.85625 Random State 16
Test 0.875 Train 0.834375 Random State 18
Test 0.85 Train 0.84375 Random State 19
Test 0.875 Train 0.84375 Random State 20
Test 0.8625 Train 0.834375 Random State 21
Test 0.875 Train 0.840625 Random State 22
Test 0.875 Train 0.840625 Random State 24
Test 0.85 Train 0.834375 Random State 26
Test 0.85 Train 0.840625 Random State 27
Test 0.8625 Train 0.834375 Random State 30
Test 0.8625 Train 0.85625 Random State 31
Test 0.875 Train 0.853125 Random State 32
Test 0.8625 Train 0.84375 Random State 33
Test 0.875 Train 0.83125 Random State 35
Test 0.8625 Train 0.853125 Random State 36
Test 0.8875 Train 0.840625 Random State 38
Test 0.875 Train 0.8375 Random State 39
Test 0.8875 Train 0.8375 Random State 42
Test 0.875 Train 0.846875 Random State 46
Test 0.9125 Train 0.83125 Random State 47
Test 0.875 Train 0.83125 Random State 51
Test 0.9 Train 0.84375 Random State 54
Test 0.85 Train 0.84375 Random State 57
Test 0.875 Train 0.84375 Random State 58
Test 0.925 Train 0.8375 Random State 61
Test 0.8875 Train 0.834375 Random State 65
Test 0.8875 Train 0.840625 Random State 68
Test 0.9 Train 0.83125 Random State 72
Test 0.8875 Train 0.8375 Random State 75
Test 0.925 Train 0.825 Random State 76
Test 0.8625 Train 0.840625 Random State 77
Test 0.8625 Train 0.859375 Random State 81
Test 0.875 Train 0.8375 Random State 82
Test 0.8875 Train 0.8375 Random State 83
Test 0.8625 Train 0.853125 Random State 84
Test 0.8625 Train 0.840625 Random State 85
Test 0.8625 Train 0.840625 Random State 87
Test 0.875 Train 0.846875 Random State 88
Test 0.9125 Train 0.8375 Random State 90
Test 0.8625 Train 0.85 Random State 95
Test 0.875 Train 0.85 Random State 99
Test 0.85 Train 0.840625 Random State 101
Test 0.85 Train 0.840625 Random State 102
Test 0.9 Train 0.825 Random State 106
Test 0.8625 Train 0.840625 Random State 107
Test 0.85 Train 0.834375 Random State 109
Test 0.85 Train 0.840625 Random State 111
Test 0.9125 Train 0.840625 Random State 112
Test 0.8625 Train 0.85 Random State 115
Test 0.8625 Train 0.840625 Random State 116
Test 0.875 Train 0.834375 Random State 119
Test 0.9125 Train 0.828125 Random State 120
Test 0.8625 Train 0.859375 Random State 125
Test 0.85 Train 0.846875 Random State 128
Test 0.875 Train 0.85 Random State 130
Test 0.9 Train 0.84375 Random State 133
Test 0.925 Train 0.834375 Random State 134
Test 0.8625 Train 0.85 Random State 135
Test 0.875 Train 0.83125 Random State 138
Test 0.8625 Train 0.85 Random State 141
Test 0.85 Train 0.846875 Random State 143
Test 0.85 Train 0.846875 Random State 146
Test 0.85 Train 0.84375 Random State 147
Test 0.8625 Train 0.85 Random State 148
Test 0.875 Train 0.8375 Random State 150
Test 0.8875 Train 0.83125 Random State 151
Test 0.925 Train 0.84375 Random State 152
Test 0.85 Train 0.840625 Random State 153
Test 0.9 Train 0.84375 Random State 154
Test 0.9 Train 0.840625 Random State 155
Test 0.8875 Train 0.846875 Random State 156
Test 0.8875 Train 0.834375 Random State 158
Test 0.875 Train 0.828125 Random State 159
Test 0.9 Train 0.83125 Random State 161
Test 0.85 Train 0.8375 Random State 163
Test 0.875 Train 0.83125 Random State 164
```

```
Test 0.8625 Train 0.85 Random State 169
Test 0.875 Train 0.840625 Random State 171
Test 0.85 Train 0.840625 Random State 172
Test 0.9 Train 0.825 Random State 180
Test 0.85 Train 0.834375 Random State 184
Test 0.925 Train 0.821875 Random State 186
Test 0.9 Train 0.83125 Random State 193
Test 0.8625 Train 0.85 Random State 195
Test 0.8625 Train 0.840625 Random State 196
Test 0.8625 Train 0.8375 Random State 197
Test 0.875 Train 0.840625 Random State 198
Test 0.8875 Train 0.8375 Random State 199
Test 0.8875 Train 0.84375 Random State 200
Test 0.8625 Train 0.8375 Random State 202
Test 0.8625 Train 0.840625 Random State 203
Test 0.8875 Train 0.83125 Random State 206
Test 0.8625 Train 0.834375 Random State 211
Test 0.85 Train 0.84375 Random State 212
Test 0.8625 Train 0.834375 Random State 214
Test 0.875 Train 0.83125 Random State 217
Test 0.9625 Train 0.81875 Random State 220
Test 0.875 Train 0.84375 Random State 221
Test 0.85 Train 0.840625 Random State 222
Test 0.9 Train 0.84375 Random State 223
Test 0.8625 Train 0.853125 Random State 227
Test 0.8625 Train 0.834375 Random State 228
Test 0.9 Train 0.840625 Random State 229
Test 0.85 Train 0.84375 Random State 232
Test 0.875 Train 0.846875 Random State 233
Test 0.9125 Train 0.840625 Random State 234
Test 0.8625 Train 0.840625 Random State 235
Test 0.85 Train 0.846875 Random State 236
Test 0.875 Train 0.846875 Random State 239
Test 0.85 Train 0.84375 Random State 241
Test 0.8875 Train 0.85 Random State 242
Test 0.8875 Train 0.825 Random State 243
Test 0.875 Train 0.846875 Random State 244
Test 0.875 Train 0.840625 Random State 245
Test 0.875 Train 0.846875 Random State 246
Test 0.8625 Train 0.859375 Random State 247
Test 0.8875 Train 0.84375 Random State 248
Test 0.8625 Train 0.85 Random State 250
Test 0.875 Train 0.83125 Random State 251
Test 0.8875 Train 0.84375 Random State 252
Test 0.8625 Train 0.846875 Random State 255
Test 0.9 Train 0.840625 Random State 257
Test 0.8625 Train 0.85625 Random State 260
Test 0.8625 Train 0.840625 Random State 266
Test 0.8625 Train 0.8375 Random State 268
Test 0.875 Train 0.840625 Random State 275
Test 0.8625 Train 0.85 Random State 276
Test 0.925 Train 0.8375 Random State 277
Test 0.875 Train 0.846875 Random State 282
Test 0.85 Train 0.846875 Random State 283
Test 0.85 Train 0.84375 Random State 285
Test 0.9125 Train 0.834375 Random State 286
Test 0.85 Train 0.840625 Random State 290
Test 0.85 Train 0.840625 Random State 291
Test 0.85 Train 0.846875 Random State 292
Test 0.8625 Train 0.8375 Random State 294
Test 0.8875 Train 0.828125 Random State 297
Test 0.8625 Train 0.834375 Random State 300
Test 0.8625 Train 0.85 Random State 301
Test 0.8875 Train 0.85 Random State 302
Test 0.875 Train 0.846875 Random State 303
Test 0.8625 Train 0.834375 Random State 305
Test 0.9125 Train 0.8375 Random State 306
Test 0.875 Train 0.846875 Random State 308
Test 0.9 Train 0.84375 Random State 311
Test 0.8625 Train 0.834375 Random State 313
Test 0.9125 Train 0.834375 Random State 314
Test 0.875 Train 0.8375 Random State 315
Test 0.9 Train 0.846875 Random State 317
Test 0.9125 Train 0.821875 Random State 319
Test 0.8625 Train 0.85 Random State 321
Test 0.9125 Train 0.828125 Random State 322
Test 0.85 Train 0.846875 Random State 328
Test 0.85 Train 0.8375 Random State 332
Test 0.8875 Train 0.853125 Random State 336
Test 0.85 Train 0.8375 Random State 337
Test 0.875 Train 0.840625 Random State 343
Test 0.8625 Train 0.84375 Random State 346
Test 0.8875 Train 0.83125 Random State 351
Test 0.8625 Train 0.85 Random State 352
Test 0.95 Train 0.81875 Random State 354
Test 0.8625 Train 0.85 Random State 356
```

```
Test 0.9125 Train 0.840625 Random State 357
Test 0.8625 Train 0.8375 Random State 358
Test 0.85 Train 0.840625 Random State 362
Test 0.9 Train 0.84375 Random State 363
Test 0.8625 Train 0.853125 Random State 364
Test 0.9375 Train 0.821875 Random State 366
Test 0.9125 Train 0.840625 Random State 369
Test 0.8625 Train 0.853125 Random State 371
Test 0.925 Train 0.834375 Random State 376
Test 0.9125 Train 0.828125 Random State 377
Test 0.8875 Train 0.85 Random State 378
Test 0.8875 Train 0.85 Random State 379
Test 0.8625 Train 0.840625 Random State 382
Test 0.8625 Train 0.859375 Random State 386
Test 0.85 Train 0.8375 Random State 387
Test 0.875 Train 0.828125 Random State 388
Test 0.85 Train 0.84375 Random State 394
Test 0.8625 Train 0.8375 Random State 395
Test 0.9 Train 0.84375 Random State 397
Test 0.8625 Train 0.84375 Random State 400
```

Code Cell 8

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

x_train, x_test, y_train, y_test = train_test_split(
    features, label, test_size=0.2, random_state=42
)

finalModel = LogisticRegression()
finalModel.fit(x_train, y_train)
```

Output

```
LogisticRegression()
```

Code Cell 9

```
print(finalModel.score(x_train,y_train))
print(finalModel.score(x_test,y_test))
```

Output

```
0.8375
0.8875
```

Code Cell 10

```
from sklearn.metrics import classification_report
print(classification_report(label,finalModel.predict(features)))
```

Output

```
              precision    recall  f1-score   support

           0       0.85      0.93      0.89       257
           1       0.85      0.70      0.77       143

    accuracy                           0.85       400
   macro avg       0.85      0.81      0.83       400
weighted avg       0.85      0.85      0.84       400
```

Experiment 10

Code Cell 1

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```

Code Cell 2

```
df=pd.read_csv('Mall_Customers.csv')
```

Code Cell 3

```
df.info()
```

Output

```
<class 'pandas.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Gender                  200 non-null    str
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), str(1)
memory usage: 8.9 KB
```

Code Cell 4

```
df.head()
```

Output

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

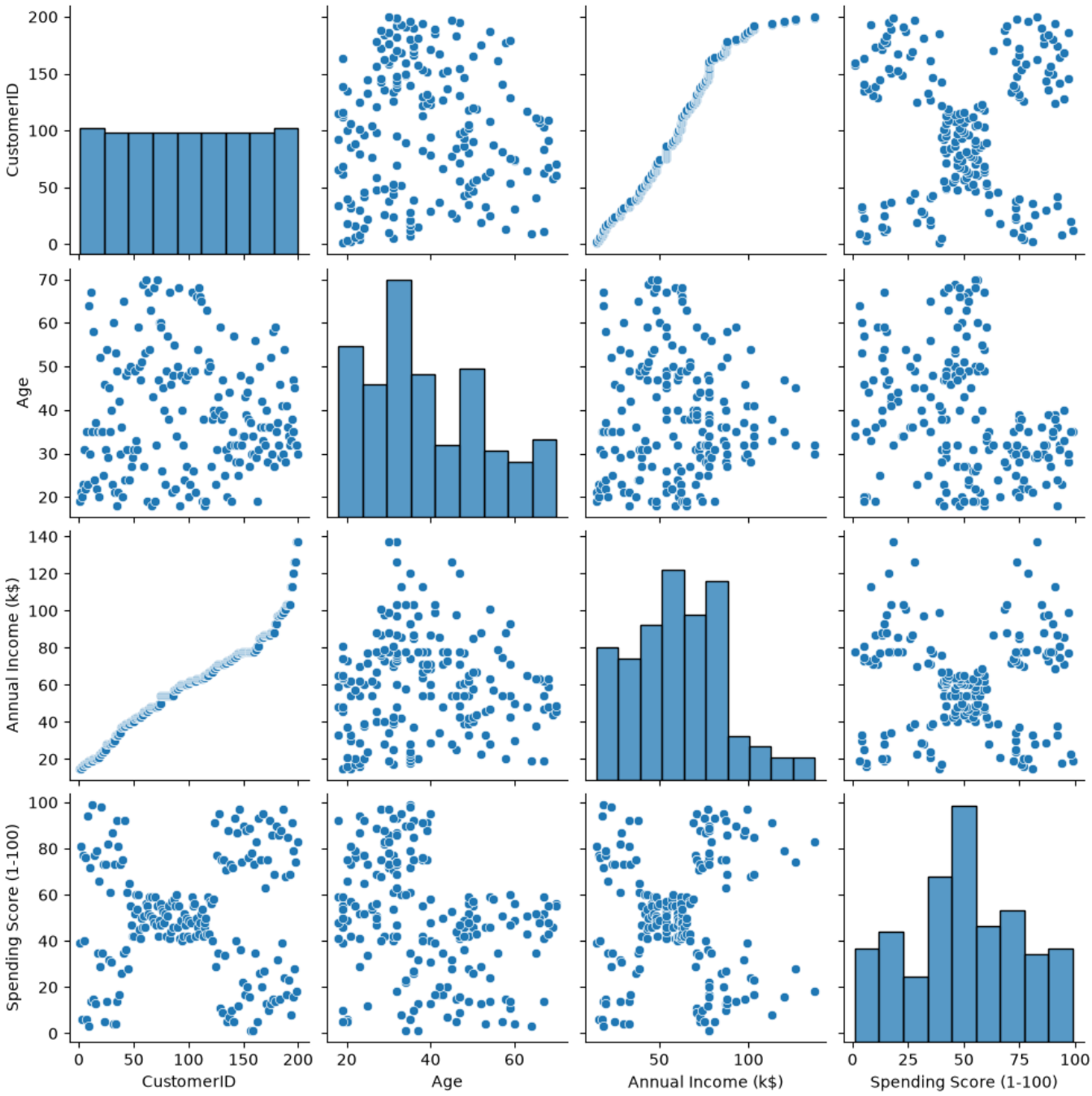
Code Cell 5

```
sns.pairplot(df)
```

Output

```
<seaborn.axisgrid.PairGrid at 0x1ed01d60050>
```

Output



Code Cell 6

```
features=df.iloc[:,[3,4]].values
```

Code Cell 7

```
from sklearn.cluster import KMeans
model=KMeans(n_clusters=5)
model.fit(features)
KMeans(n_clusters=5)
```

Output

```
C:\Users\shrey\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1428: UserWarning: KMeans is known to
have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it
by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
```

Output

```
KMeans(n_clusters=5)
```

Code Cell 8

```
Final=df.iloc[:,[3,4]]
Final['label']=model.predict(features)
Final.head()
```

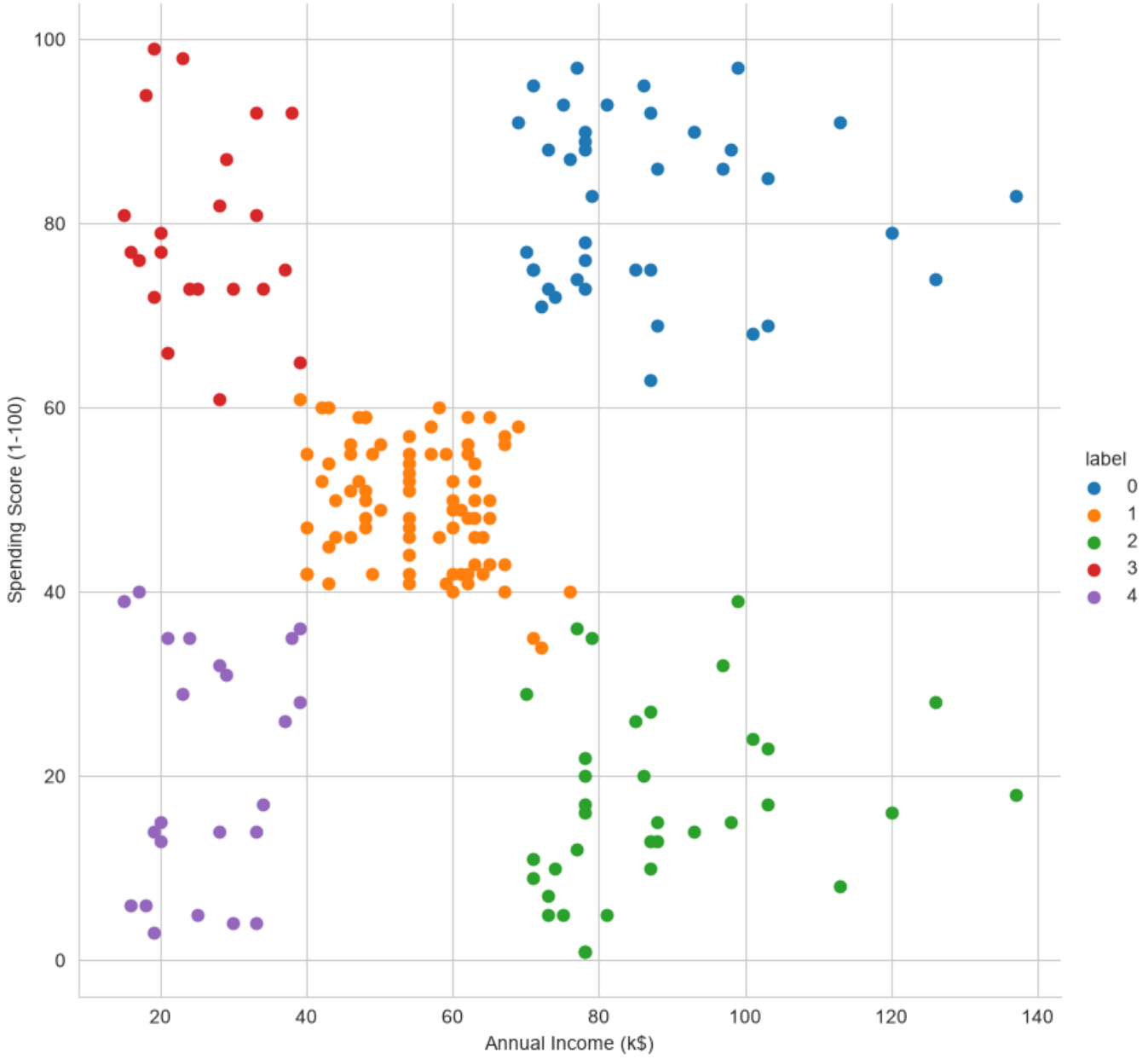
Output

```
   Annual Income (k$)  Spending Score (1-100)  label
0                  15                      39      4
1                  15                      81      3
2                  16                       6      4
3                  16                      77      3
4                  17                      40      4
```

Code Cell 9

```
sns.set_style("whitegrid")
sns.FacetGrid(Final,hue="label",height=8) \
.map(plt.scatter,"Annual Income (k$)", "Spending Score (1-100)") \
.add_legend();
plt.show()
```

Output



Code Cell 10

```
features_el=df.iloc[:,[2,3,4]].values
from sklearn.cluster import KMeans
wcss=[]
for i in range(1,10):
    model=KMeans(n_clusters=i)
    model.fit(features_el)
    wcss.append(model.inertia_)
plt.plot(range(1,10),wcss)
```

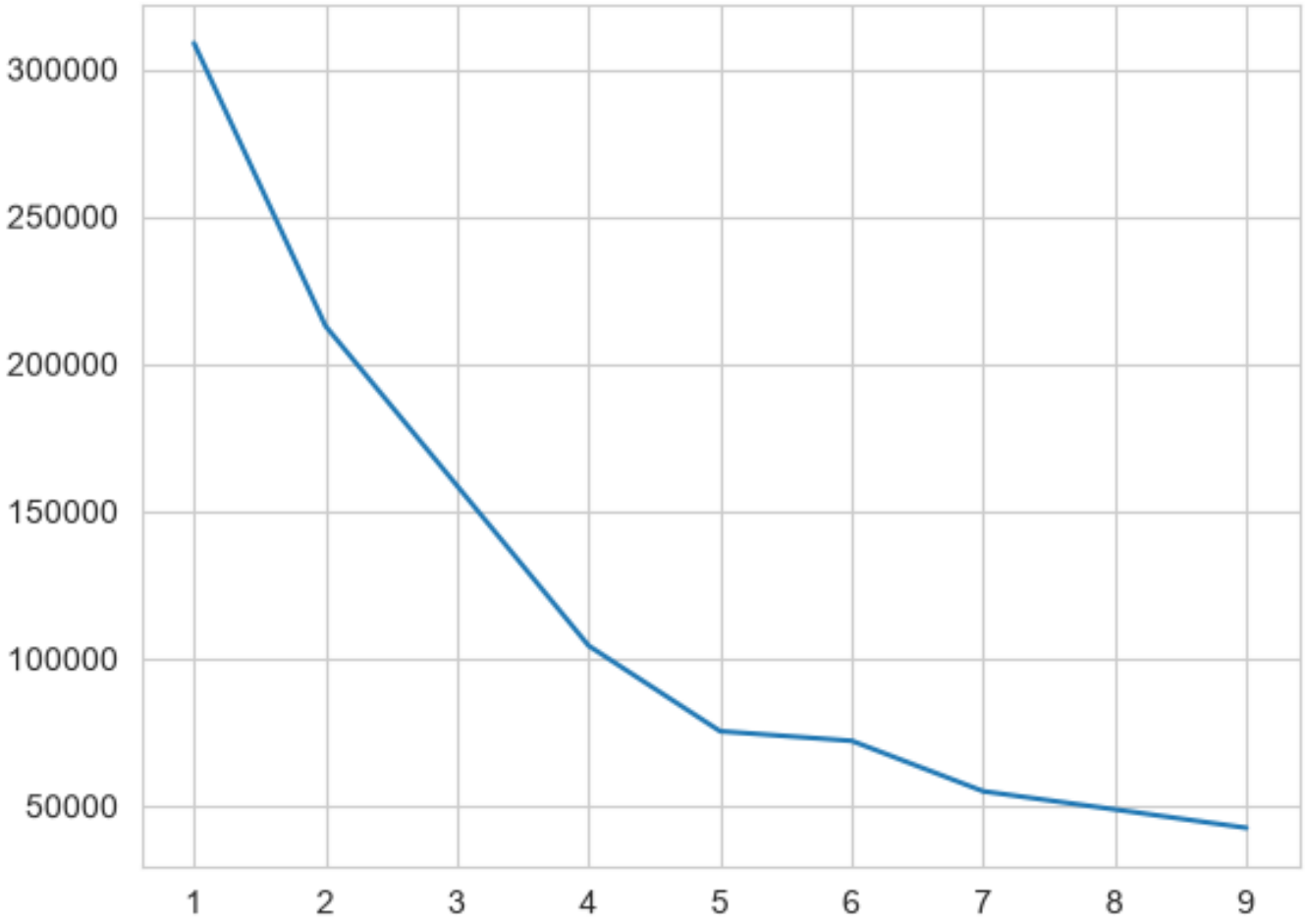
Output

```
C:\Users\shrey\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1428: UserWarning: KMeans is known to
have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it
by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\shrey\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1428: UserWarning: KMeans is known to
have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it
by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\shrey\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1428: UserWarning: KMeans is known to
have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it
by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\shrey\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1428: UserWarning: KMeans is known to
have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it
by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\shrey\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1428: UserWarning: KMeans is known to
have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it
by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\shrey\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1428: UserWarning: KMeans is known to
have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it
by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\shrey\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1428: UserWarning: KMeans is known to
have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it
by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\shrey\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1428: UserWarning: KMeans is known to
have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it
by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\shrey\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1428: UserWarning: KMeans is known to
have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it
by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\shrey\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1428: UserWarning: KMeans is known to
have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it
by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
```

Output

```
[<matplotlib.lines.Line2D at 0x1ed05502cf0>]
```

Output



Experiment 11

Code Cell 1

```
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, classification_report
```

Code Cell 2

```
df = pd.read_csv("Iris.csv")
```

Code Cell 3

```
df.info()
```

Output

```
<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype  
---  ------        --------------  -----  
 0   sepal.length  150 non-null    float64
 1   sepal.width   150 non-null    float64
 2   petal.length  150 non-null    float64
 3   petal.width   150 non-null    float64
 4   variety       150 non-null    str    
dtypes: float64(4), str(1)
memory usage: 7.2 KB
```

Code Cell 4

```
print(df["variety"].value_counts())
```

Output

```
variety
Setosa        50
Versicolor    50
Virginica     50
Name: count, dtype: int64
```

Code Cell 5

```
print(df.head())
```

Output

```
   sepal.length  sepal.width  petal.length  petal.width variety
0           5.1          3.5           1.4          0.2  Setosa
1           4.9          3.0           1.4          0.2  Setosa
2           4.7          3.2           1.3          0.2  Setosa
3           4.6          3.1           1.5          0.2  Setosa
4           5.0          3.6           1.4          0.2  Setosa
```

Code Cell 6

```
features = df.iloc[:, :-1].to_numpy()
label = df.iloc[:, -1].to_numpy()
```

Code Cell 7

```
from sklearn.model_selection import train_test_split

xtrain, xtest, ytrain, ytest = train_test_split(
    features.tolist(),
    label.tolist(),
    test_size=0.2,
    random_state=42
)
```

Code Cell 8

```
print(len(xtrain))
print(len(xtest))
print(len(ytrain))
print(len(ytest))
```

Output

```
120
30
120
30
```

Code Cell 9

```
from sklearn.neighbors import KNeighborsClassifier

model_KNN = KNeighborsClassifier(n_neighbors=5)
```

Code Cell 10

```
model_KNN.fit(xtrain, ytrain)
```

Output

```
KNeighborsClassifier()
```

Code Cell 11

```
print(model_KNN.score(xtrain, ytrain))
```

Output

```
0.9666666666666667
```

Code Cell 12

```
print(model_KNN.score(xtest, ytest))
```

Output

```
1.0
```

Code Cell 13

```
print(confusion_matrix(label, model_KNN.predict(features)))
```

Output

```
[[50  0  0]
 [ 0 47  3]
 [ 0  1 49]]
```

Code Cell 14

```
print(classification_report(label, model_KNN.predict(features)))
```

Output

```
              precision    recall  f1-score   support

      Setosa       1.00      1.00      1.00        50
  Versicolor       0.98      0.94      0.96        50
   Virginica       0.94      0.98      0.96        50

    accuracy                           0.97       150
   macro avg       0.97      0.97      0.97       150
weighted avg       0.97      0.97      0.97       150
```

Experiment 12

Code Cell 1

```
import numpy as np
```

Code Cell 2

```
from scipy import stats
```

Code Cell 3

```
marks=np.array([72,68,75,70,74,69,71,73,70,72])
```

Code Cell 4

```
mu_0 = 70
```

Code Cell 5

```
t_stat, p_value = stats.ttest_1samp(marks, mu_0)
print(f"T-statistic: {t_stat:.3f}")
print(f"P-value: {p_value:.4f}")
alpha = 0.05
if p_value < alpha:
    print("Reject Null Hypothesis → Mean is significantly different from 70.")
else:
    print("Fail to Reject Null Hypothesis → No significant difference.")
```

Output

```
T-statistic: 1.993
P-value: 0.0774
Fail to Reject Null Hypothesis → No significant difference.
```

Code Cell 6

```
import numpy as np
```

Code Cell 7

```
from math import sqrt
from scipy.stats import norm
```

Code Cell 8

```
x_bar = 51.2
mu_0 = 50
sigma = 3
n = 36
```

Code Cell 9

```
z_stat = (x_bar - mu_0)/(sigma/sqrt(n))
```

Code Cell 10

```
p_value = 2 * (1 - norm.cdf(abs(z_stat)))
print(f"Z-statistic: {z_stat:.3f}")
print(f"P-value: {p_value:.4f}")
alpha = 0.05
if p_value < alpha:
    print("Reject Null Hypothesis → Mean is significantly different from 50 g.")
else:
    print("Fail to Reject Null Hypothesis → No significant difference.")
```

Output

```
Z-statistic: 2.400
P-value: 0.0164
Reject Null Hypothesis → Mean is significantly different from 50 g.
```

Code Cell 11

```
import numpy as np
from scipy import stats
```

Code Cell 12

```
A = [20, 22, 23]
B = [19, 20, 18]
C = [25, 27, 26]
```

Code Cell 13

```
f_stat, p_value = stats.f_oneway(A, B, C)

print(f"F-statistic: {f_stat:.3f}")
print(f"P-value: {p_value:.4f}")

alpha = 0.05
if p_value < alpha:
    print("Reject Null Hypothesis → Means are significantly different.")
else:
    print("Fail to Reject Null Hypothesis → No significant difference.")
```

Output

```
F-statistic: 25.923
P-value: 0.0011
Reject Null Hypothesis → Means are significantly different.
```

Experiment 13

Code Cell 1

```
import matplotlib.pyplot as plt
```

Code Cell 2

```
Overs = list(range(5, 51, 5))

Indian_Score = [30, 55, 90, 129, 165, 200, 239, 270, 310, 350]

Srilankan_Score = [25, 70, 90, 120, 140, 170, 195, 220, 255, 279]
```

Code Cell 3

```
plt.plot(Overs, Indian_Score, color="green", label="INDIA")

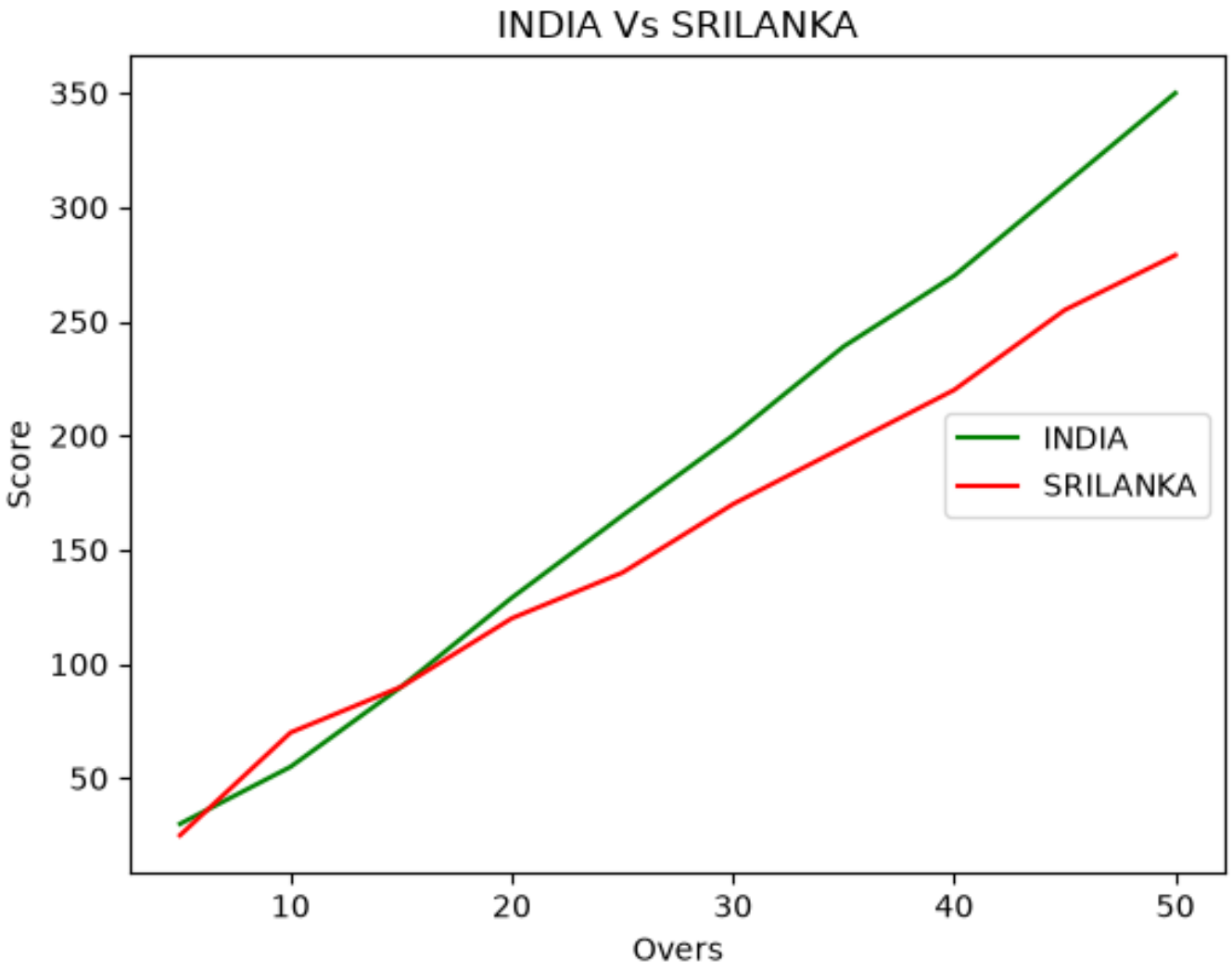
plt.plot(Overs, Srilankan_Score, color="red", label="SRILANKA")

plt.title("INDIA Vs SRILANKA")
plt.xlabel("Overs")
plt.ylabel("Score")

plt.legend(loc="center right")

plt.show()
```

Output



Code Cell 4

```
import matplotlib.pyplot as plt
```

Code Cell 5

```
Names = ['SHREE', 'DEV', 'KEERTHI', 'PRIYA', 'SHAN', 'KUMARAN']

Percentage_hsc = [96, 91, 94, 75, 45, 81]
```

Code Cell 6

```
plt.bar(Names, Percentage_hsc)

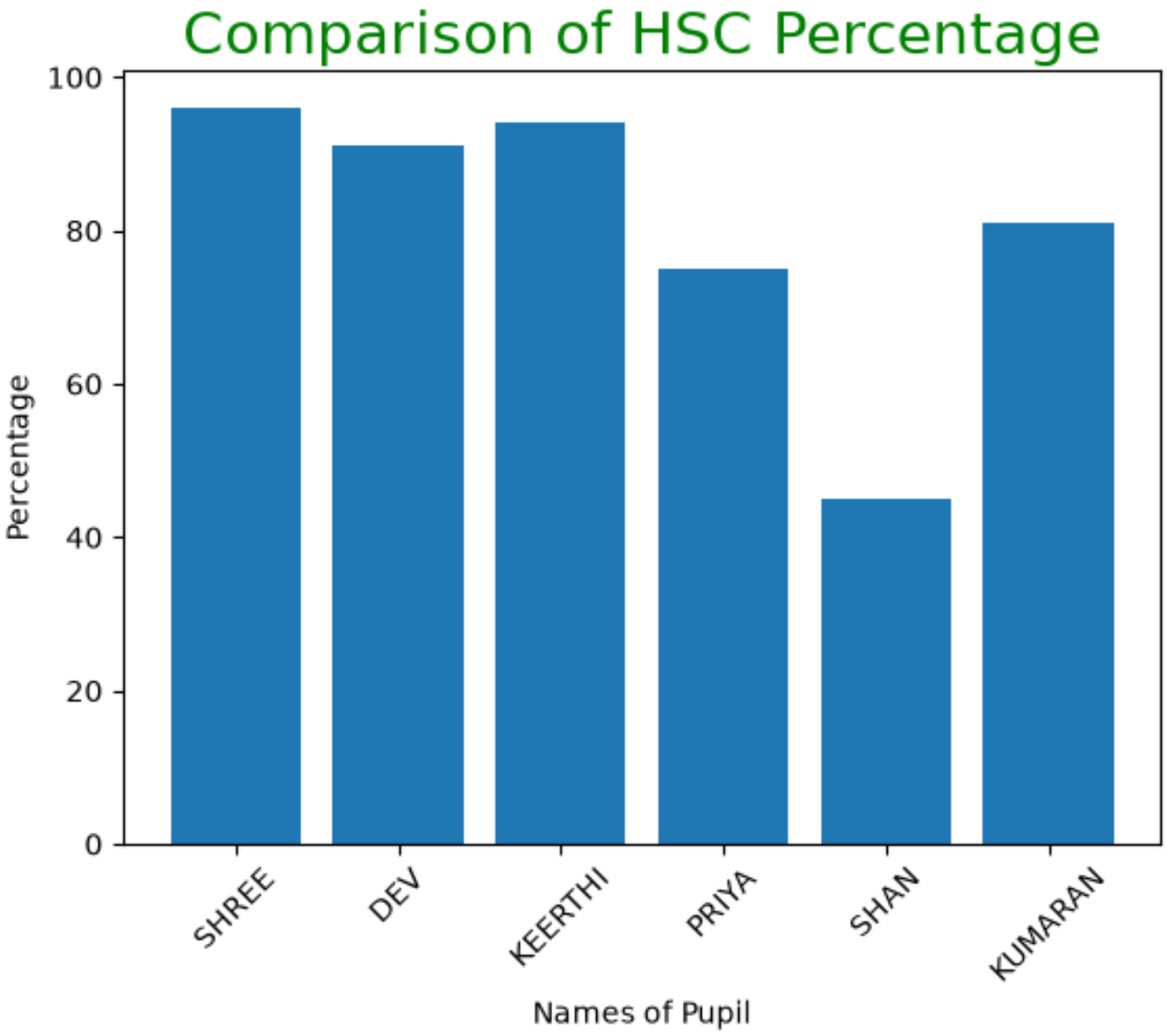
plt.xticks(rotation=45)

plt.xlabel("Names of Pupil")
plt.ylabel("Percentage")

plt.title("Comparison of HSC Percentage", fontsize=20, color="green")

plt.show()
```

Output



Code Cell 7

```
import matplotlib.pyplot as plt
```

Code Cell 8

```
labels = ['CANDIDATE 1', 'CANDIDATE 2', 'CANDIDATE 3', 'CANDIDATE 4']

Votes = [315, 130, 245, 210]

colors = ['green', 'yellow', 'red', 'orange']

explode = (0.2, 0, 0, 0)
```

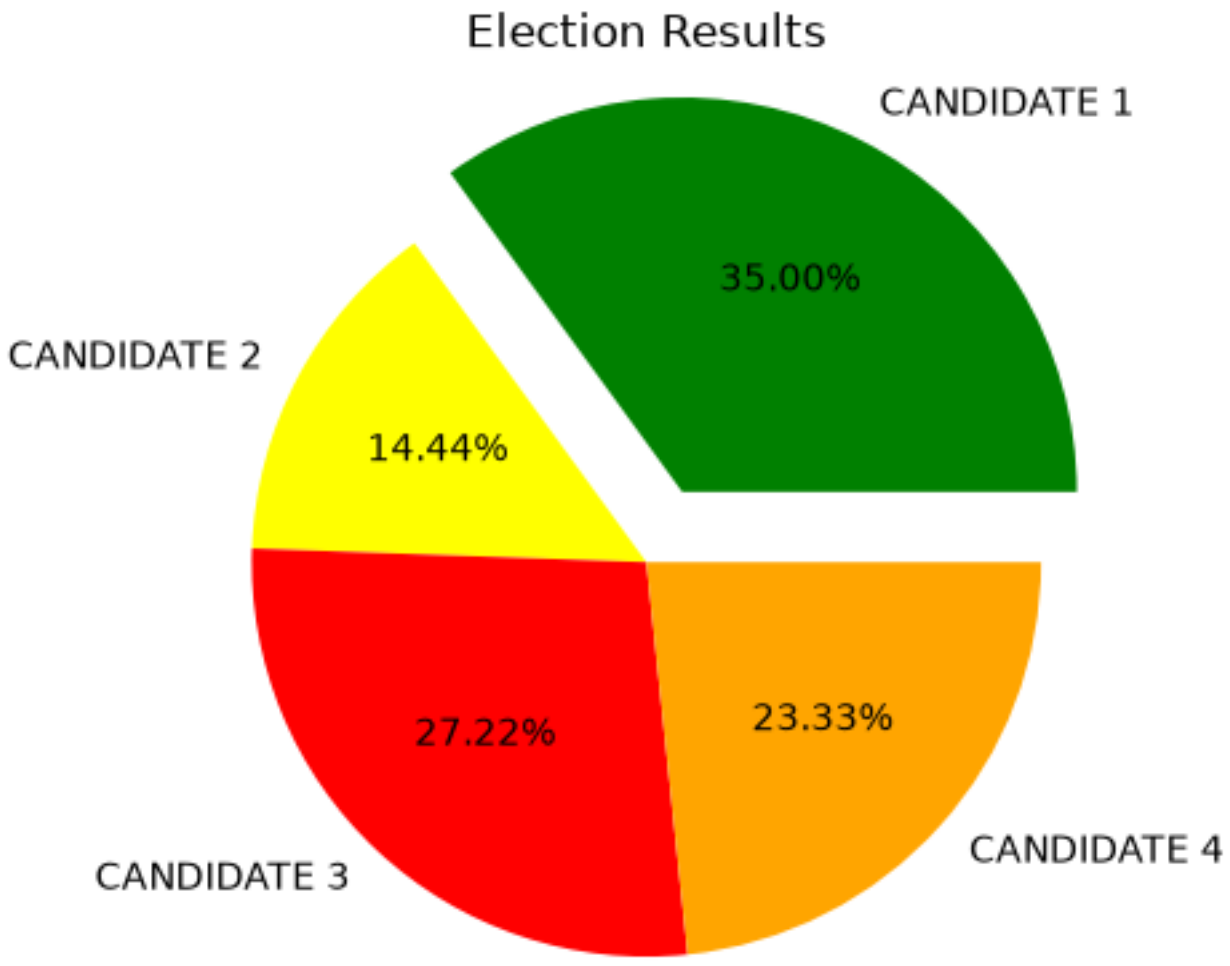
Code Cell 9

```
plt.pie(
    Votes,
    labels=labels,
    colors=colors,
    explode=explode,
    autopct='%0.2f%%'
)

plt.title("Election Results")

plt.show()
```

Output



Experiment 14

Code Cell 1

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import gutenberg
```

Code Cell 2

```
nltk.download('gutenberg')
nltk.download('punkt')
nltk.download('punkt_tab')
```

Output

```
[nltk_data] Downloading package gutenberg to
[nltk_data]     C:\Users\shrey\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping corpora\gutenberg.zip.
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\shrey\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping tokenizers\punkt.zip.
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\shrey\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping tokenizers\punkt_tab.zip.
```

Output

```
True
```

Code Cell 3

```
sample = gutenberg.raw("austen-emma.txt")
token = word_tokenize(sample)

print(token[:10])
```

Output

```
['[', 'Emma', 'by', 'Jane', 'Austen', '1816', ']', 'VOLUME', 'I', 'CHAPTER']
```

Code Cell 4

```
wlist = []

for i in range(50):
    wlist.append(token[i])

print(wlist)
```

Output

```
['[', 'Emma', 'by', 'Jane', 'Austen', '1816', ']', 'VOLUME', 'I', 'CHAPTER', 'I', 'Emma', 'Woodhouse', ',',
'handsome', ',', 'clever', ',', 'and', 'rich', ',', 'with', 'a', 'comfortable', 'home', 'and', 'happy',
'disposition', ',', 'seemed', 'to', 'unite', 'some', 'of', 'the', 'best', 'blessings', 'of', 'existence', ';',
 'and', 'had', 'lived', 'nearly', 'twenty-one', 'years', 'in', 'the', 'world', 'with']
```

Code Cell 5

```
wordfreq = [wlist.count(w) for w in wlist]

print("Pairs\n" + str(list(zip(wlist, wordfreq))))
```

Output

```
Pairs
[('[', 1), ('Emma', 2), ('by', 1), ('Jane', 1), ('Austen', 1), ('1816', 1), (']', 1), ('VOLUME', 1), ('I', 2),
 ('CHAPTER', 1), ('I', 2), ('Emma', 2), ('Woodhouse', 1), (',', 5), ('handsome', 1), (',', 5), ('clever', 1),
(',', 5), ('and', 3), ('rich', 1), (',', 5), ('with', 2), ('a', 1), ('comfortable', 1), ('home', 1), ('and',
3), ('happy', 1), ('disposition', 1), (',', 5), ('seemed', 1), ('to', 1), ('unite', 1), ('some', 1), ('of',
2), ('the', 2), ('best', 1), ('blessings', 1), ('of', 2), ('existence', 1), (';', 1), ('and', 3), ('had', 1),
('lived', 1), ('nearly', 1), ('twenty-one', 1), ('years', 1), ('in', 1), ('the', 2), ('world', 1), ('with',
2)]
```

Experiment 15

Code Cell 1

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Code Cell 2

```
diabetes_df = pd.read_csv("diabetes.csv")
print(diabetes_df.head())
```

Output

```
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1
```

Code Cell 3

```
print(diabetes_df.info())
```

Output

```
<class 'pandas.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               20 non-null     int64
 1   Glucose                   20 non-null     int64
 2   BloodPressure             20 non-null     int64
 3   SkinThickness             20 non-null     int64
 4   Insulin                   20 non-null     int64
 5   BMI                       20 non-null     float64
 6   DiabetesPedigreeFunction  20 non-null     float64
 7   Age                       20 non-null     int64
 8   Outcome                   20 non-null     int64
dtypes: float64(2), int64(7)
memory usage: 1.5 KB
None
```

Code Cell 4

```
print(diabetes_df.shape)
```

Output

```
(20, 9)
```

Code Cell 5

```
print(diabetes_df.isnull().sum())
```

Output

```
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64
```

Code Cell 6

```
print(diabetes_df.describe())
```

Output

```
       Pregnancies     Glucose  BloodPressure  SkinThickness     Insulin  \
count     20.00000   20.000000      20.000000      20.000000   20.000000
mean       4.50000  129.400000      61.700000      17.800000  116.150000
std        3.56149   35.354446      26.159631      17.733703  215.843821
min        0.00000   78.000000       0.000000       0.000000    0.000000
25%        1.00000  106.000000      57.500000       0.000000    0.000000
50%        4.50000  117.000000      70.000000      21.000000    0.000000
75%        7.25000  152.500000      74.000000      32.750000  114.000000
max       10.00000  197.000000      96.000000      47.000000  846.000000

             BMI  DiabetesPedigreeFunction        Age    Outcome
count  20.000000                 20.000000  20.000000  20.000000
mean   30.950000                  0.511650  37.450000   0.600000
std     9.654424                  0.513691  11.591626   0.502625
min     0.000000                  0.134000  21.000000   0.000000
25%    26.975000                  0.198500  30.750000   0.000000
50%    30.300000                  0.374500  32.000000   1.000000
75%    35.875000                  0.560000  50.250000   1.000000
max    45.800000                  2.288000  59.000000   1.000000
```

Code Cell 7

```
print(diabetes_df.columns)
```

Output

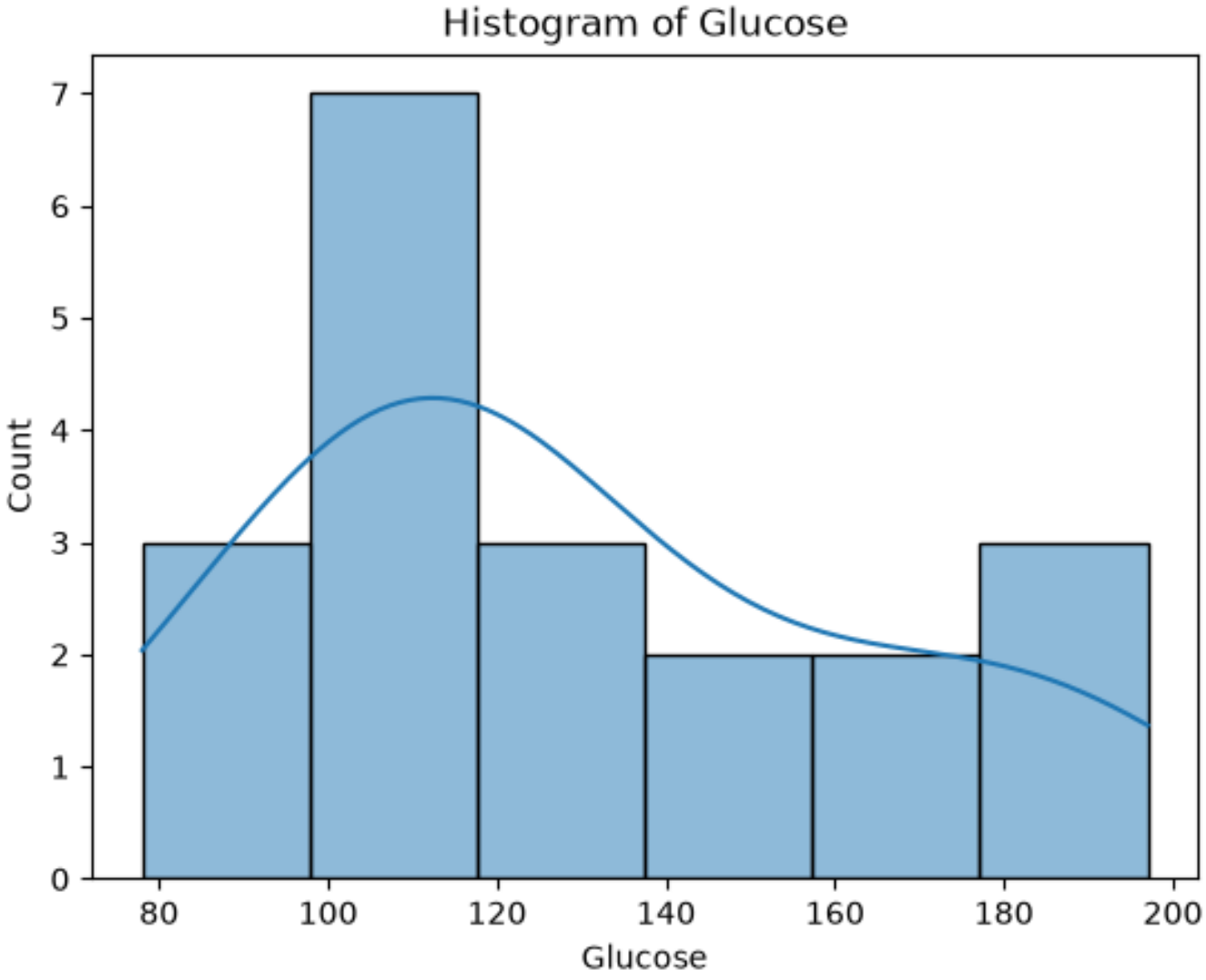
```
Index(['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin',
       'BMI', 'DiabetesPedigreeFunction', 'Age', 'Outcome'],
      dtype='str')
```

Code Cell 8

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.histplot(diabetes_df['Glucose'], kde=True)
plt.title("Histogram of Glucose")
plt.show()
```

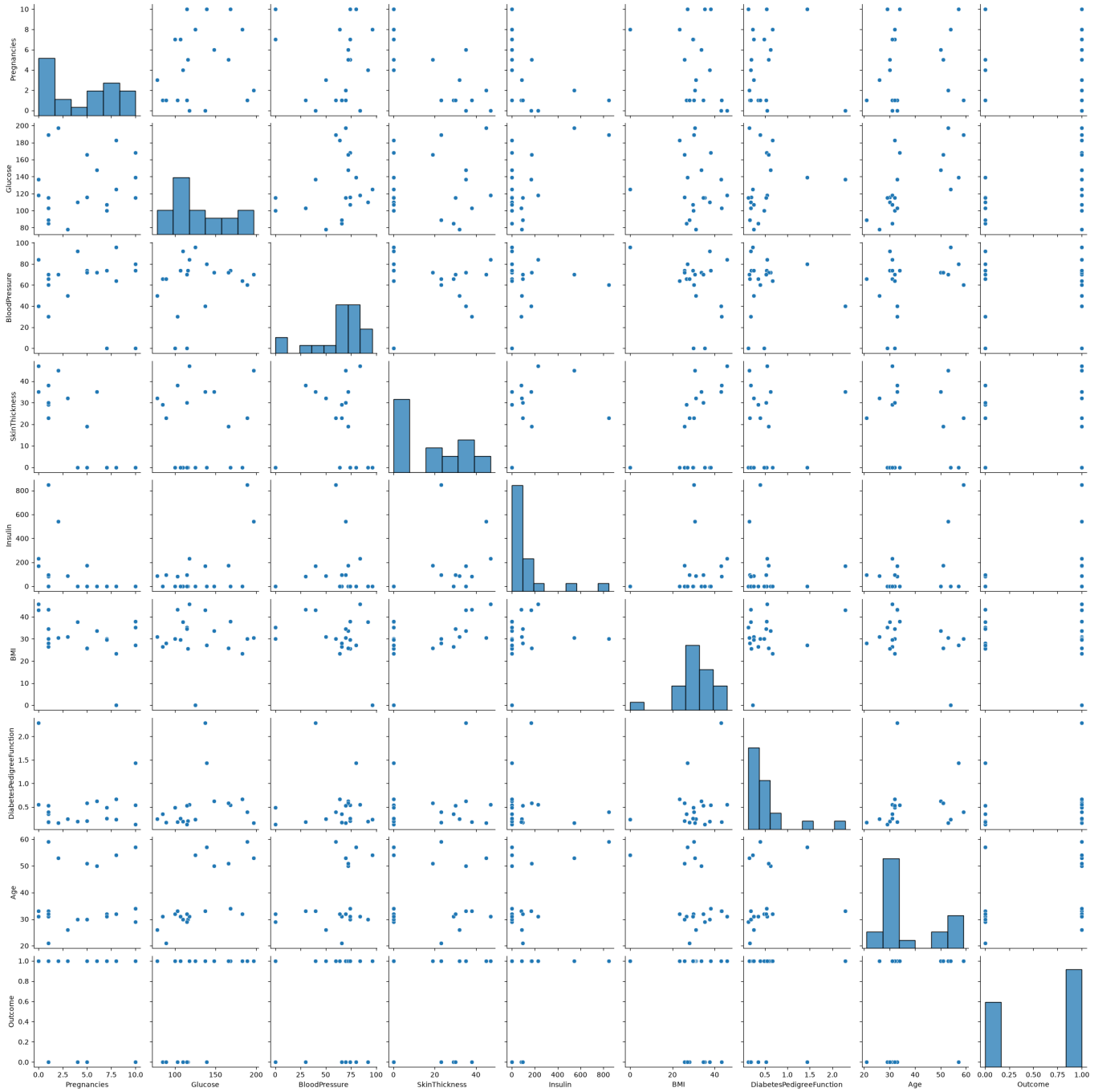
Output



Code Cell 9

```
sns.pairplot(diabetes_df)
plt.show()
```

Output


Pregnancies
Glucose
BloodPressure
SkinThickness
Insulin
BMI
DiabetesPedigreeFunction
Age
Outcome

Experiment 16

Code Cell 1

```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose

data = pd.read_csv("website_traffic.csv", parse_dates=["Date"], index_col="Date")

print(data.head())
```

Output

```
            Page_Views  Unique_Visitors  Bounce_Rate
Date
2026-01-01         512              356        64.59
2026-01-02         499              290        65.93
2026-01-03         583              323        67.24
2026-01-04         526              308        63.33
2026-01-05         534              339        59.14
```

Code Cell 2

```
data = data.interpolate(method="linear")

print(data.isnull().sum())
```

Output

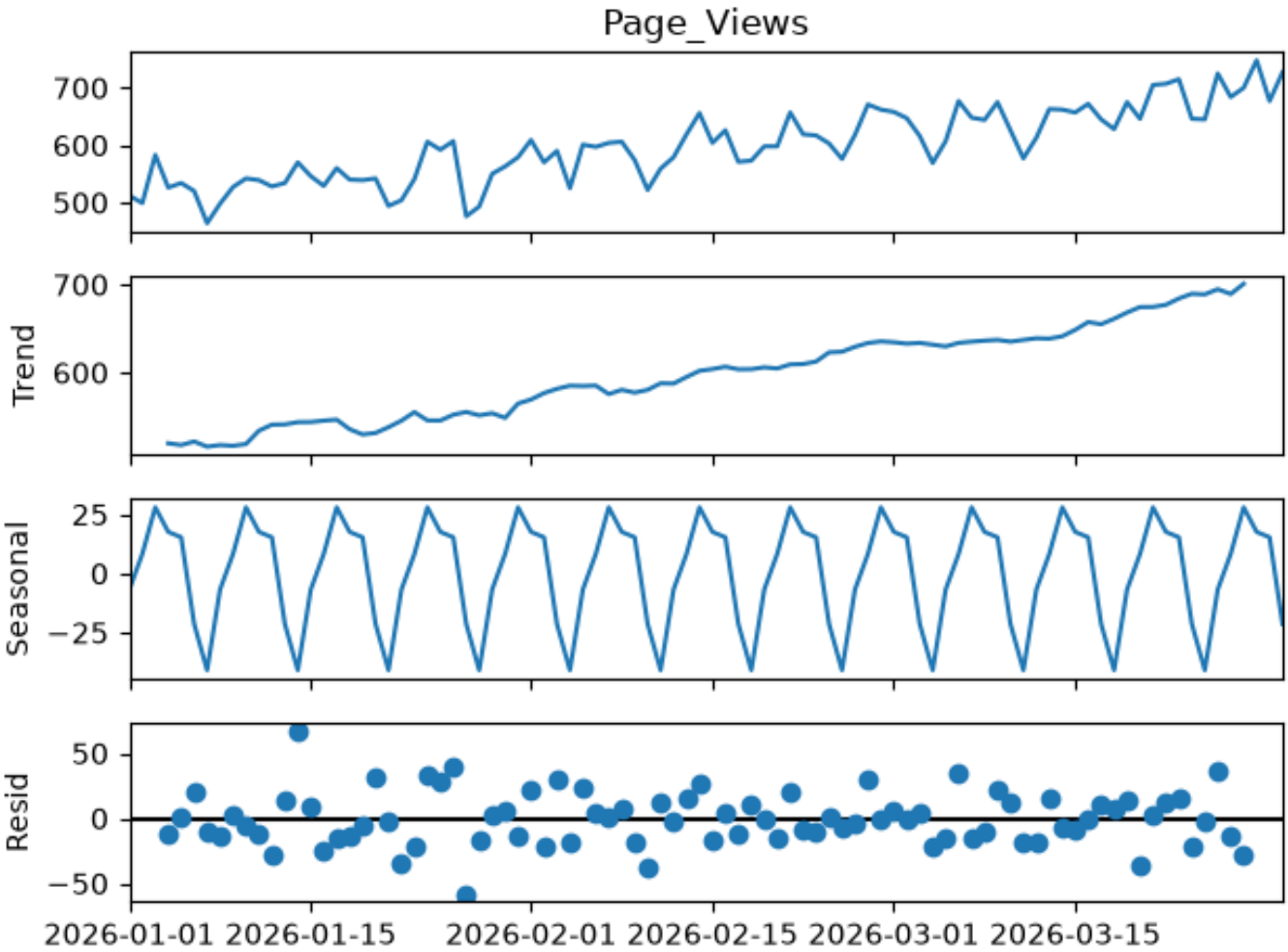
```
Page_Views         0
Unique_Visitors    0
Bounce_Rate        0
dtype: int64
```

Code Cell 3

```
decomposition = seasonal_decompose(data["Page_Views"], model="additive", period=7)

decomposition.plot()
plt.show()
```

Output



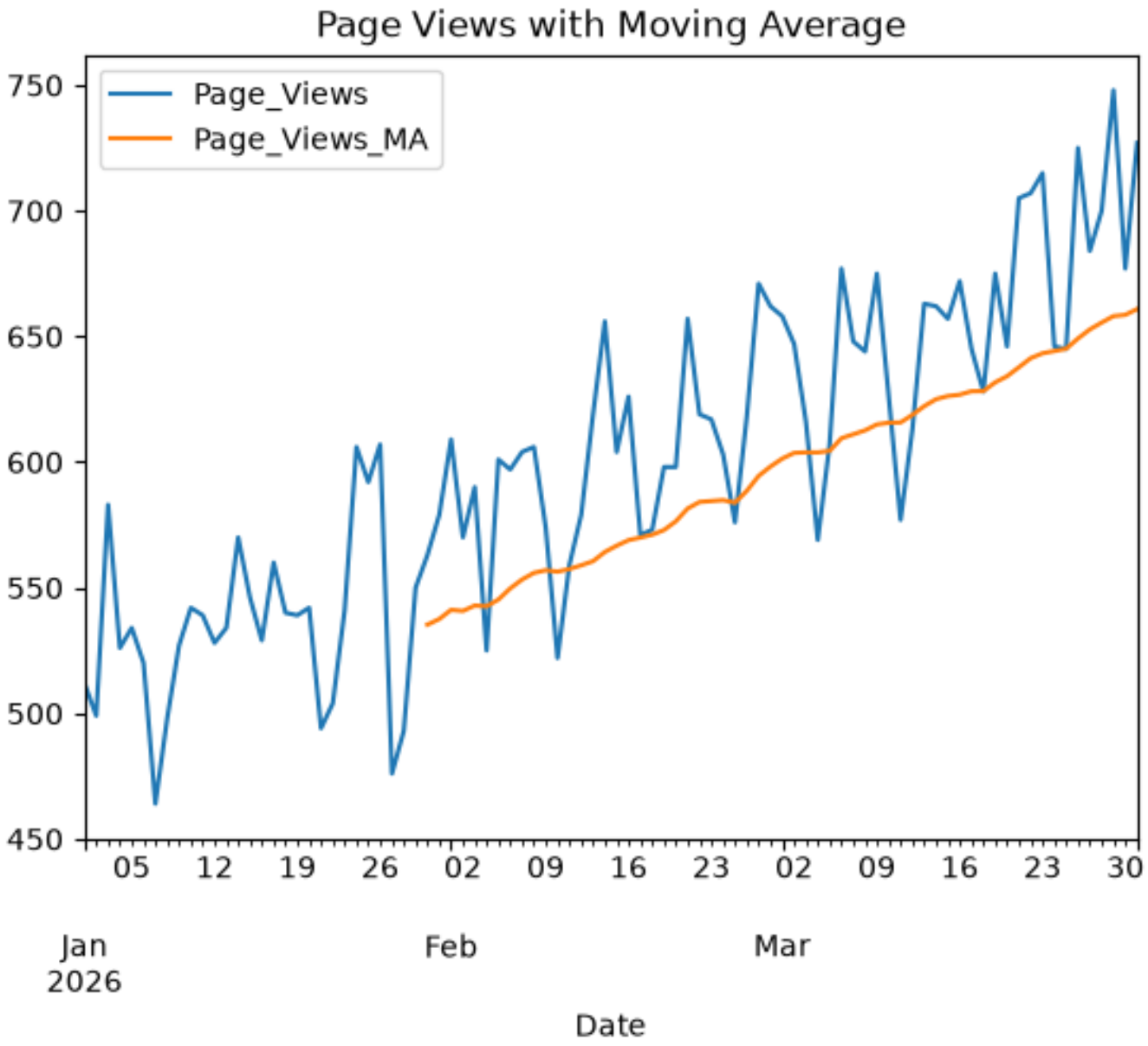
Code Cell 4

```
data["Page_Views_MA"] = data["Page_Views"].rolling(window=30).mean()

data[["Page_Views", "Page_Views_MA"]].plot()

plt.title("Page Views with Moving Average")
plt.show()
```

Output



Code Cell 5

```
data["Page_Views_Z"] = (
    (data["Page_Views"] - data["Page_Views"].mean())
    / data["Page_Views"].std()
)

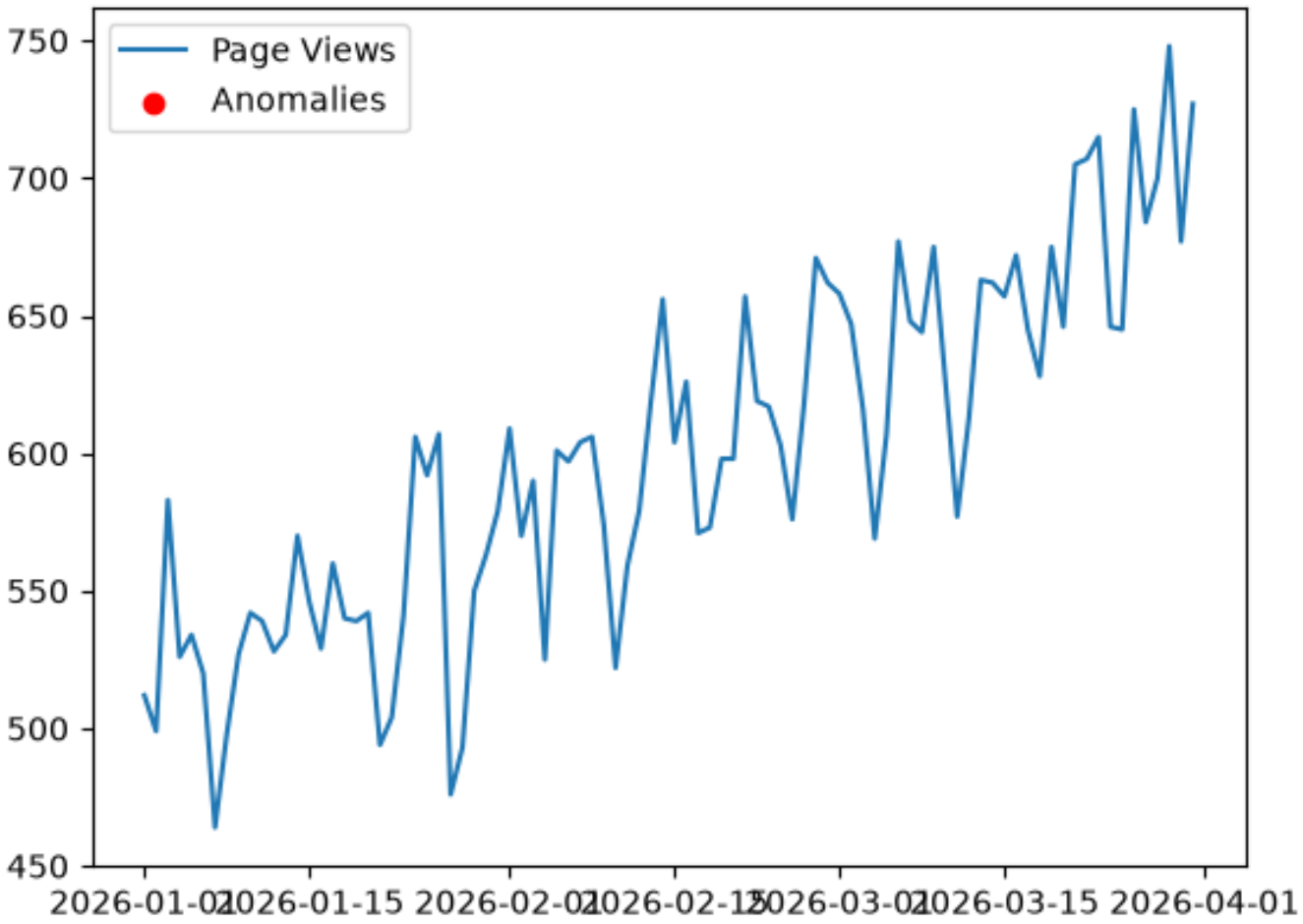
anomalies = data[data["Page_Views_Z"].abs() > 3]

plt.plot(data.index, data["Page_Views"], label="Page Views")
plt.scatter(anomalies.index, anomalies["Page_Views"], color="red", label="Anomalies")

plt.legend()
plt.show()

print(anomalies)
```

Output



Output

```
Empty DataFrame
Columns: [Page_Views, Unique_Visitors, Bounce_Rate, Page_Views_MA, Page_Views_Z]
Index: []
```

Experiment 17

Code Cell 1

```
import numpy as np
import matplotlib.pyplot as plt
```

Code Cell 2

```
population_mean = 50
population_std = 10
population_size = 100000

population = np.random.normal(population_mean, population_std, population_size)

print(population[:10])
```

Output

```
[49.12533246 74.6596986  34.29545046 53.09931966 57.53343135 48.74374867
 53.84722889 55.67916014 38.6108799  52.49300819]
```

Code Cell 3

```
sample_sizes = [30, 50, 100]
num_samples = 1000

sample_means = {}

for size in sample_sizes:
    sample_means[size] = []

    for _ in range(num_samples):
        sample = np.random.choice(population, size=size, replace=False)
        sample_means[size].append(np.mean(sample))

print("Sampling completed successfully")
```

Output

```
Sampling completed successfully
```

Code Cell 4

```
plt.figure(figsize=(12, 8))

for i, size in enumerate(sample_sizes):
    plt.subplot(len(sample_sizes), 1, i + 1)

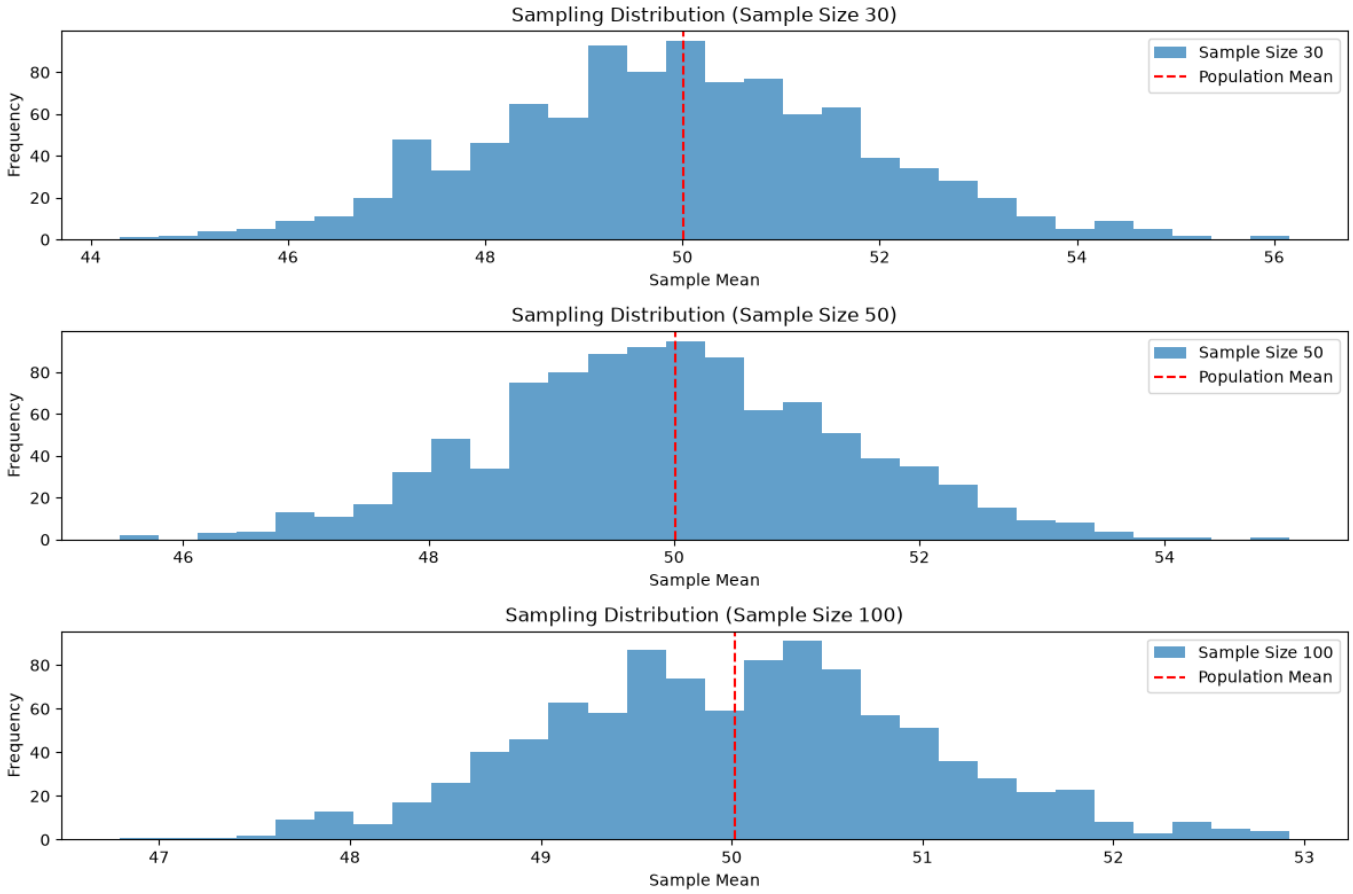
    plt.hist(sample_means[size], bins=30, alpha=0.7,
             label=f'Sample Size {size}')

    plt.axvline(np.mean(population), color='red',
                linestyle='dashed', linewidth=1.5,
                label='Population Mean')

    plt.title(f'Sampling Distribution (Sample Size {size})')
    plt.xlabel('Sample Mean')
    plt.ylabel('Frequency')
    plt.legend()

plt.tight_layout()
plt.show()
```

Output


Sampling Distribution (Sample Size 30)
Sample Size 30
Population Mean
Frequency
Sample Mean
Sampling Distribution (Sample Size 50)
Sample Size 50
Population Mean
Frequency
Sample Mean
Sampling Distribution (Sample Size 100)
Sample Size 100
Population Mean
Frequency
Sample Mean

Experiment 18

Code Cell 1

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

%matplotlib inline
```

Code Cell 2

```
data = pd.read_csv("Iris_Dataset.csv")
data
```

Output

```
      Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm  \
0      1            5.1           3.5            1.4           0.2
1      2            4.9           3.0            1.4           0.2
2      3            4.7           3.2            1.3           0.2
3      4            4.6           3.1            1.5           0.2
4      5            5.0           3.6            1.4           0.2
..   ...            ...           ...            ...           ...
145  146            6.7           3.0            5.2           2.3
146  147            6.3           2.5            5.0           1.9
147  148            6.5           3.0            5.2           2.0
148  149            6.2           3.4            5.4           2.3
149  150            5.9           3.0            5.1           1.8

            variety
0       Iris-setosa
1       Iris-setosa
2       Iris-setosa
3       Iris-setosa
4       Iris-setosa
..              ...
145  Iris-virginica
146  Iris-virginica
147  Iris-virginica
148  Iris-virginica
149  Iris-virginica

[150 rows x 6 columns]
```

Code Cell 3

```
data.info()
```

Output

```
<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Id             150 non-null    int64
 1   SepalLengthCm  150 non-null    float64
 2   SepalWidthCm   150 non-null    float64
 3   PetalLengthCm  150 non-null    float64
 4   PetalWidthCm   150 non-null    float64
 5   variety        150 non-null    str
dtypes: float64(4), int64(1), str(1)
memory usage: 9.1 KB
```

Code Cell 4

```
data.describe()
```

Output

```
               Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm
count  150.000000     150.000000    150.000000     150.000000    150.000000
mean    75.500000       5.843333      3.057333       3.758000      1.199333
std     43.445368       0.828066      0.435866       1.765298      0.762238
min      1.000000       4.300000      2.000000       1.000000      0.100000
25%     38.250000       5.100000      2.800000       1.600000      0.300000
50%     75.500000       5.800000      3.000000       4.350000      1.300000
75%    112.750000       6.400000      3.300000       5.100000      1.800000
max    150.000000       7.900000      4.400000       6.900000      2.500000
```

Code Cell 5

```
data.value_counts("variety")
```

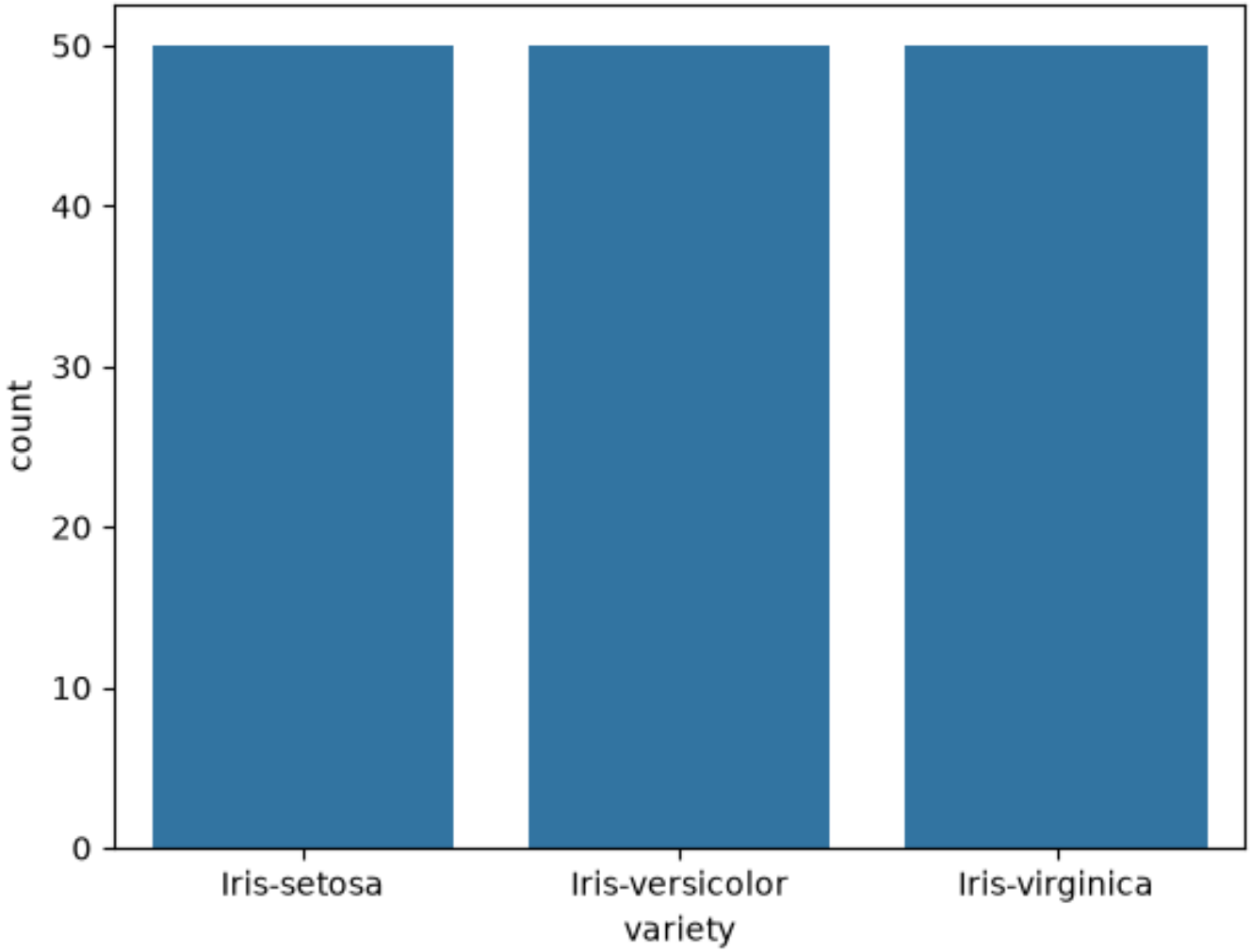
Output

```
variety
Iris-setosa        50
Iris-versicolor    50
Iris-virginica     50
Name: count, dtype: int64
```

Code Cell 6

```
sns.countplot(x="variety", data=data)
plt.show()
```

Output



Code Cell 7

```
dummies = pd.get_dummies(data.variety)
```

Code Cell 8

```
FinalDataset = pd.concat(
    [pd.get_dummies(data.variety), data.iloc[:, [0, 1, 2, 3]]],
    axis=1
)

FinalDataset.head()
```

Output

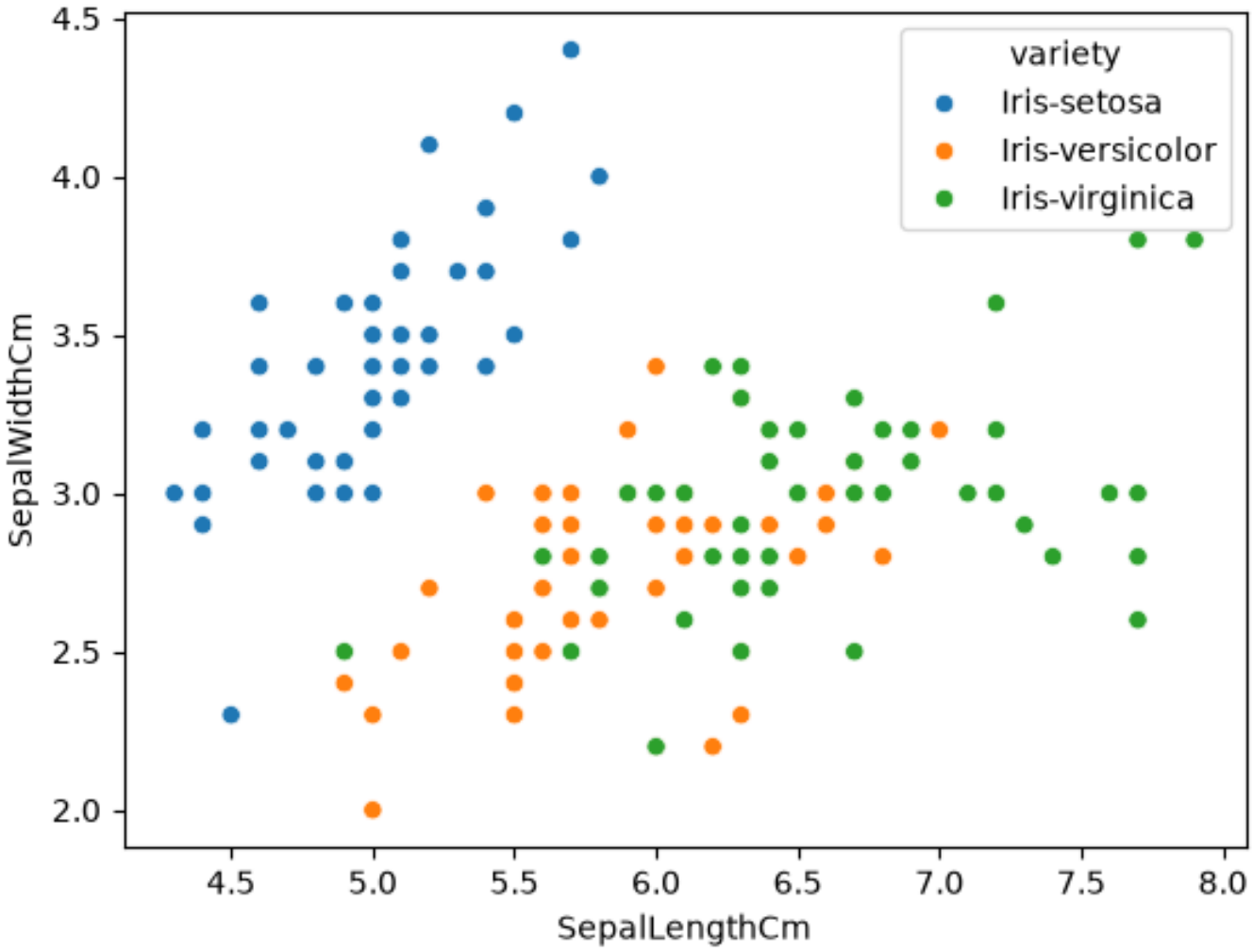
```
   Iris-setosa  Iris-versicolor  Iris-virginica  Id  SepalLengthCm  \
0         True            False           False   1            5.1
1         True            False           False   2            4.9
2         True            False           False   3            4.7
3         True            False           False   4            4.6
4         True            False           False   5            5.0

   SepalWidthCm  PetalLengthCm
0           3.5            1.4
1           3.0            1.4
2           3.2            1.3
3           3.1            1.5
4           3.6            1.4
```

Code Cell 9

```
sns.scatterplot(
    x="SepalLengthCm",
    y="SepalWidthCm",
    hue="variety",
    data=data
)
plt.show()
```

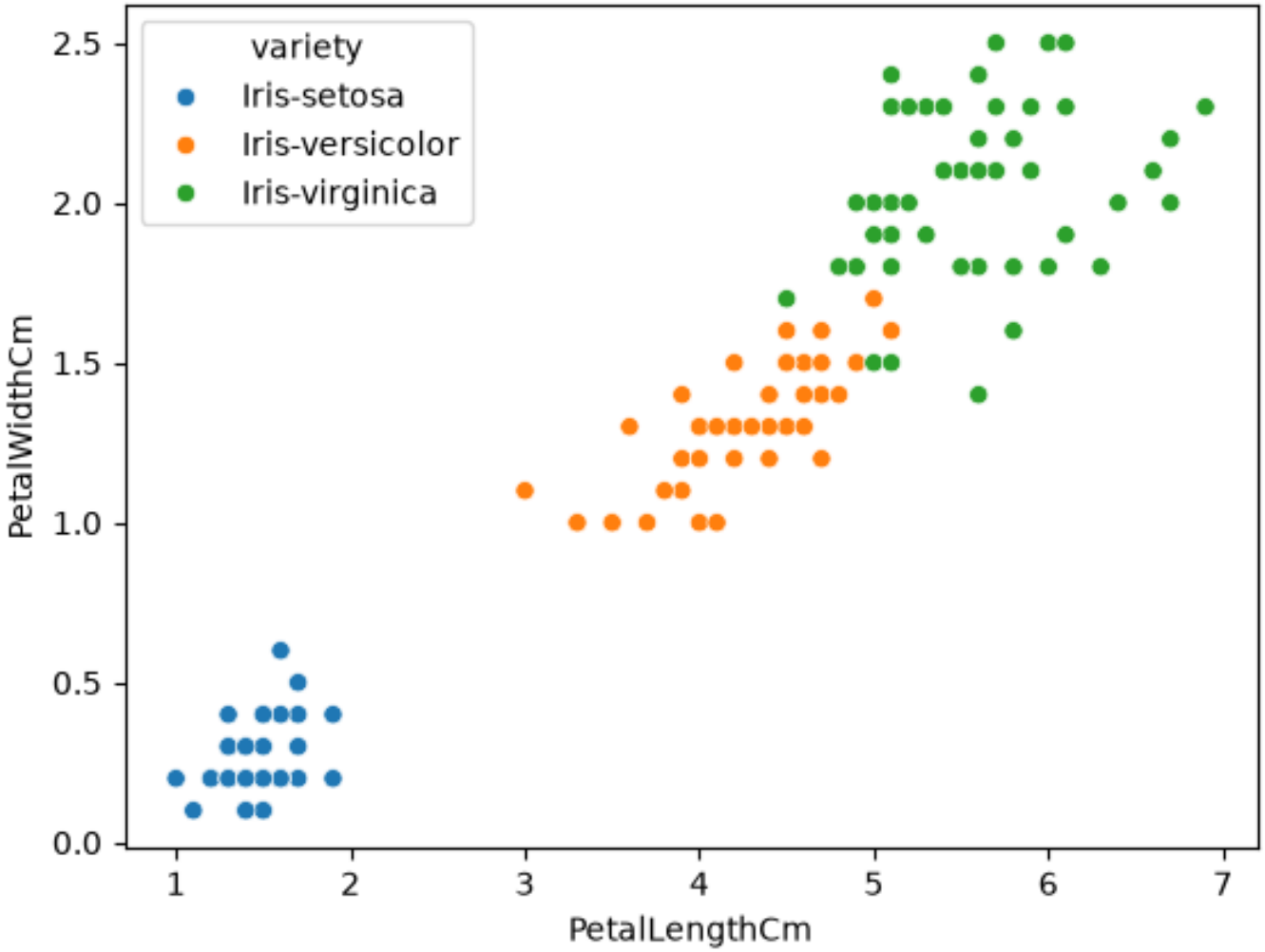
Output



Code Cell 10

```
sns.scatterplot(
    x="PetalLengthCm",
    y="PetalWidthCm",
    hue="variety",
    data=data
)
plt.show()
```

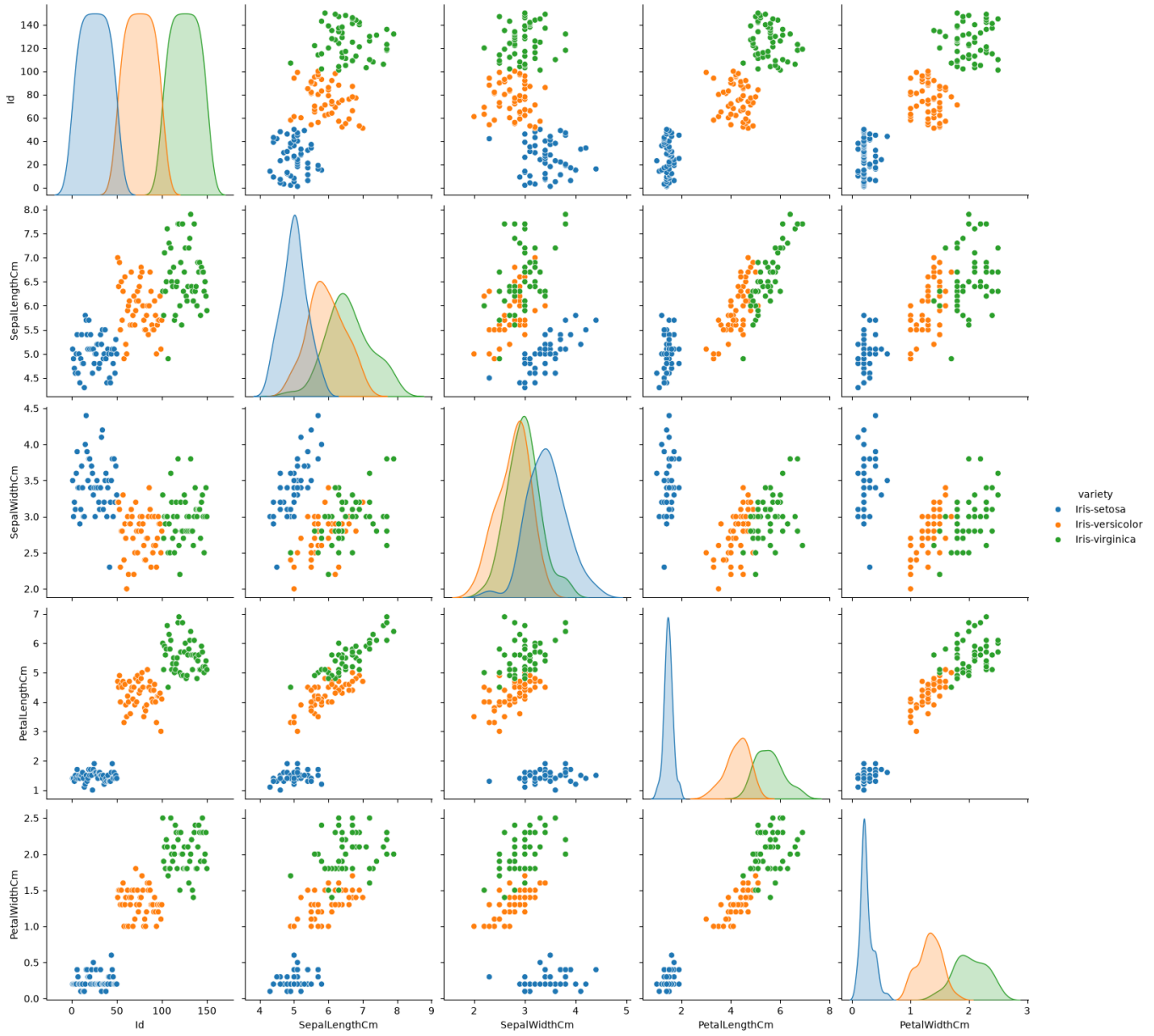
Output



Code Cell 11

```
sns.pairplot(data, hue="variety", height=3)
plt.show()
```

Output

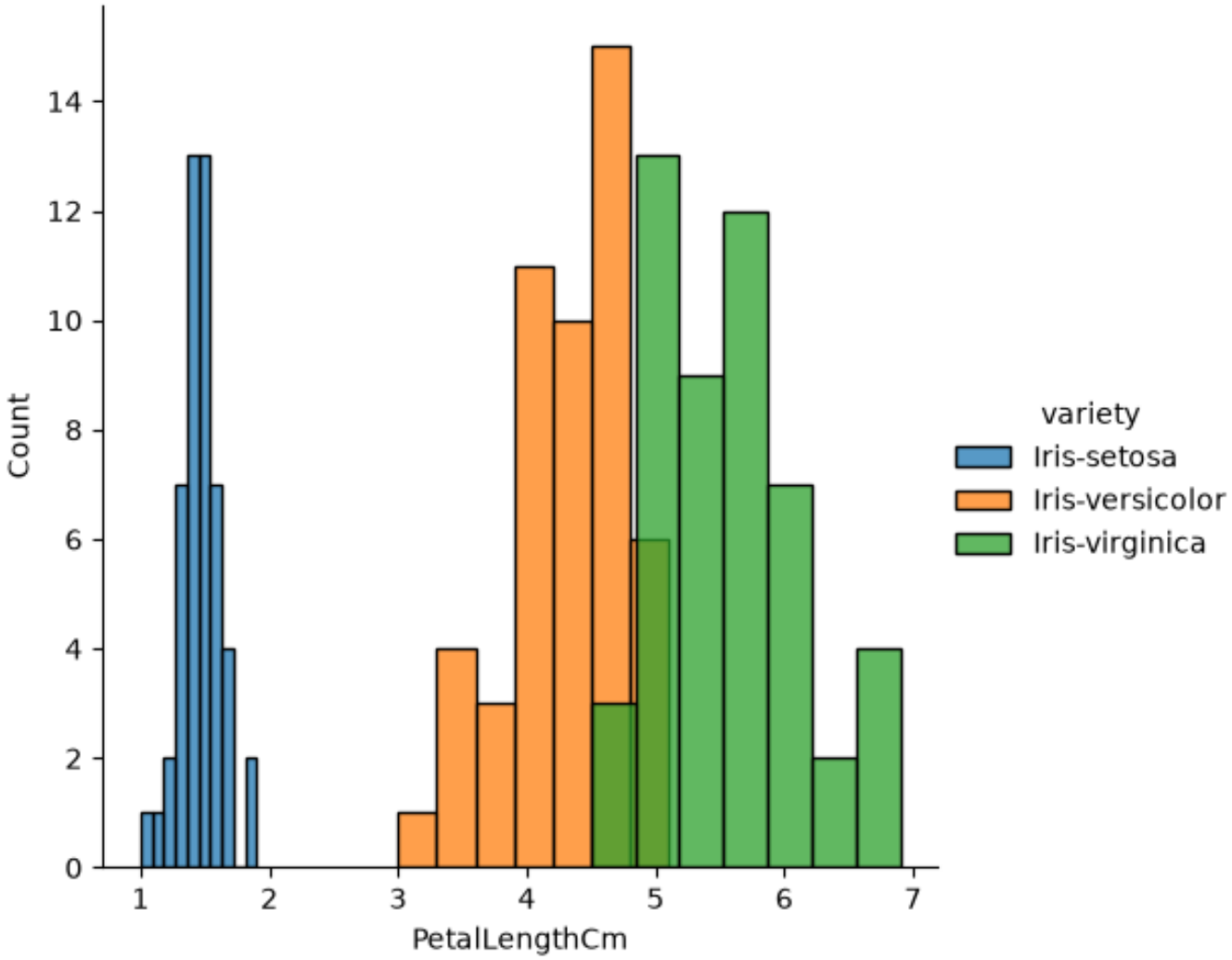


Code Cell 12

```
sns.FacetGrid(data, hue="variety", height=5).map(
    sns.histplot, "PetalLengthCm"
).add_legend()

plt.show()
```

Output

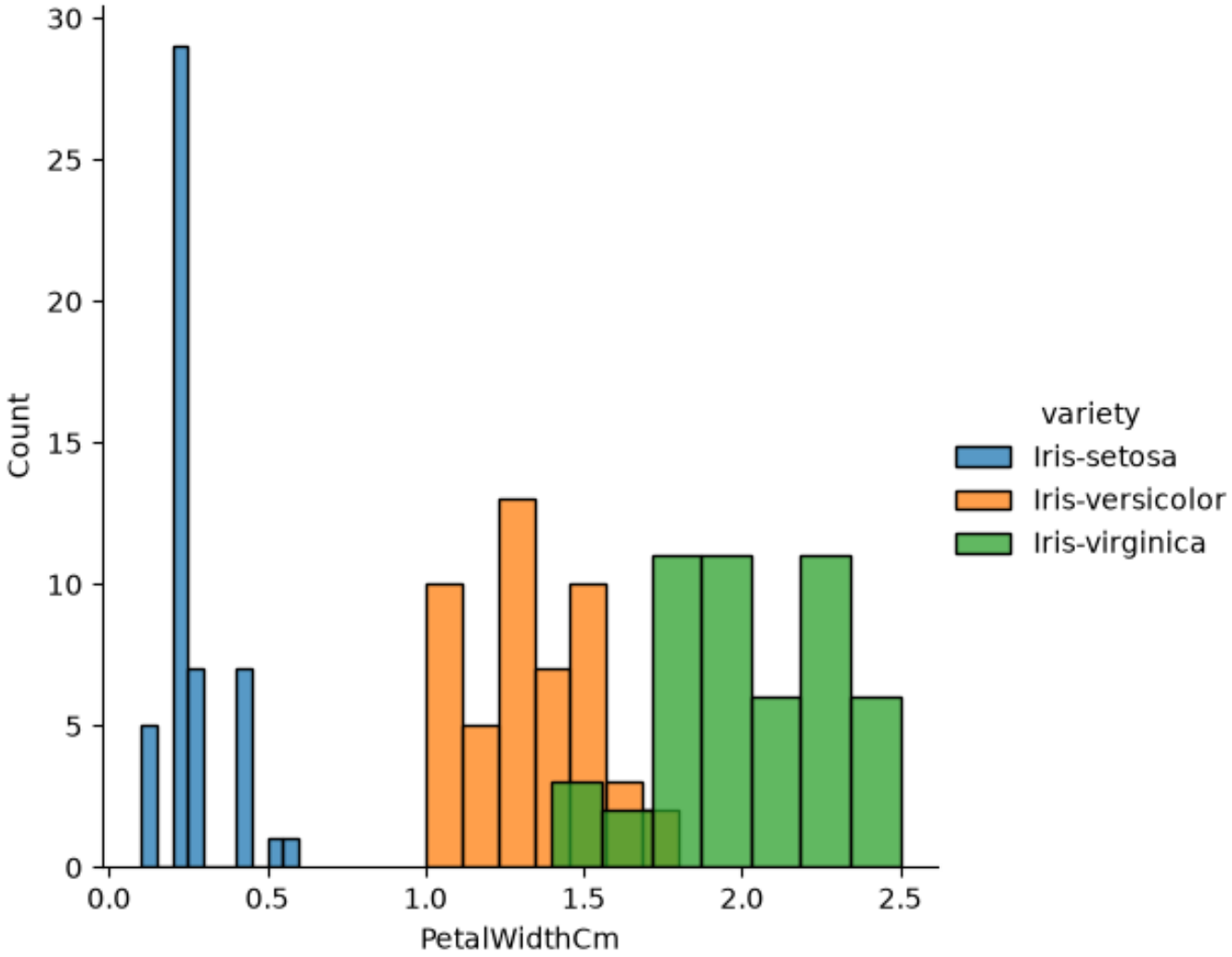


Code Cell 13

```
sns.FacetGrid(data, hue="variety", height=5).map(
    sns.histplot, "PetalWidthCm"
).add_legend()

plt.show()
```

Output

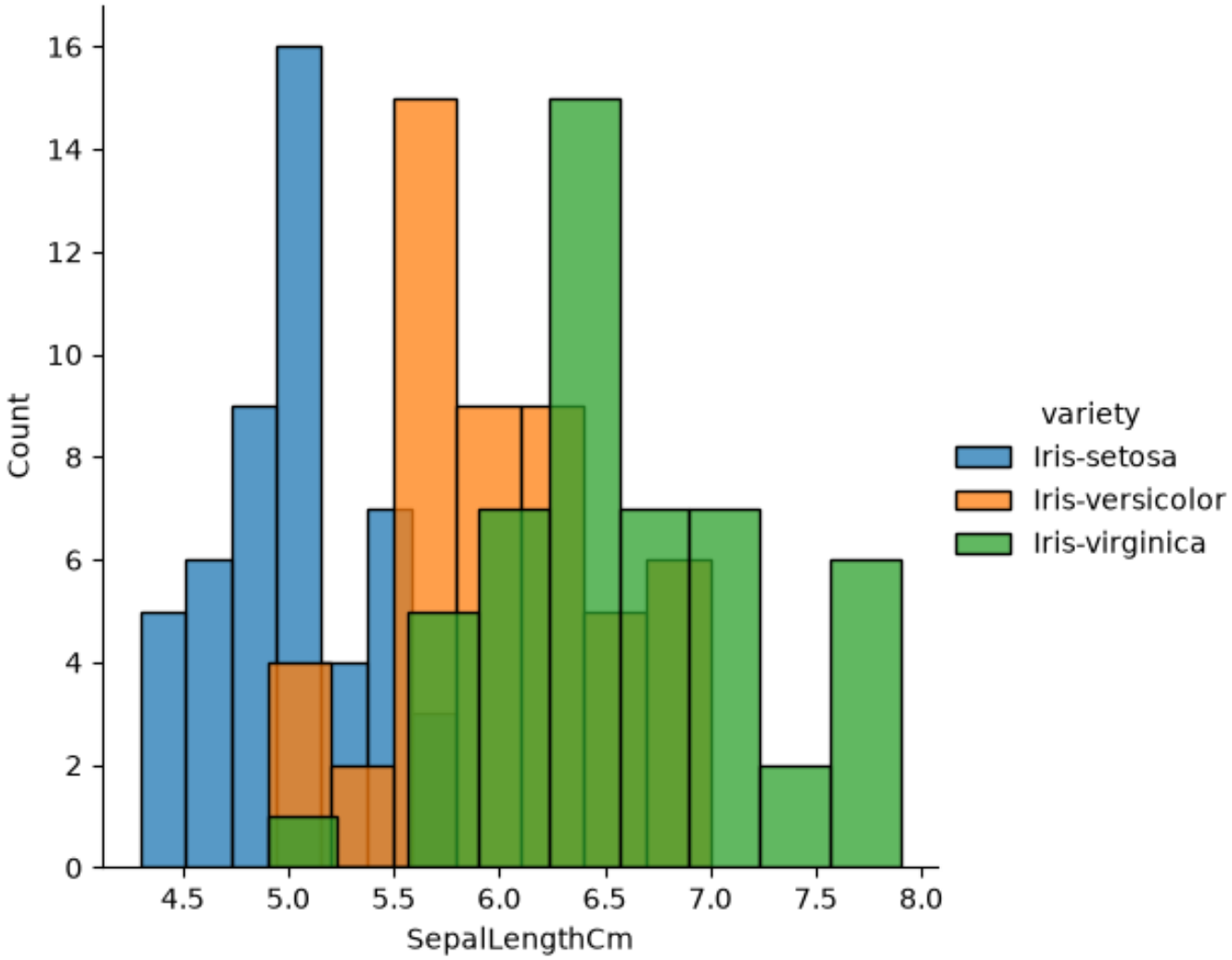


Code Cell 14

```
sns.FacetGrid(data, hue="variety", height=5).map(
    sns.histplot, "SepalLengthCm"
).add_legend()

plt.show()
```

Output

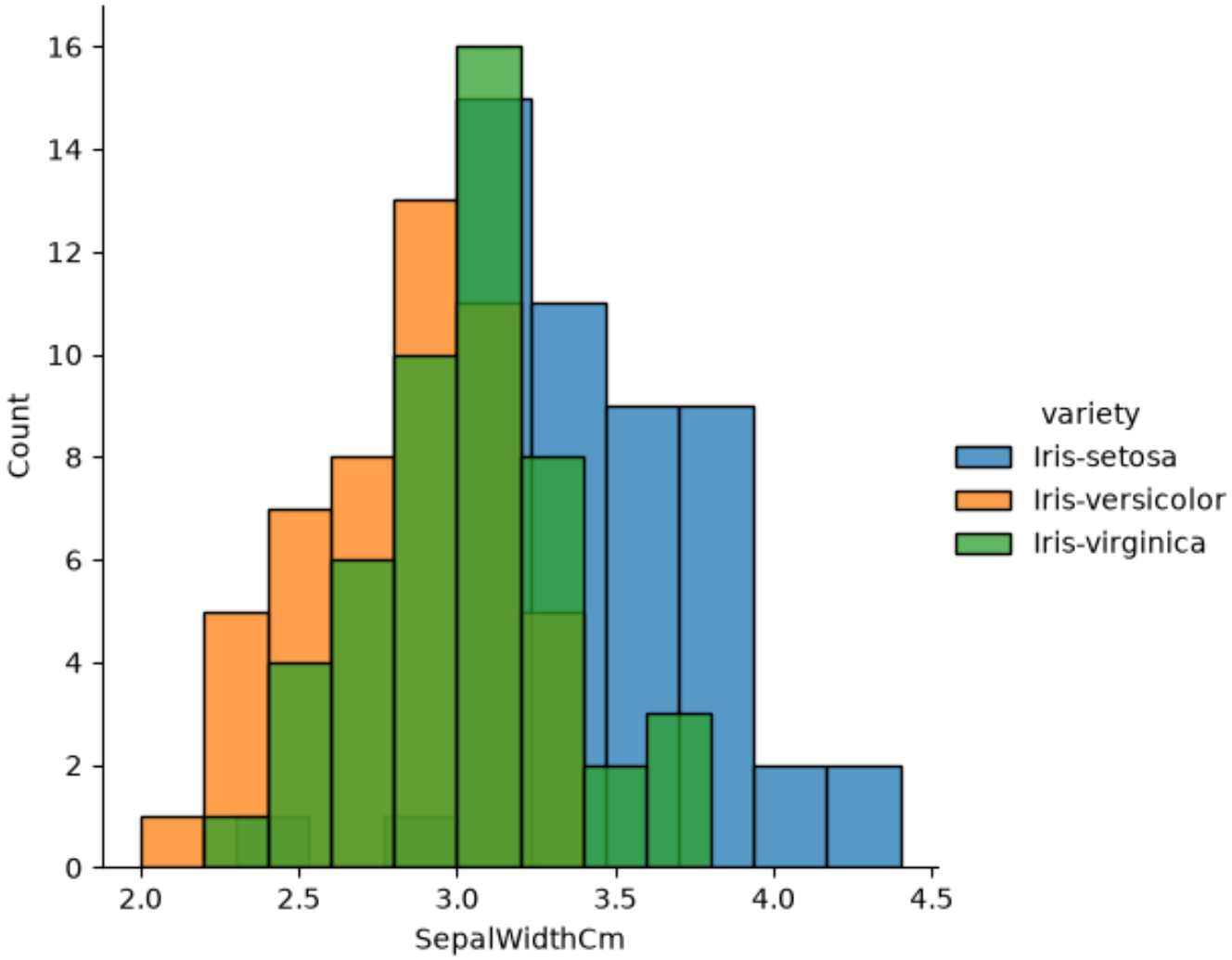


Code Cell 15

```
sns.FacetGrid(data, hue="variety", height=5).map(
    sns.histplot, "SepalWidthCm"
).add_legend()

plt.show()
```

Output


Count
SepalWidthCm
variety
Iris-setosa
Iris-versicolor
Iris-virginica

Experiment 19

Code Cell 1

```
import numpy as np
```

Code Cell 2

```
array = np.random.randint(1, 100, 9)
array
```

Output

```
array([68, 98, 16, 74, 40, 89, 26, 48, 62], dtype=int32)
```

Code Cell 3

```
np.sqrt(array)
```

Output

```
array([8.24621125, 9.89949494, 4.        , 8.60232527, 6.32455532,
       9.43398113, 5.09901951, 6.92820323, 7.87400787])
```

Code Cell 4

```
array.ndim
```

Output

```
1
```

Code Cell 5

```
new_array = array.reshape(3, 3)
new_array
```

Output

```
array([[68, 98, 16],
       [74, 40, 89],
       [26, 48, 62]], dtype=int32)
```

Code Cell 6

```
new_array.ndim
```

Output

```
2
```

Code Cell 7

```
new_array.ravel()
```

Output

```
array([68, 98, 16, 74, 40, 89, 26, 48, 62], dtype=int32)
```

Code Cell 8

```
newm = new_array.reshape(3, 3)
newm
```

Output

```
array([[68, 98, 16],
       [74, 40, 89],
       [26, 48, 62]], dtype=int32)
```

Code Cell 9

```
newm[2, 1:3]
```

Output

```
array([48, 62], dtype=int32)
```

Code Cell 10

```
newm[1:2, 1:3]
```

Output

```
array([[40, 89]], dtype=int32)
```

Code Cell 11

```
new_array[0:3, 0:0]
```

Output

```
array([], shape=(3, 0), dtype=int32)
```

Code Cell 12

```
new_array[0:2, 0:1]
```

Output

```
array([[68],
       [74]], dtype=int32)
```

Code Cell 13

```
new_array[0:3, 0:1]
```

Output

```
array([[68],
       [74],
       [26]], dtype=int32)
```

Code Cell 14

```
new_array[1:3]
```

Output

```
array([[74, 40, 89],
       [26, 48, 62]], dtype=int32)
```

Experiment 20

Code Cell 1

```
import numpy as np
import pandas as pd
```

Code Cell 2

```
data = [
    [1, 'Smith', 50000],
    [2, 'Jones', 60000]
]

df = pd.DataFrame(data)
df
```

Output

```
   0      1      2
0  1  Smith  50000
1  2  Jones  60000
```

Code Cell 3

```
df.columns = ['Empd', 'Name', 'Salary']
df
```

Output

```
   Empd   Name  Salary
0     1  Smith   50000
1     2  Jones   60000
```

Code Cell 4

```
df.info()
```

Output

```
<class 'pandas.DataFrame'>
RangeIndex: 2 entries, 0 to 1
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Empd    2 non-null      int64
 1   Name    2 non-null      str
 2   Salary  2 non-null      int64
dtypes: int64(2), str(1)
memory usage: 190.0 bytes
```

Code Cell 5

```
df = pd.read_csv("employee.csv")
df
```

Output

```
   emp id                  name  salary
0       1      SREE VARSSINI K S    5000
1       2            SREEMATHI B    6000
2       3                SREYA G    7000
3       4   SREYASKARI MULLAPUDI    5000
4       5          SRI AKASH U G    8000
5       6  SRI HARSHAVARDHANAN R    3000
6       7  SRI HARSHAVARDHANAN R    6000
```

Code Cell 6

```
df.head()
```

Output

```
   emp id                  name  salary
0       1     SREE VARSSINI K S    5000
1       2           SREEMATHI B    6000
2       3               SREYA G    7000
3       4  SREYASKARI MULLAPUDI    5000
4       5         SRI AKASH U G    8000
```

Code Cell 7

```
df.tail()
```

Output

```
   emp id                   name  salary
2       3                SREYA G    7000
3       4   SREYASKARI MULLAPUDI    5000
4       5           SRI AKASH U G    8000
5       6  SRI HARSHAVARDHANAN R    3000
6       7  SRI HARSHAVARDHANAN R    6000
```

Code Cell 8

```
df.info()
```

Output

```
<class 'pandas.DataFrame'>
RangeIndex: 7 entries, 0 to 6
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   emp id  7 non-null      int64
 1   name    7 non-null      str
 2   salary  7 non-null      int64
dtypes: int64(2), str(1)
memory usage: 410.0 bytes
```

Code Cell 9

```
df.salary
```

Output

```
0    5000
1    6000
2    7000
3    5000
4    8000
5    3000
6    6000
Name: salary, dtype: int64
```

Code Cell 10

```
type(df.salary)
```

Output

```
pandas.Series
```

Code Cell 11

```
df.salary.mean()
```

Output

```
np.float64(5714.285714285715)
```

Code Cell 12

```
df.salary.median()
```

Output

```
np.float64(6000.0)
```

Code Cell 13

```
df.salary.mode()
```

Output

```
0    5000
1    6000
Name: salary, dtype: int64
```

Code Cell 14

```
df.salary.var()
```

Output

```
np.float64(2571428.5714285714)
```

Code Cell 15

```
df.salary.std()
```

Output

```
np.float64(1603.5674514745463)
```

Code Cell 16

```
df.describe()
```

Output

```
         emp id       salary
count  7.000000     7.000000
mean   4.000000  5714.285714
std    2.160247  1603.567451
min    1.000000  3000.000000
25%    2.500000  5000.000000
50%    4.000000  6000.000000
75%    5.500000  6500.000000
max    7.000000  8000.000000
```

Code Cell 17

```
df.describe(include='all')
```

Output

```
          emp id                    name       salary
count   7.000000                       7     7.000000
unique       NaN                       6          NaN
top          NaN  SRI HARSHAVARDHANAN R          NaN
freq         NaN                       2          NaN
mean    4.000000                     NaN  5714.285714
std     2.160247                     NaN  1603.567451
min     1.000000                     NaN  3000.000000
25%     2.500000                     NaN  5000.000000
50%     4.000000                     NaN  6000.000000
75%     5.500000                     NaN  6500.000000
max     7.000000                     NaN  8000.000000
```

Code Cell 18

```
empCol = df.columns
empCol
```

Output

```
Index(['emp id', 'name', 'salary'], dtype='str')
```

Code Cell 19

```
emparray = df.values
emparray
```

Output

```
array([[1, 'SREE VARSSINI K S', 5000],
       [2, 'SREEMATHI B', 6000],
       [3, 'SREYA G', 7000],
       [4, 'SREYASKARI MULLAPUDI', 5000],
       [5, 'SRI AKASH U G', 8000],
       [6, 'SRI HARSHAVARDHANAN R', 3000],
       [7, 'SRI HARSHAVARDHANAN R', 6000]], dtype=object)
```

Code Cell 20

```
employee_DF = pd.DataFrame(emparray, columns=empCol)
employee_DF
```

Output

```
  emp id                   name salary
0      1      SREE VARSSINI K S   5000
1      2            SREEMATHI B   6000
2      3                SREYA G   7000
3      4   SREYASKARI MULLAPUDI   5000
4      5          SRI AKASH U G   8000
5      6  SRI HARSHAVARDHANAN R   3000
6      7  SRI HARSHAVARDHANAN R   6000
```